Automaten Und Berechenbarkeit Skript

Automaten und Berechenbarkeit (Friedrich-Schiller-Universität Jena)



messages.pdf_cover_qr_code_label

messages.downloaded_by
messages.studocu_not_sponsored_or_endorsed_by_college

Automaten und Berechenbarkeit

Dozentin Dr. Jana Grajetzki

Semester Wintersemester 2022/2023

Dieses Dokument stellt eine studentische Mitschrift während der Veranstaltungen dar.
Zuletzt Aktualisiert: 15. Februar 2023

studocu

messages.pdf_cover_qr_code_label
messages.downloaded_by



Inhaltsverzeichnis

1 Sprachen und Grammatiken **4**
- 1.1 Grundlagen . . . 4
  - 1.1.1 Alphabete und Wörter . . . 4
  - 1.1.2 Formale Sprachen . . . 6
- 1.2 Einführung . . . 7
- 1.3 Die Chomsky-Hierarchie . . . 11

2 Reguläre Sprachen **14**
- 2.1 Endliche Automaten . . . 14
- 2.2 Nichtdeterministische endliche Automaten . . . 22
  - Potenzmengenkonstruktion . . . 26
- 2.3 Reguläre Ausdrücke . . . 29
- 2.4 Das Pumping-Lemma . . . 33
- 2.5 Minimalautomaten und Äquivalenzrelation . . . 35
- 2.6 Abschlusseigenschaften . . . 40

3 Kontextfreie Sprachen **41**
- 3.1 Die Chomsky-Normalform . . . 41
- 3.2 Das Pumping-Lemma für kontextfreie Sprachen . . . 43
- 3.3 Abschlusseigenschaft . . . 46
- 3.4 Kellerautomaten (Push-Down-Automaten) . . . 47
- 3.5 Deterministische kontextfreie Sprachen . . . 54
- 3.6 Das Wortproblem für kontextfreie Sprachen . . . 56

4 kontextsensitiv und L_0-Sprachen **59**
- 4.1 Turingmaschinen . . . 59
- 4.2 linear beschränkte Turingmaschine . . . 62
- 4.3 Grammatiken . . . 65
- 4.4 Das Wortproblem für kontextsensitive Sprachen . . . 66
- 4.5 Abschlusseigentschaften von CS . . . 67

5 Berechenbarkeit **69**
- 5.1 Intuitiver Begriff der Berechenbarkeit . . . 69
- 5.2 Grundlagen . . . 69
- 5.3 Turing-Berechenbarkeit . . . 71
- 5.4 Andere Typen von Turingmaschinen . . . 73
- 5.5 Churchsche These . . . 76
- 5.6 Deterministische und nichtdeterministische TM . . . 76

6 Entscheidbarkeit und Aufzählbarkeit **78**
- 6.1 Entscheidbarkeit . . . 78
- 6.2 Semi- Entscheidbarkeit und Aufzählbarkeit . . . 80

studocu

messages.pdf_cover_qr_code_label
messages.downloaded_by



6.3 Beziehungen zwischen REC und RE 83
6.4 Kodieren von Turingmaschinen über 0,1 83
6.5 Das Halteproblem 85
6.5.1 Problemstellung und intuitive Argumentation 85
6.5.2 Formale Argumentation mittels Turingmaschinen 87
6.6 Reduzierbarkeit 88

7 NP-Vollständigkeit **94**
7.1 Die Klasse P 95
7.2 Die Klasse NP 97
7.3 NP-Vollständigkeit 98

Abkürzungen **104**

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by



1 Sprachen und Grammatiken

1.1 Grundlagen

1.1.1 Alphabete und Wörter

Definition 1.1.1.1

Ein Alphabet Σ ist ein endliche nichtleere Menge. Die Elemente von Σ heißen Buchstaben

Beispiel

$$\Sigma = \{a, b, c, \ldots, x, y, z\}$$

$$\Sigma_{latein} = \{a, b, \ldots, z\}$$

$$\Sigma_{griech} = \{\alpha, \beta, \ldots, \omega\}$$

$$\Sigma_{decimal} = \{0, \ldots, 9\}$$

$$\Sigma_{tastatur} = \{a, b, c, \ldots, z\} \cup \{A, B, \ldots, Z\} \cup \{,, \ldots\} \cup \{0, \ldots, 9\} \cup \{!, ?, -, \ldots\}$$

$$\Sigma = \{*, \circ, \Delta\}$$

$$\Sigma = a, b$$

Definition 1.1.1.2

Es sei $\Sigma = \{a_1, \ldots, a_n\}$ ein Alphabete. Ein Wort w ist ein endliche Folge von Buchstaben aus Σ, etwa $w = a_{i1}, a_{i2}, \ldots, a_{in}, a_{ij} \in \Sigma$ Die Länge $|w|$ dieser Wörter ist n - die Anzahl seiner Buchstaben.
λ sei das eindeutig bestimmte Wort der Länge 0

Definition 1.1.1.3

Es sei Σ ein Alphabet, so ist

$$\Sigma^0 =_{def} \{\lambda\}$$

$$\Sigma^{i+1} =_{def} \Sigma^i\Sigma =_{def} \{wa | w \in \Sigma^i, a \in \Sigma\}$$

$$\Sigma^* =_{def} \bigcup_{i=0}^{\infty} \Sigma^i$$

$$\Sigma^1 = \Sigma^0\Sigma = \{\lambda\}\,\Sigma = \{wa | w \in \{\lambda\}\,, a \in \Sigma\}$$

$$\Sigma^2 = \Sigma \cdot \Sigma$$

Σ^* ... Menge aller endlichen Folgen über Σ, d.h. Menge aller Wörter über Σ
Σ^i ... Menge aller Wörter der Länge i über Σ

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beispiel

$$\Sigma = \{a, b\}$$

$\Sigma^* = \{a, b\}^*$ ... Menge aller endlichen Zeichenketten aus a's und b's und λ

$\Sigma = \{0, 1, \#\}$ ... Die Adjazenzmatrix eines Graphen ist als Wort über $\{0, 1, \#\}$ darstellbar

Wörter der Umgebungssprache sind auch formale Wörter über $\Sigma_{latein} \vee \{\ddot{a}, \ddot{A}, \ldots\}$, aber auch Sätze, Texte und Romane sind formale Wörter über $\Sigma_{tastatur}$. Alle möglichen Daten, (z.B. Eingaben von Algorithmen) und Informationen können als formale Wörter dargestellt werden.

Operation auf Wörter

Für zwei Wörter $u = x_1 x_2 \ldots x_n$ und $v = y_1 y_2 \ldots y_k$ mit $x_i, y_i \in \Sigma$
ist $u \circ v =_{def} x_1 x_2 \ldots x_n y_1 y_2 \ldots y_k$ die Kombination.
Schreibweise $w = uv$ (ohne $\circ$)
Für Wörter $w = x_1 x_2 \ldots x_n, x_i \in \Sigma$ ist das **Spiegelwort** von w.
$Sp(w) =_{def} x_n x_{n-1} \ldots x_1 =_{def} w^R$.

Potenzen von Wörtern

Es sei $w = x_1 \ldots x_n \in \Sigma^*$ ein Wort der Länge n.
$w^0 =_{def} \lambda$ $\quad (w^1 = w^0 \circ w = \lambda \circ w = w)$
$w^{k+1} =_{def} w^k \circ w$ $\quad w^2 = w \circ w$

Beispiel

$w = abb$ $\quad w^3 = (abb)^3 = abbabbabb$

Relationen über Wörter (über Σ^*)

$T \subseteq \Sigma^* \times \Sigma^*$ ... Teilwortrelation
$(u, v) \subseteq T \leftrightarrow_{def} \bigvee_{l,r \subseteq \Sigma^*} v = l\,u\,r$ „u ist Teilwort von v"
$P \subseteq \Sigma^* \times \Sigma^*$ ... Präfixrelation
$(u, v) \subseteq P \leftrightarrow_{def} \bigvee_{w \subseteq \Sigma^*} u\,w = v$ „u ist Präfix von v"

Beispiel

(PAPA, PAPAYABAUM) $\in P$

Behauptung: T und P sind reflexiv, transitiv, antisymmetrisch

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

1.1.2 Formale Sprachen

Definition 1.1.2.1

Jede $A \subseteq \Sigma^*$ heißt formale Sprache über Σ

Beispiel

$\Sigma = \{a, b\}$ $\quad A_1 = \{a, abb, ba\}$

$A_2 = \{w \in \{a, b\}^* \,|\text{w beginnt mit a}\}$

$\Sigma = \{0, 1\}$ $\quad L_{binär} = \left\{ w \in \{0,1\}^* \,|\, \bigvee_{u \in \{0,1\}} w = 1u \right\} \cup \{0\}$

$\Sigma = \{a, b, c, \wedge, \vee, \neg, \Leftrightarrow, \Rightarrow\}$

A ... Menge aller aussagenlogischen Ausdrücke in den Atomen a,b,c
$a \wedge b \in A$
$\Leftrightarrow\Leftrightarrow \vee \notin A$
Menge von Worten bestimmter Eigenschaften bilden formale Sprachen

Operationen auf Sprachen

Formale Sprachen sind Mengen
$\hookrightarrow$ Alle Mengenoperationen sind für Sprachen definiert.
$l_1 \subseteq \Sigma_1^*$ $l_2 \subseteq \Sigma_2^*$ und somit $l_1, l_2 \subseteq \Sigma^*$ mit $\Sigma = \Sigma_1 \cup \Sigma_2$
Dann sind auch $l_1 \cup l_2$
$l_1 \cap l_2$
$l_1 \setminus l_2$
$\bar{l_1} = \Sigma_1 \setminus l_1 = \{w \in \Sigma_1^* | w \notin l_1\}$
$l_1 \times l_2$ formale Sprachen

Definition 1.1.2.2

Es seien $l_1 \subseteq \Sigma_1^*, l_2 \subseteq \Sigma_2^*$ dann ist $l_1 \circ l_2 = \{xy | x \in l_1, y \in l_2\}$ die Verkettung (Kombination) der Sprachen l_1 und l_2

Definition 1.1.2.3

Es sei $L \subseteq \Sigma^*$. Dann ist

$$L^0 =_{def} \{\lambda\}$$

$$L^{m+1} =_{def} L^m \circ L = L^m \cdot L = \{uv \in \Sigma^* | u \in L^m, v \in L\}$$

und $L^* = \bigcup_{m \in \mathbb{N}} L^m$ die Kleene-Hülle von L.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Bemerkung:

$$\begin{aligned} L^1 = L^0 L = \{\lambda\} \cdot L &= \{uv | u \in \lambda, v \in L\} \\ &= \{uv | u = \lambda, v \in L\} \\ &= \{\lambda v | v \in L\} = L \end{aligned}$$

Beispiel

$$\Sigma = \{a, b\} \qquad A = \{a, ba\} \qquad B = \{aa, ab, b\}$$

$$A \cdot B = \{aaa, aab, ab, baaa, baab, aab\} \qquad aa \notin A \cdot B$$

$\Sigma = \{a\}$ $A = \{aa\}$ $A^* = \{aa\}^*$ ... Menge aller Wörter über $\{a\}$ die gerade Länge haben

$$\Sigma = \{a, b, f, m, p, u\}$$

$$A = \{ba, umpf\}$$

$$A_2 = \{baba, baumpf, umpfba, umpfumpf\}$$

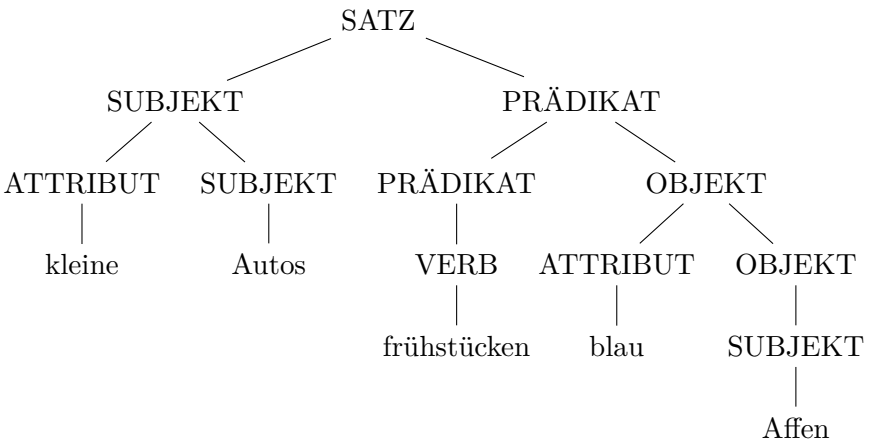
$a\,\{a, b\}^* \cup \{a, b\}^*\, b$ ... Menge aller Wörter die mit a beginnen oder b enden

1.2 Einführung

Natürliche Sprachen durch Grammatikregeln beschreiben, die korrekte Sätze erzeugen.

Ein Versuch:

SATZ	⇒ SUBJEKT_PRÄDIKAT	SUBJEKT	⇒ Affen
SUBJEKT	⇒ ATTRIBUT_SUBJEKT	SUBJEKT	⇒ Menschen
PRÄDIKAT	⇒ PRÄDIKAT_OBJEKT	SUBJEKT	⇒ Autos
PRÄDIKAT	⇒ VERB	VERB	⇒ schlafen
OBJEKT	⇒ ATTRIBUT_OBJEKT	VERB	⇒ frühstücken
OBJEKT	⇒ SUBJEKT	ATTRIBUT	⇒ klein
		ATTRIBUT	⇒ blau



studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

$\Rightarrow$ Regeln mit deren Hilfe genau die Wörter der Sprache erzeugt werden können
$\hookrightarrow$ formale Grammatik

Die Erzeugung einer Sprache durch Ableitungsregeln bedarf folgende Dinge:

1) Symbole der Sprache (Buchstaben)
2) Hilfssymbole
3) Startsymbol
4) Ableitungsregeln

Definition 1.2.1

Ein 4-Tupel $G = (\Sigma, N, S, R)$ heißt Grammatik $\leftrightarrow_{def}$

1) Σ ist eine endliche Menge ... Menge der Buchstaben oder terminale Symbole
2) N ist eine endliche Menge ... Hilfssymbole oder nichtterminale Symbole
3) $S \in N$ ... Startsymbol
4) $R \subseteq (\Sigma \cup N)^* N (\Sigma \cup N)^* \times (\Sigma \cup N)^*$ ... Regelmenge mindestens ein Nichtterminal auf der linken Seite

$(p, q) \in R$ heißen Regeln, auch $p \Rightarrow q$

Beispiel : G_1

$$G_1 = (\{a, b\}, \{S, X, Y, Z\}, S, R_1)$$

$$R_1 = \{S \Rightarrow SX, S \Rightarrow bY, Y \Rightarrow aZ, Z \Rightarrow bY, Z \Rightarrow b\}$$

Beispiel

$\Sigma =$ Autos, Affen, klein, blau, frühstücken, schlafen, ...
$N =$ SATZ, SUBJEKT, PRÄDIKAT, ...
$S =$ SATZ
$R =$ SATZ $\Rightarrow$ SUBJEKT_PRÄDIKAT, ...

(siehe Einfürhungsbeispiel)

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Definition 1.2.2

Es sei Tupel $G = (\Sigma, N, S, R)$ eine Grammatik w' heißt unmittelbare Ableitung von w bezüglich G

$w \vdash_G w' \Leftrightarrow$ 1) $w, w' \in (\Sigma \cup N)^*$

$$2) \bigvee_{p_1} \bigvee_{p_2} \bigvee_{p} \bigvee_{q} p_1, p_2, p, q \in (\Sigma \cup N)^*, (p, q) \in R, w = p_1 p p_2, w' = p_1 q p_2$$

w' heißt Ableitung von w bezüglich G

$w \vdash_G^* w' \leftrightarrow_{def}$

1) $w = w'$ oder

2) $\bigvee_{n \in \mathbb{N}} \bigvee_{w_0} \bigvee_{w_1} \dots \bigvee_{w_n} w_0 = w, w_n = w', w_0 \vdash_G w_1, w_1 \vdash_G w_2, \dots, w_{n-1} \vdash_G w_n$

Schreibweise: $w \vdash_G^n w'$ ... Ableitung der Länge n
$\vdash_G, \vdash_G^*$ sind binäre Relationen $\vdash_G \subseteq (\Sigma \cup N)^* \times (\Sigma \cup N)^*$
$\vdash_G^*$ ist die reflexive und transitiv Hülle von $\vdash_G$

Beispiel

$$S \vdash SX \vdash SXX \vdash bYXX$$
$$S \vdash bY \vdash baZ \vdash bab$$
$$S \vdash bY \vdash baZ \vdash babY \vdash babaZ \vdash babab$$

(siehe Beispiel: G_1)

Wir wollen Sprachen erzeugen, d.h. am Ende sollen keine Hilfssymbole mehr da sein.

Definition 1.2.3

Es sei Tupel $G = (\Sigma, N, S, R)$ eine Grammatik. $L = \{w \in \Sigma^* | S \vdash_G^* w\}$ heißt die von G erzeugte Sprache. Schreibweise $\mathcal{L}(G)$

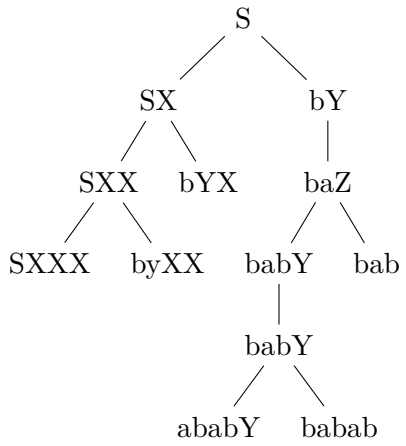
Beispiel

X werden wir nicht mehr los
$\hookrightarrow$ bleibt $S \Rightarrow bY, Y \Rightarrow aZ, Z \Rightarrow bY, Z \Rightarrow b$
$\Rightarrow$ es entstehen Wörter der Form $b\,(ab)^n \quad n \geq 1$
Alle Wörter dieser Form sind ableitbar, alle anderen nicht.
$\hookrightarrow \mathcal{L}(G_1) = \{b\,(ab)^n \,| n \geq 1\}$ (siehe Beispiel: G_1)

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

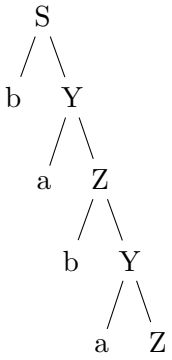
$B = \{w \in \{a,b\}^* \,|\text{w endet auf a}\}$
$G = (\{a,b\}, \{S\}, S, R)$
$R = \{S \Rightarrow a|aS|bS\}$

Ableitungsbaum einer Grammatik $G = (\Sigma, N, S, R)$ am Beispiel von G_1



- Startsymbol ist die Wurzel
- Zeichenketten w' mit $w \vdash_G w'$ sind Knoten
- Kinder eines Knotens sind genau die Zeichenketten w' mit $w \vdash_G w'$
- Wörter aus $\mathcal{L}(G_1)$ sind Blätter
- unendlich lange Pfade sind möglich

Syntaxbaum für ein Wort nur möglich, wenn für alle Regeln $p \Rightarrow q \quad p \in N$ gilt.
Syntaxbaum für babab



Blätter von links nach rechts gelesen ergeben w

Zu jeder Grammatik gibt es eine eindeutig bestimmte Sprache. Die Umkehrung ist falsch.

Beispiel

$$G_1' = (\{a,b\}, \{S,Y,Z\}, S, R_1')$$
$$R_1' = \{S \Rightarrow bY, Y \Rightarrow aZ, Z \Rightarrow bY|b\}$$
erzeugt ebenfalls $\mathcal{L}(G_1)$

Definition 1.2.4

Zwei Grammatiken G_1, G_2 heißten äquivalent, $G_1 \sim G_2 \leftrightarrow_{def} \mathcal{L}(G_1) = \mathcal{L}(G_2)$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

1.3 Die Chomsky-Hierarchie

Definition 1.3.1

Sei $G = (\Sigma, N, S, R)$ eine Grammatik.

1) G heißt Typ 0 Grammatik
   $aA \Rightarrow aa$

2) G heißt kontextsensitiv (nicht verkürzend, Typ 1), wenn für alle Regeln $(u, v) \in R$ gilt $|u| \leq |v|$
   $BB \Rightarrow b$ ist nicht erlaubt

3) G heißt kontextfrei (Typ 2), wenn G kontextsensitiv ist und für alle Regeln $(u, v) \in R$ gilt $u \in N$
   $A \Rightarrow aA$
   $A \Rightarrow aBa \mid aX$

4) G heißt regulär (rechtslinear oder Typ 3), wenn G kontextfrei ist und falls für alle $(u, v) \in R$ gilt $v \in \Sigma$ oder $v \in \Sigma \cdot N$
   nur Regeln: $A \Rightarrow b$
   $A \Rightarrow bB$

Definition 1.3.2

Eine Sprache $L \subseteq \Sigma^*$ heißt Typ 0 (1, 2, 3), falls es eine Typ 0 (1, 2, 3) – Grammatik gibt, mit $\mathcal{L}(G) = L$. Eine Sprache heißt erzeugbar von einer Grammatik, falls es eine Grammatik gibt, die die Sprache erzeugt.

Sprachfamilien

$\mathcal{L}_0$ ... Klasse der von einer Grammatik erzeugbaren Sprache

$\mathcal{L}_1 = \text{CS}$ ... Klasse der kontextsensitiven Sprachen

$\mathcal{L}_2 = \text{CF}$ ... Klasse der kontextfreien Sprachen

$\mathcal{L}_3 = \text{REG}$ ... Klasse der regulären Sprachen

SATZ 1.3.3

$$\text{REG} \subseteq \text{CF} \subseteq \text{CS} \subseteq \mathcal{L}_0$$

Beweis: Folgt unmittelbare aus Definition 1.3.1
Jede Grammatik vom Typ i, ist auch Grammatik vom Typ $i + 1$, $0 \leq i \leq 2$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

kontextfrei:
$L = \{a^n b^n | n \geq 1\}$
$G = (\{a, b\}, \{S\}, S, R)$
$R = \{S \Rightarrow aSb | ab\}$
$S \vdash aSb \vdash aaSbb \vdash aaaSbbb \vdash a^4 b^4$

$L_2 = \{w \in \{a, b\}^* | \#_a(w) = \#_b(w), |w| \geq 2\}$ $\quad \#_t(w)$ ... Anzahl der Terminalzeichen t im Wort w
$G_2 = (\{a, b\}, \{S, A, B\}, S, R_2)$
$R_2 = \{S \Rightarrow ABS | AB, AB \Rightarrow BA, BA \Rightarrow AB, A \Rightarrow a, B \Rightarrow b\}$
G_2 ist nicht verkürzend (Typ 1)
$\mathcal{L}(G_2) = L_2$

$L_3 = \{w \in \{0, 1\}^* |$w endet auf 11$\}$
$G_3 = (\{0, 1\}, \{S\}, S, R_3)$
$R_3 = \{S \to 0S | 1S | 11\}$
G_3 ist kontextfrei
$G_3' = (\{0, 1\}, \{S, T\}, S, R_3')$
$R_3' = \{S \to 0S | 1S | 1T, T \to 1\}$
G_3' ist regulär, damit ist L_3 regulär

Behauptung: $\mathcal{L}(G_3') = L_3$
$\mathcal{L}(G_3') \subseteq L_3$ „jedes von G_3 erzeugte Wort gehört zu L_3“

z.Z.: jedes erzeugte Wort endet auf 11

- die Grammatik ist regulär, jedes Wort entsteht von links nach rechts
- Abbruch nur mit $T \to 1$ möglich
- zum Nichtterminal T gelangt man nur mit $S \to 1T$

↪ w endet auf 11

$L_3 \subseteq \mathcal{L}(G_3')$ „jedes Wort aus L_3 kann durch G_3' erzeugt werden “
Sei $w = u11, u \in \{0, 1\}^*$

$i = 1$ to k	$w = x_1, x_2, \ldots, x_k 11 \quad x_i \in \{0, 1\}$
$S \Rightarrow x_i S$	$S \vdash_{G_3'}^k x_1 x_2 \ldots x_k S$
$S \Rightarrow 1T$	$x_1 x_2 \ldots x_k 1T$
$T \Rightarrow 1$	$x_1 x_2 \ldots x_k 11$

Gemäß Definition 1.3.1 können kontextsensitive, kontextfreie und reguläre Sprachen niemals das leere Wort λ enthalten. Das ist schade, da viele Sprachen die λ enthalten trotzdem von einfacher Struktur sind. Sondervereinbarung:

i) $S \Rightarrow \lambda$ ist als einzige verkürzende Regel erlaubt
ii) tritt $S \Rightarrow \lambda$ auf, so darf S nicht in anderen Regeln auf der rechten Seite vorkommen

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

SATZ 1.3.4

Zu jeder Grammatik $G = (\Sigma, N, S, R)$ vom Typ i $\quad 1 \leq i \leq 3$ gibt es eine Grammatik G', so dass gilt:

a) G' ist mit Ausnahme der Regel $S \Rightarrow \lambda$ auch vom Typ i
b) $\mathcal{L}\left(G'\right) = \mathcal{L}\left(G\right) \cup \lambda$

Beweis

setzen $G' = (\Sigma, N \cup \{S'\}, S, R')$
$S' \notin \Sigma \cup N$ (neues Symbol)

$R' = \{S \Rightarrow R\}$
$\cup \{$alle Regeln $S \Rightarrow u \in R$ werden alle vorkommen von S auf der rechten Seite durch S' ersetzt$\}$
$\cup \{$alle Regeln aus R mit allen Nichtterminalen S durch S' ersetzt$\}$

Beispiel

$$S \Rightarrow aSa|a|X$$
$$X \Rightarrow bX|b$$
$$S \Rightarrow \lambda \left|aS'a\right| a|X$$
$$S' \Rightarrow aS'a \left|a\right| X$$
$$X \Rightarrow bX|b$$
$$S' \Rightarrow S|\lambda$$
$$S \Rightarrow aSa \left|a\right| X$$
$$X \Rightarrow bX|b$$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

2 Reguläre Sprachen

Beispiel

alle endlichen Sprachen sind regulär

- $\{a\}^*, \Sigma^*$ sind regulär
- Menge aller Wörter, die auf 11 enden
- Menge aller Wörter, die genau 5 a's enthalten

2.1 Endliche Automaten

Grammatiken erzeugen Wörter.
Automaten ... akzeptieren Wörter
... entscheiden, ob ein Eingabewort zur Sprache gehört

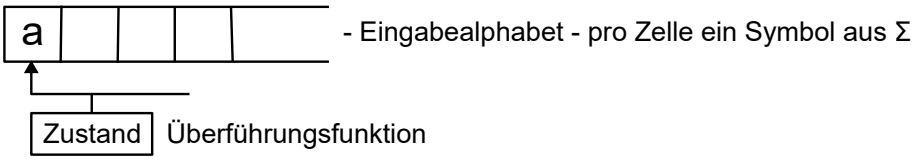
Definition 2.1.1

Ein deterministischer endlicher Automat (DFA) M ist ein 5-Tupel.
$M = (\Sigma, Z, \delta, z_0, Z_E)$ mit folgenden Eigenschaften:

1) Σ ist eine endliche Menge ... Eingabealphabet
2) Z ist eine endliche Menge ... Zustandsmenge
3) $\delta : Z \times \Sigma \Rightarrow Z$ ... Überführungsfunktion
4) $z_0 \in Z$ ... Startzustand
5) $Z_E \subseteq Z$ ... Menge der Endzustände

DFA ... deterministic finite automaton

Veranschaulichung:



pro Takt:

- Kopf liest aktuelles Zeichen
- Zustand wird in Abhängigkeit von gelesenem Symbol und aktuellen Zustand geändert
- Kopf bewegt sich um eine Stelle nach rechts

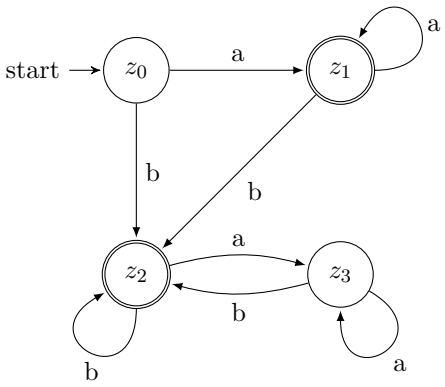
Der nach Abarbeitung des Wortes erreichte Zustand gibt an, ob das Wort akzeptiert wird oder nicht.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beispiel

$$M = (\{a, b\}, \{z_0, z_1, z_2, z_3\}, \delta, z_0, \{z_1, z_2\})$$

δ	z_0	z_1	z_2	z_3
a	z_1	z_1	z_3	z_3
b	z_2	z_2	z_2	z_2



$$L(M) = \{ub | u \in \{a, b\}^*\} \cup \{a^n | n \geq 1\}$$

$\hookrightarrow$ alle Folgen aus a's oder die auf b enden

Die von einem DFA akzeptierte Sprache ist die Menge aller Wörter, bei deren Eingabe der Automat in einem Zustand aus Z_E endet.

Definition 2.1.2

Es sei $M = (\Sigma, Z, \delta, z_0, Z_E)$ ein DFA. Wir definieren die erweiterte Überführungsfunktion

$$\delta^* : Z \times \Sigma^* \Rightarrow Z \text{ wie folgt für alle } z \in Z, a \in \Sigma, w \in \Sigma^*$$
$$\delta^*(z, \lambda) = z$$
$$\delta^*(z, aw) = \delta^*(\delta(z, a), w)$$

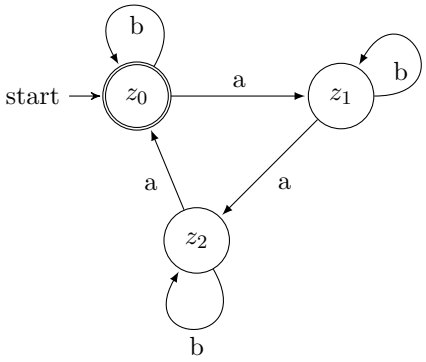
Die von M akzeptierte Sprache ist dann:
$L(M) = \{w \in \Sigma^* | \delta^*(z_0, w) \in Z_E\}$

Bemerkung: δ^* erweitert δ von Buchstaben zu Wörtern.
Offenbar ist $\delta^*(z, a_1, a_2, \ldots, a_n) = \delta(\ldots\delta(\delta(\delta(z, a_1), a_2), a_3), \ldots, a_n)$ gerade der von z aus nach Ableitung von $a_1, a_2, \ldots, a_n$ erreichte Zustand.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beispiel

$$M = (\{a, b\}, \{z_0, z_1, z_2\}, \delta, z_0, \{z_0\})$$



$$L(M) = \{w \in \{a, b\}^* \mid \#_a(w) \equiv_3 0\}$$

$$\delta^*(z_0, aaba) = z_0$$

$$\delta^*(z_0, aabaabb) = z_1$$

SATZ 2.1.3

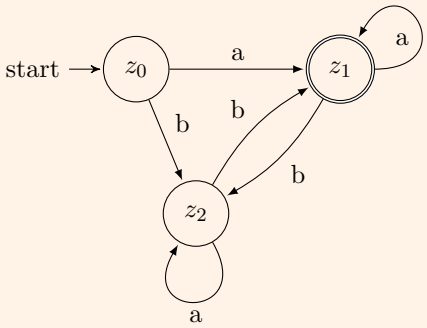
Jede Sprache, die von einer DFA akzeptiert werden kann, ist regulär

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

$A \subseteq \Sigma^*$ Sprache
$M = (\Sigma,\ Z, \delta, z_0, Z_E)$ DFA mit $L(M) = A$
Wir geben eine reguläre Grammatik G an, für die gerade $L(G) = A$ gilt.

1. Fall (brauchen also keine Regel $S \Rightarrow \lambda$)



$S = z_0$

$z_0 \Rightarrow az_1|bz_2|a$

$z_1 \Rightarrow az_1|bz_2|a$

$z_2 \Rightarrow az_2|bz_1$

$\delta(z_0, a) = z_1 \quad z_0 \Rightarrow az_1 \quad z_0 \Rightarrow a$

$\delta(z_0, b) = z_2 \quad z_0 \Rightarrow bz_2$

formal sei $G = (\Sigma, N, S, R)$. Wir setzen $N = Z, S = z_0$,
$R = \{(z \Rightarrow az)|\delta(z, a) = z'\} \cup \{(z \Rightarrow a)|\delta(z, a) = z', z' \in Z_E\}$
Das sind rechtslineare Regeln
Offenbar gilt für $w = a_1 a_2 \dots a_n \in \Sigma^*$

$w \in A \Leftrightarrow w \in L(M)$

$\Leftrightarrow$ Es gibt eine Folge von Zuständen von $Mz_0, \dots, z_n$ mit z_0 als Startzustand, $z_n \in Z_E$ und für alle $0 \leq i \leq n-1$ gilt $\delta(z_i, a_{i+1}) = z_{i+1}$

$\Leftrightarrow$ Es gibt eine Folge von Nichtterminalen $z_0, z_1, \dots, z_n$ mit z_0 als Startsymbol und

$z_0 \vdash_G a_1 z_1 \vdash_G a_1 a_2 z_2 \vdash_G \dots \vdash_G a_1 a_2 \dots a_{n-1} z_{n-1} \vdash_G a_1 \dots a_n$

$\Leftrightarrow z_0 \vdash^* w$

$\Leftrightarrow w \in \mathcal{L}(G) \Rightarrow L(M) = \mathcal{L}(G)$

2. Fall folgt aus Fall 1 unter Berücksichtigung von SATZ 1.3.4

Definition 2.1.4

Es sei $M = (\Sigma,\ Z, \delta, z_0, Z_E)$ ein DFA.
Dann ist
$R_M = \{(x, y) \in \Sigma^* \times \Sigma^* | \delta^*(z_0, x) = \delta^*(z_0, y)\}$
$(x, y) \in R_m \Leftrightarrow$ Bei Abarbeitung im DFA enden x und y im gleichen Zustand.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

SATZ 2.1.5

Für einen DFA $M = (\Sigma, Z, \delta, z_0, Z_E)$ ist R_M eine Äquivalenzrelation.

Beweis

Reflexivität

$\bigwedge_{x \in \Sigma^*} (x, x) \in R_M$ denn $\bigwedge_{x \in \Sigma^*} \delta^*(z_0, x) = \delta^*(z_0, x)$

Symmetrie

Es seien $x, y \in \Sigma^*$

$(x, y) \in R_M \Rightarrow \delta^*(z_0, x) = \delta^*(z_0, y) \Rightarrow (y, x) \in R_M$

Transitivität

Sei $x, y, w \in \Sigma^*$

Es seien $(x, y) \in R_M$ und $(y, w) \in R_M$,

d.h.$\delta^*(z_0, x) = \delta^*(z_0, y)$ und $\delta^*(z_0, y) = \delta^*(z_0, w)$

$\hookrightarrow \delta^*(z_0, x) = \delta^*(z_0, w)$

$\hookrightarrow (x, w) \in R_M$

SATZ 2.1.6

Sei $M = (\Sigma,\ Z, \delta, z_0, Z_E)$ ein DFA. Dann gilt:

$(x, y) \in R_M \Leftrightarrow \bigwedge_{w \in \Sigma^*} (xw, yw) \in R_M$

R_M ist rechtsinvariant gegen Verkettung

Beweis

„ $\Rightarrow$ “ $(x, y) \in R_M \Rightarrow \delta^*(z_0, x) = \delta^*(z_0, y)$

$\hookrightarrow \bigwedge_{w \in \Sigma^*} \delta^*(z_0, xw) = \delta^*(z_0, yw)$

$\hookrightarrow \bigwedge_{w \in \Sigma^*} (xw, yw) \in R_M$

„ $\Leftarrow$ “Kontraposition $\neg A \Rightarrow \neg B$

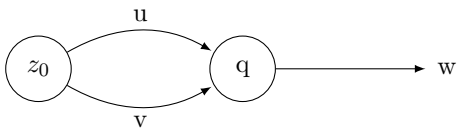
z.Z.: $(x, y) \notin R_M \Rightarrow \bigwedge_{w \in \Sigma^*} (xw, yw) \notin R_M$

Das gilt für $w = \lambda$

studocu

messages.pdf_cover_qr_code_label
messages.downloaded_by

Konstruktion deterministischer endlicher Automaten
Seien $u, v \in \Sigma^*$ mit $(u, v) \in R_M$, d.h. $\delta^*(z_0, u) = \delta^*(z_0, v) = q$

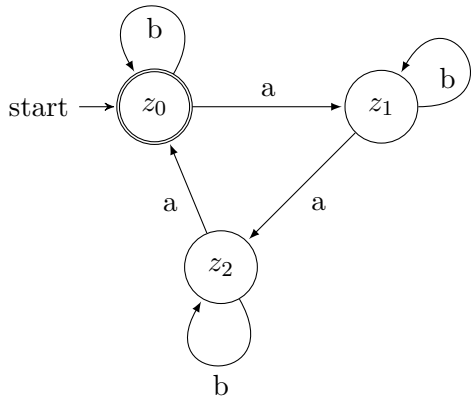


Bemerkung
Für jedes beliebige Wort $w \in \Sigma^*$ kann M nicht zwischen u's und v's unterscheiden.

- Die einzige Möglichkeit sich Informationen eines bereits gelesenen Wortteils zu merken ist der Zustand des DFA, indem dieser Wortteil endet. $(\delta^*(z_0, u))$
- Die für die Zugehörigkeit beziehungsweise Nichtzugehörigkeit eines Wortes u zu L relevanten Informationen müssen in q gespeichert sein.

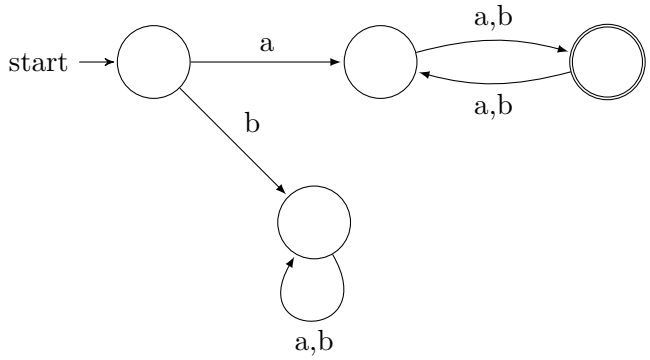
$L_1 = \{w \in \{a, b\}^* \mid \#a(w) \equiv_3 0\}$

$M = (\{a, b\}, \{z_0, z_1, z_2\}, \delta, z_0, \{z_0\})$

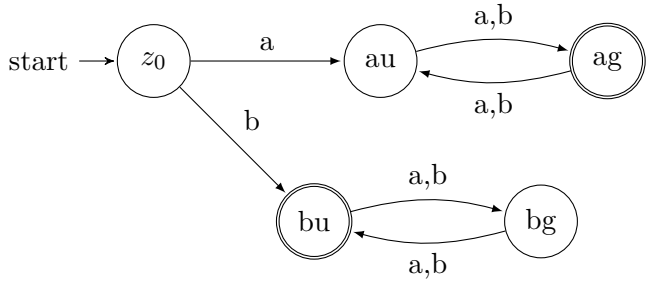


studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

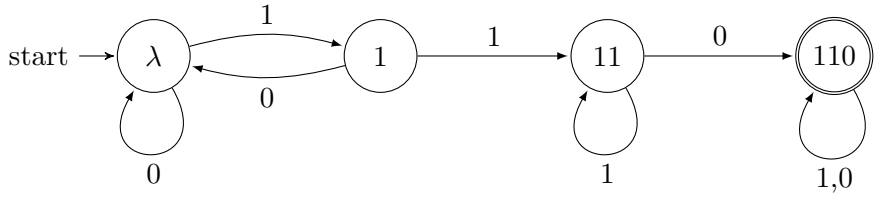
$L_2 = \{w \in \{a,b\}^* \,|\text{w beginnt mit a und } |w| \equiv_2 0\}$



$L_3 = \{w \in \{a,b\}^* \,|\text{w beginnt mit a gdw. } |w| \equiv_2 0\}$
$aa \in L_3,\ \lambda \in L_3,\ b \in L_3,\ baaaa \in L_3$

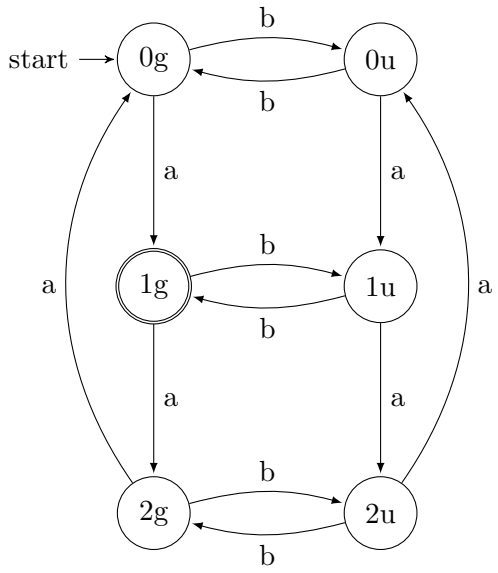


$L_4 = \{w \in \{a,b\}^* \,|\text{w enthält das Teilwort 110}\}$



studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

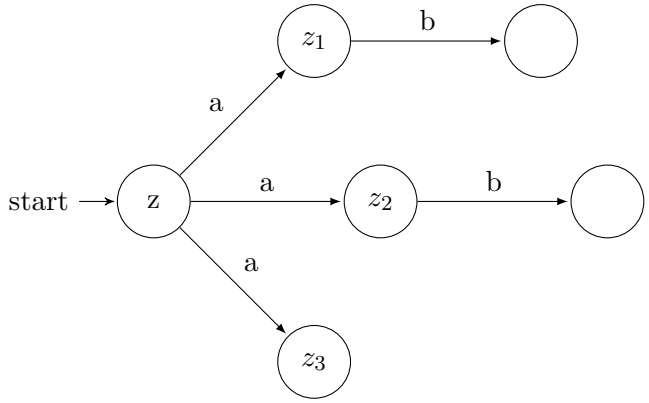
$L_5 = \{w \in \{a,b\}^* \,|\#a\,(w) \equiv_3 1 \text{ und } \#b\,(w) \equiv_2 0\}$



studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

2.2 Nichtdeterministische endliche Automaten

Idee: $\delta(z,a) = \{z_1, z_2, z_3\}$ $\quad \delta(z,b) = \emptyset$



Ein nichtdeterministischer endlicher Automat akzeptiert ein Wort w, wenn er bei Eingabe w eine Zustandsfolge durchlaufen kann, die zu einem Endzustand führt.

Definition 2.2.1

Ein nichtdeterministischer endlicher Automat (NFA) ist ein 5 Tupel $M = (\Sigma, Z, \delta, S, Z_E)$ mit folgenden Eigenschaften:

1) Σ ist eine endliche Menge ... Eingabealphabet
2) Z ist eine endliche Menge ... Zustandsmenge
3) $\delta: \ \Sigma \times \Sigma \Rightarrow \gamma(z)$ ... Überführungsfunktion
4) $S \subseteq Z$ ... Menge der Startzustände
5) $Z_E \subseteq Z$ ... Menge der Endzustände

Definition 2.2.2

Die erweiterte Überführungsfunktion $\delta^*: \gamma(z) \times \Sigma^* \Rightarrow \gamma(z)$ sei wie folgt definiert:
Für alle $\widetilde{z} \in Z, a \in \Sigma, w \in \Sigma^*$ ist
$\delta^*(\widetilde{z}, \lambda) = \widetilde{z}$
$\delta^*(\widetilde{z}, aw) = \bigcup_{z \in \widetilde{z}} \delta^*(\delta(z,a), w)$
$\delta(z,a)$ ... Menge der von z aus nach Abarbeitung von a erreichbaren Zustände
$\delta^*(\widetilde{z}, w)$ ... Menge der Zustände, die erreicht werden können, wenn in irgendeinem $z \in \widetilde{z}$ gestartet und w abgearbeitet wird.

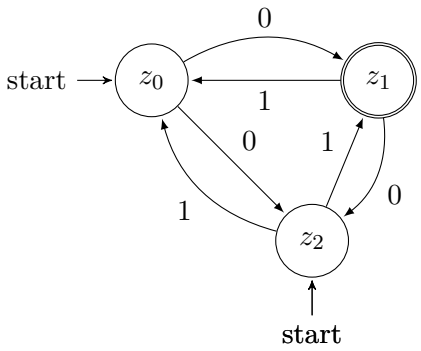
Definition 2.2.3

Die von einem NFA $M = (\Sigma, Z, \delta, S, Z_E)$ akzeptierte Sprache ist
$L(M) = \{w \in \Sigma^* | \delta^*(S, w) \cap Z_E \neq \emptyset\}$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beispiel

$$M = (\{0,1\}, \{z_0, z_1, z_2, \}, \delta, \{z_0, z_2\}, \{z_1\})$$



$$\begin{aligned}
\delta^*(\{z_0, z_2\},\ 100) &= \delta^*(\delta(z_0, 1), 00) \cup \delta^*(\delta(z_2, 1), 00) \\
&= \delta^*(\emptyset, 00) \cup \delta^*(\{z_0, z_1\}, 00) \\
&= \underbrace{\bigcup_{z \in \emptyset} \delta^*(\delta(z, 0), 0)}_{\emptyset} \cup \delta^*(\delta(z_0, 0) \cup \delta^*(z_1, 0), 0) \\
&= \delta^*(\{z_1, z_2\}, 0) \cup \underbrace{\delta^*(\{z_2\}, 0)}_{\emptyset} \\
&= \delta^*(\delta(z_1, 0), \lambda) \cup \delta^*(\delta(z_2, 0), \lambda) \\
&= \delta^*(\{z_2\}, \lambda) \cup \delta^*(\emptyset, \lambda) = \{z_2\}
\end{aligned}$$

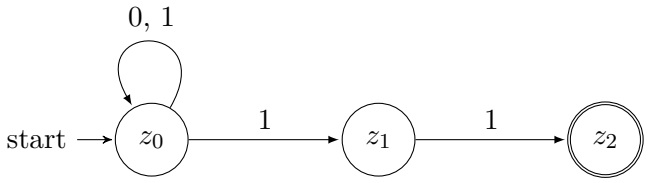
$\hookrightarrow$ 100 wird nicht akzeptiert

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beispiel

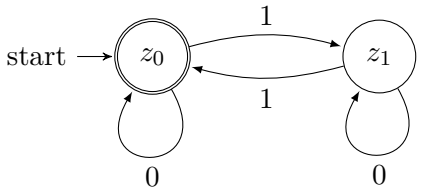
$$L = \{w \in \{0,1\}^* \,|\text{w endet auf } 11\}$$

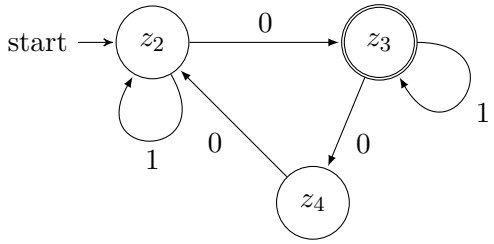
$$M = (\{0,1\}, \{z_0, z_1, z_2\}, \delta, \{z_0\}, \{z_2\})$$



$$L = \{w \in \{0,1\}^* \,|\#_1(w) \equiv_2 0 \text{ oder } \#_0\{w\} \equiv_3 1\}$$

$$M = (\{0,1\}, \{z_0, \ldots, z_4\}, \delta, \{z_0, z_2\}, \{z_0, z_3\})$$





SATZ 2.2.4

Jede reguläre Sprache kann von einem NFA akzeptiert werden.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

(Hinweis: X' Zustand, X Nichtterminal)
Sei $A \in \text{REG}$, $A \subseteq \Sigma^*$ Nach Definition gibt es eine reguläre Grammatik $G = (\Sigma, N, S, R)$ mit $A = \mathcal{L}(G)$. Wir betrachten folgenden NFA:

$$M = \left(\Sigma, Z, \delta, S', Z_E\right)$$

$$Z = N \cup \{F\}, F \notin N$$

$$S' = \{S\}$$

$$Z_E = \begin{cases} \{F\} & \text{falls } (S \to \lambda) \notin R \\ \{F, S\} & \text{falls } (S \to \lambda) \in R \end{cases}$$

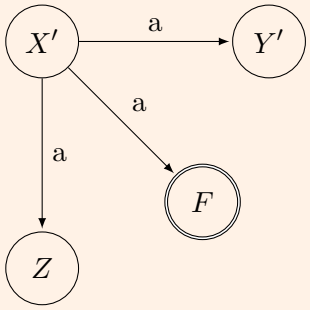
$$\delta\left(X', a\right) = \begin{cases} \left\{Y' \middle| (X \to aY) \in R\right\} & \text{falls } (X \to a) \notin R \\ \left\{Y' \middle| (X \to aY) \in R\right\} \cup \{F\} & \text{falls } (X \to a) \in R \end{cases}$$

Idee: In Grammatik

$X' \to aY'$

$X' \to aZ$

$X' \to a$

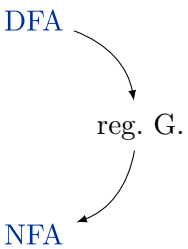


Offenbar gilt für ein Wort $w = a_1 \ldots a_n \in A \Leftrightarrow a_1 \ldots a_n \in \mathcal{L}(G)$

$\Leftrightarrow$ es gibt eine Folge von Nichtterminalen $X_1, X_2, \ldots, X_{n-1} \in N$, so dass
$S \vdash_G a_1 X_1 \vdash_G a_1 a_2 X_2 \vdash_G \ldots \vdash_G a_1 a_2 \ldots a_{n-1} X_{n-1} \vdash_G a_1 \ldots a_n$

$\Leftrightarrow$ es gibt eine Folge von Zuständen $X'_1, X'_2, \ldots, X'_{n-1} \in Z$ mit
$X'_1 \in \delta\left(S', a_1\right), X'_2 \in \delta\left(X'_1, a_2\right), \ldots, X'_{n-1} \in \delta\left(X'_{n-2}, a_{n-1}\right)$
und $F \in \delta(X'_{n-1}, a_n)$

$\Leftrightarrow a_1 \ldots a_n \in L(M)$



studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Potenzmengenkonstruktion

SATZ 2.2.5

Jede Sprache, die von einem NFA akzeptiert wird, kann auch von einem DFA akzeptiert werden.

studocu

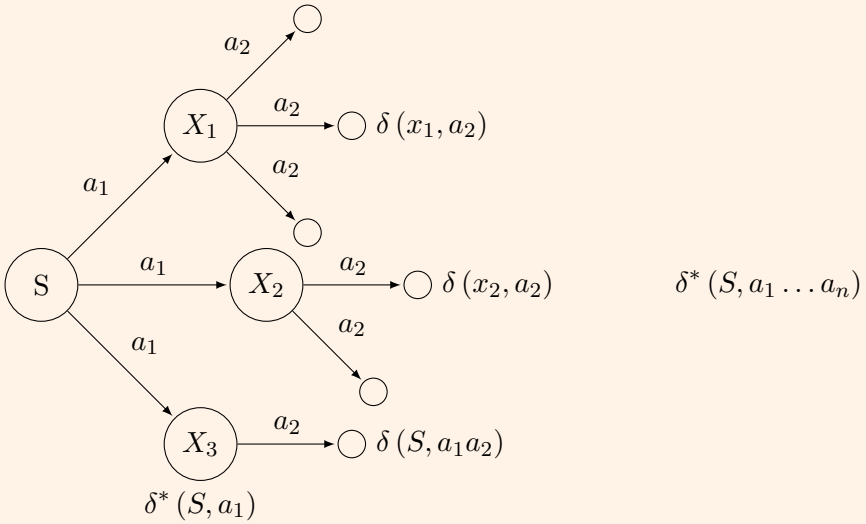
messages.pdf_cover_qr_code_label
messages.downloaded_by



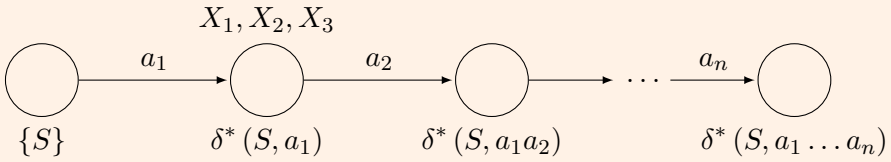
Beweis

Sei $A \subseteq \Sigma^*$ und sei $M = (\Sigma, Z, \delta, S, Z_E)$ ein NFA mit $L(M) = A$
Idee: Abarbeitung von $w = a_1, a_2, \ldots, a_n$

NFA:



DFA:



Wir definieren DFA $M' = (\Sigma, \zeta, \delta', S', Z'_E)$ mit $\zeta = \gamma(Z)$
$\delta'(z', a) = \bigcup_{z \in Z'} \delta(z, a)$ für $Z' \in \zeta \quad (Z' \subseteq Z)$
$S' = S$
$Z'_E = Z' \subseteq Z | Z' \cap Z_E \neq \emptyset$
Für jedes Wort $w = a_1 a_2 \ldots a_n \in \Sigma^*$ gilt:
$w \in L(M) \Leftrightarrow \delta^*(S, w) \cap Z_E \neq \emptyset$

Für jedes Wort $w = a_1 a_2 \ldots a_n \in \Sigma^*$ gilt:

$w \in L(M) \Leftrightarrow \delta^*(S, w) \cap Z_E \neq \emptyset$

$\Leftrightarrow$ es gibt eine Folge von Teilmengen $z_1, \ldots, z_n$ von Z mit

$\delta^*(S, a_1) = z_1, \delta^*(z_1, a_2) = z_2 \ldots \delta^*(z_{n-1}, a_n) = z_n$ und $z_n \cap Z_E \neq \emptyset$

$\Leftrightarrow$ es gibt eine Folge von Zuständen $z_1, \ldots, z_n$ von M' mit

$\delta'(S', a_1) = z_1, \delta'(z_1, a_2) = z_2 \ldots \delta'(z_{n-1}, a_n) = z_n$

$\Leftrightarrow \delta'^*(S, a_1 \ldots a_n) \in Z'_E$

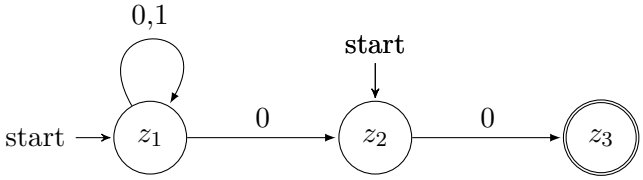
$\Leftrightarrow w \in L(M') \Leftrightarrow A = L(M')$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beispiel

NFA $M = (\{0,1\}, \{z_1, z_2, z_3\}, S, \{z_1, z_2\}, \{z_3\})$

δ	0	1
z_1	$\{z_1, z_2\}$	$\{z_1\}$
z_2	$\{z_3\}$	$\emptyset$
z_3	$\emptyset$	$\emptyset$



Menge aller Wörter die auf 00 enden und das Wort 0

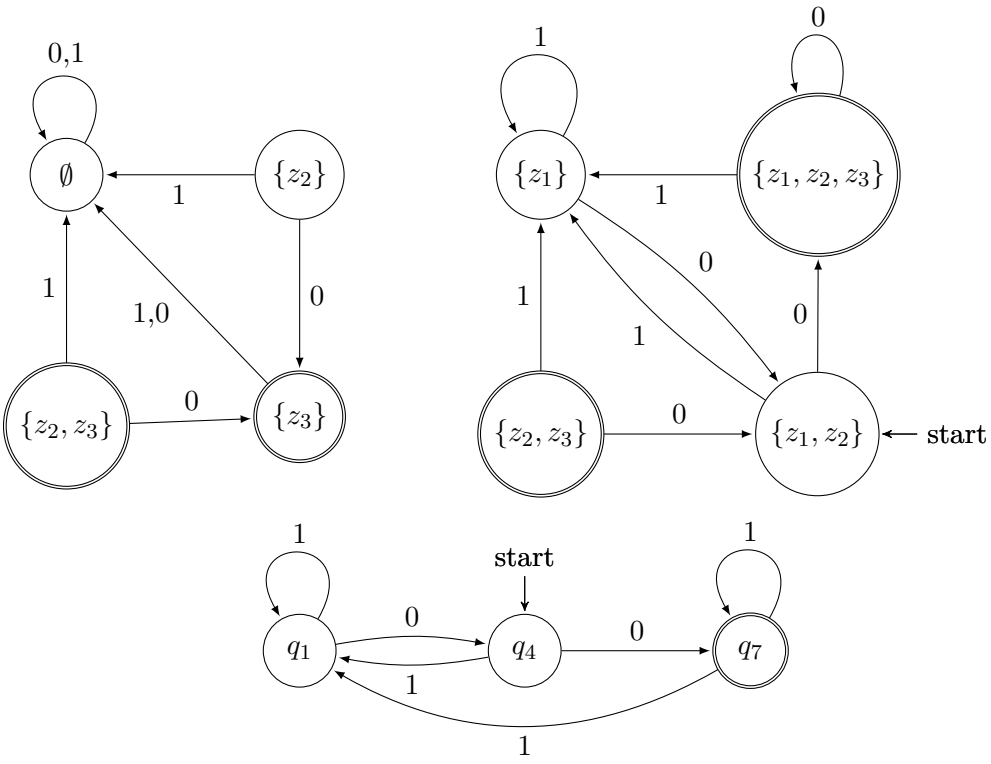
DFA $M' = (\{0,1\}, Z', \delta', S', Z'_E)$

$S' = \{z_1, z_2\} = q_4$

$Z'_E = \{\{Z_3\}, \{z_1, z_3\}, \{z_2, z_3\}, \{z_1, z_2, z_3\}\} = q_3, q_5, q_6, q_7$

$Z' = \gamma(\{z_1, z_2, z_3\}) = \{\emptyset, \{z_1\}, \{z_2\}, \{z_3\}, \{z_1, z_2\}, \{z_1, z_3\}, \{z_2, z_3\}, \{z_1, z_2, z_3\}\}$

	q_0	q_1	q_2	q_3	q_4	q_5	q_6	q_7
δ'	$\emptyset$	$\{z_1\}$	$\{z_2\}$	$\{z_3\}$	$\{z_1, z_2\}$	$\{z_1, z_3\}$	$\{z_2, z_3\}$	$\{z_1, z_2, z_3\}$
0	$\emptyset$	$\{z_1, z_2\}$	$\{z_3\}$	$\emptyset$	$\{z_1, z_2, z_3\}$	$\{z_1, z_2\}$	$\{z_3\}$	$\{z_1, z_2, z_3\}$
1	$\emptyset$	$\{z_1\}$	$\emptyset$	$\emptyset$	$\{z_1\}$	$\{z_1\}$	$\emptyset$	$\{z_1\}$



$M'(\{0,1\}, \{q_1, q_4, q_7\}, \delta'', q_4, \{q_7\})$ Zustände, die aus dem Startzustand nie erreicht werden können, können weggelassen werden.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Bemerkung:

- Zu gegebenen NFA mit n Zuständen erhält man durch Potenzmengenkonstruktion einen DFA mit 2^n Zuständen.
- In unserem Beispiel ist dieser noch verkleinerbar.
- Es gibt Beispiele, bei denen alle 2^n Zustände gebraucht werden.

2.3 Reguläre Ausdrücke

Definition 2.3.1

Sei Σ ein Alphabet. Die Menge der regulären Ausdrücke über Σ $RA(\Sigma)$ wird definiert durch:

1) λ und $\emptyset$ sind reguläre Ausdrücke
2) Für jedes $a \in \Sigma$ ist a ein regulärer Ausdruck
3) Seien α und β reguläre Ausdrücke, so sind auch $\alpha \cdot \beta, (\alpha + \beta), (\alpha)^*$ reguläre Ausdrücke
4) Andere reguläre Ausdrücke gibt es nicht

Bemerkung: $RA(\Sigma)$ ist eine Sprache über $\{\emptyset, \lambda, (,), +, *\} \cup \Sigma$
$RA(\Sigma)$ ist nicht regulär, aber kontextfrei

Definition 2.3.2

Sei γ ein regulärer Ausdruck über $\Sigma, \gamma \in RA(\Sigma)$. Die Sprache $L(\gamma) \subseteq \Sigma^*$ ist wie folgt definiert:

1) $L(\emptyset) = \emptyset$ und $L(\lambda) = \{\lambda\}$ $(\gamma = \emptyset, \gamma = \lambda)$
2) $L(a) = \{a\}$ für alle $a \in \Sigma$
3) $$L(\gamma) = \begin{cases} L(\alpha) = L(\beta) \text{ falls} & \gamma = \alpha \cdot \beta \\ L(\alpha) \cup L(\beta) & \gamma = (\alpha + \beta) \\ (L(\alpha))^* & \gamma = (\alpha)^* \end{cases}$$

Beispiel

$$\Sigma = \{a, b\}$$
$$L(((a + b))^*) = \{a, b\}^*$$
$$L(a((a + b))^* abba((a + b))^*) = \{a\}\{a, b\}^*\{abba\}\{a, b\}^*$$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Lemma 2.3.3

Jede endliche Sprache kann durch einen regulären Ausdruck beschreiben werden.

Beweis

Sei $w_1 = a_{11}a_{12}\dots a_{1m_1}$
$w_2 = a_{21}a_{22}\dots a_{2m_2}$
$\vdots$
$w_n = a_{n1}a_{n2}\dots a_{nm_n}$

Idee: $(\dots(((w_1 + w_2) + w_3) + w_4)\dots + w_n)$

$\gamma = ((\dots((a_{11}a_{12}\dots a_{1m_1} + a_{21}a_{22}\dots a_{2m_2}) + a_{31}\dots a_{3m_3}) + \dots) + a_{n1}\dots a_{nm_n})$

SATZ 2.3.4 (Kleene)

Eine Sprache ist genau dann regulär, wenn sie durch einen regulären Ausdruck beschrieben werden kann.

Beweis

„ $\Rightarrow$ “

Sei $A \subseteq \Sigma^*$ eine reguläre Sprache und sei $M = (\Sigma, Z, \delta, z_0, Z_E)$ ein DFA mit $L(M) = A$. Wir geben einen regulären Ausdruck γ an, für den gerade $L(\gamma) = L(M)$ gilt. Sei $Z = \{z_1, \dots, z_n\}$. Wir definieren für $i, j \in \{0, \dots, n\}$ und $k = \{1, \dots, n\}$.

$$L_{i,j}^k = \Big\{ w = x_1 \dots x_m \in \Sigma^* | \delta^*(z_i, w) = z_j \quad \text{und} \bigwedge_{1 \le l \le m-1} \delta^*(z_i, x_1 \dots x_l) = z_r \Rightarrow r \le k \Big\}$$

(keiner der erreichten Zwischenzustände hat Index größer k)

$k = 0 \quad i \neq j \qquad L_{i,j}^0 = \{a \in \Sigma | \delta(z_i, a) = z_j\}$

$k = 0 \quad i = j \qquad L_{i,j}^0 = \{\lambda\} \cup \{a \in \Sigma | \delta(z_i, a) = z_i\}$

Offenbar ist $L_{i,j}^0$ endlich und lässt sich somit durch einen regulären Ausdruck beschreiben (Lemma 2.3.3).

Behauptung: Für alle $i, j \in 1, \dots, n$ und $k \in 0, \dots, n$ lässt sich $L_{i,j}^k$ durch einen regulären Ausdruck beschreiben.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

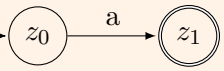
Beweis

Induktion über k
IA: $k = 0$ siehe oben
IV: sei $k \geq 0, L_{i,j}^k$ lässt sich für jedes i, j durch regulären Ausdruck beschreiben.
IB: Auch $L_{i,j}^{k+1}$ lässt sich für jedes i, j durch regulären Ausdruck beschreiben.
$k \to k+1$ Sei $i, j \in \{1, \ldots, n\}$ beliebig aber fest gewählt.
Offenbar ist $L_{i,j}^{k+1} = \underbrace{L_{i,j}^k}_{\substack{Z_{k+1}\text{wird} \\ \text{nicht benutzt}}} \cup \underbrace{\left(L_{i,k+1}^k \left(L_{k+1,k+1}^k\right)^* L_{k+1,j}^k\right)}_{\substack{Z_{k+1}\text{wird ein oder} \\ \text{mehrmals benutzt}}}$
Sein nun $L_{i,j}^k, L_{i,k+1}^k, L_{k+1,k+1}^k, L_{k+1,j}^k$ reguläre Ausdruck für $L_{i,j}^k, L_{i,k+1}^k, L_{k+1,k+1}^k, L_{k+1,j}^k$
Diese gibt es nach IV
Dann ist $L_{i,j}^{k+1} = L\left(\left(L_{i,j}^k + L_{i,k+1}^k \left(L_{k+1,k+1}^k\right)^* L_{k+1,j}^k\right)\right)$ ■Behauptung
Nun lässt sich $L(M)$ ausdrücken als $L(M) = \bigcup_{z_i \in Z_E} L_{1,i}^n$
sind $i_1, i_2, \ldots, i_m$ die Indizes der Endzustände, so ist
$L(M) = L\left(\alpha_{1,i_1}^n + \left(\alpha_{1,i_2}^n + \left(\ldots\left(\ldots\left(\ldots + \alpha_{1,i_m}^n\right)\ldots\right)\ldots\right)\right)\right)$

„ $\Leftarrow$ “
Idee: Die Induktive Definition von $RA(\Sigma)$ kann mit NFA's nachvollzogen werden.
Sei $A \subseteq \Sigma^*$ und sei $\gamma \in RA(\Sigma)$ mit $L(\gamma) = A$. Wir geben einen NFA M für A an.

1) ist $\gamma = \emptyset, M = (\Sigma, \{z_0\}, \delta, \{z_0\}, \emptyset)$ mit 

2) ist $\gamma = \{\lambda\}, M = (\Sigma, \{z_0\}, \delta, \{z_0\}, \{z_0\})$ mit 

3) ist $\gamma = a \in \Sigma, M = (\Sigma, \{z_0, z_1\}, \delta, \{z_0\}, \{z_1\})$ mit 

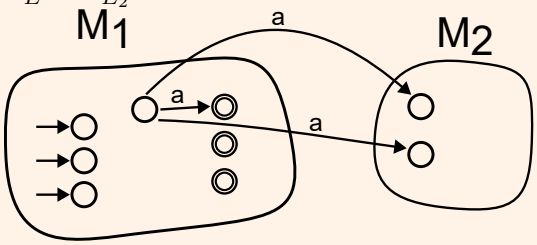
4) $\gamma = (\alpha + \beta), \gamma = \alpha \cdot \beta, \gamma = (\alpha)^*$
Seien $M_1 = (\Sigma, Z_1\delta_1, Z_{0_1}, Z_{E_1})$ und $M_2 = (\Sigma, Z_2\delta_2, Z_{0_2}, Z_{E_2})$ NFA's mit
$L(M_1) = L(\alpha)$ und $L(M_2) = L(\beta)$
$o.B.d.A. Z_1 \cap Z_2 = \emptyset$
Wir konstruieren $M = (\Sigma, Z, \delta, Z_0, Z_E)$ mit $L(M) = L(\gamma)$

a) $\gamma = (a + b)$
$Z = Z_1 \cup Z_2$
$$\delta(z, a) = \begin{cases} \delta_1(z, a) & \text{falls } z \in Z_1, Z_0 = Z_{0_1} \cup Z_{0_2} \\ \delta_2(z, a) & \text{falls } z \in Z_2, Z_E = Z_{E_1} \cup Z_{E_2} \end{cases}$$

b) $\gamma = \alpha \cdot \beta$
$Z = Z_1 \cup Z_2$

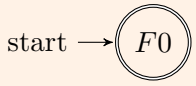
studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

$$Z_0 = \begin{cases} Z_{0_1} \cup Z_{0_2} & \text{falls } \lambda \in L(M_1) \\ Z_{0_1} & \text{falls } \lambda \notin L(M_1) \end{cases}$$
$$Z_E = Z_{E_2}$$



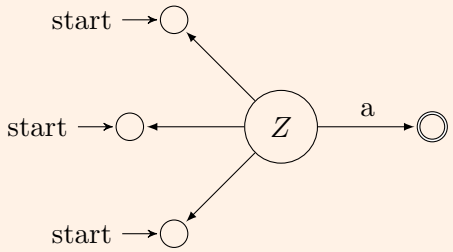
$$\delta(z,a) = \begin{cases} \delta_1(z,a) \cup Z_{0_2} & \text{falls } z \in Z_1 \text{ und } \delta(z,a) \in Z_{E_1} \\ \delta_1(z,a) & \text{falls } z \in Z_1 \text{ und } \delta(z,a) \notin Z_{E_1} \\ \delta_2(z,a) & \text{falls } z \in Z_2 \end{cases}$$

c) $\gamma = (\alpha)^*$ NFA $M_1 = (\Sigma, Z, \delta, Z_0, Z_E)$ akzeptiert $L(\alpha)$
In zwei Stufen vorgehen
$M_{1\prime} = (\Sigma, Z_1, \delta\prime, Z_0\prime, Z_E\prime)$
Ist $(\lambda \in L(M_1))$, so ist $M_1' = M_1$
Ist $\lambda \notin L(M_1)$, so ist
$M_1' = \left(\Sigma, Z_1 \cup \{FO\}, \delta', Z_0 \cup \{FO\}, Z_E \cup \{FO\}\right)$



M_1

Offenbar ist $L\left(M_1'\right) = L(M_1) \cup \{\lambda\}$
$M_1'' = (\Sigma, Z_1', \delta'', Z_0', Z_E')$ mit
$$\delta''(z,a) = \begin{cases} \delta'(z,a) \cup Z_0 & \text{falls } \delta'(z,a) \in Z_E' \\ \delta'(z,a) \end{cases}$$

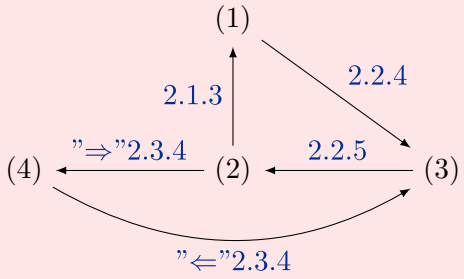


studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Folgerung 2.3.5

Sei $A \subseteq \Sigma^*$. Die folgenden Aussagen sind äquivalent:

1) A ist eine reguläre Sprache
2) A kann von einem DFA akzeptiert werden
3) A kann von einem NFA akzeptiert werden
4) A kann durch einen regulären Ausdruck beschrieben werden



2.4 Das Pumping-Lemma

Bisher: wir wissen, wann eine Sprache regulär ist.

Das Nachweisen der Nichtregularität ist schwieriger, da man zeigen muss, dass es keinen DFA, keinen NFA bzw. keine reguläre Grammatik gibt, der die Sprache akzeptiert bzw. erzeugt.

SATZ 2.4.1 Pumping-Lemma

Sei A eine reguläre Sprache. Dann gibt es ein $n \in \mathbb{N}$, so dass sich alle Wörter $x \in A$ mit $|x| \geq n$, so in $x = uvw$ zerlegen lassen, dass folgende Bedingungen erfüllt sind:

(1) $|v| \geq 1$

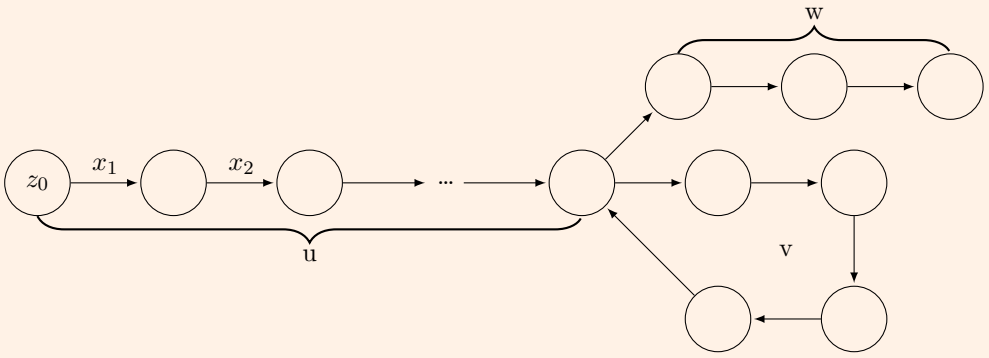
(2) $|uv| \leq n$

(3) Für $i \geq 0$ gilt $uv^iw \in A$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

Sei $A \in$ REG. Sei$M = (\Sigma, Z, \delta, z_0, Z_E)$ ein DFA mit$A = L(M)$. Wir wählen $n = |Z|$.
Sei nun $x \in A$ mit $|x| \geq n$. beim Abarbeiten von x durchläuft M $|x| + 1$ Zustände (inklusive Startzustand). Da $|x| \geq n$ können diese Zustände nicht alle verschieden sein.
M hat bei Abarbeitung x eine Schleife durchlaufen.



Wir wählen nun u, v, w mit $x = uvw$ so, dass $\delta^*(z_0, u) = \delta^*(z_0, uv)$.
Man kann diese Zerlegung so wählen, dass $|v| \geq 1$ und $|uv| \leq n$ gilt.
Wegen $\delta^*(z_0, u) = \delta^*(z_0, uv)$ gilt auch $\delta^*(z_0, uw) = \delta^*(z_0, uvw) = \delta^*\left(z_0, uv^i w\right)$
Mit anderen Worten:
$uvw \in A \Leftrightarrow uv^i w \in A$
wegen $x \in A$ folgt Behauptung (3)■

Beispiel

$A = \{a^m b^m | m \geq 1\}$
Annahme: $A \in$ REG
Dann gibt es $n \in \mathbb{N}$, so dass sich alle $x \in A$ mit $|x| \geq n$ derart in $x = uvw$ zerlegen lassen, dass gilt $|v| \geq 1, |uv| \leq n, uv^i w \in A$ für alle $i \geq 0$.
Wählen $x = a^n b^n, |x| = 2n \geq n$
Sei $x = uvw$ eine geeignete Zerlegung gemäß Pumping-Lemma,
d.h. $|v| \geq 1$ und $|uv| \leq n$
Folglich kann v nur aus $a's$ bestehen $v = a^k, 1 \leq k \leq n$
Damit gilt $uv^0 w = uw = a^{n-k} b^n \notin A$
Das ist ein Widerspruch und somit $A \notin$ REG

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beispiel

$B = \{0^m | m \text{ ist Quadratzahl}\}$
Annahme: $B \in$ REG
Sei n die Pumpingzahl
Wählen $x = 0^{n^2}, x \in B, |x| \geq n$
Sei $x = uvw$ eine geeignete Zerlegung gemäß Pumping-Lemma,
d.h. $|v| \geq 1, |uv| \leq n$
$\hookrightarrow v = 0^k, 1 \leq k \leq n$
$n^2 = |x| = |uvw| < |uv^2w| = |uvw| + |v| = n^2 + k \leq n^2 + n < n^2 + 2n + 1 = (n+1)^2$
$\hookrightarrow uv^2w \notin B$. Damit ist $B \notin$ REG

Das Pumping-Lemma liefert keine Charakterisierung der regulären Sprachen. Es stellt nur ein notwendiges Kriterium dar.
$A \in \text{REG} \Rightarrow \bigvee_n \bigwedge_{x,|x|\geq n} \bigvee_{u,v,w} x = uvw, |v| \geq 1, |uv| \leq n, \bigwedge_{i\geq 0} uv^iw \in A$

2.5 Minimalautomaten und Äquivalenzrelation

Es kann jeder Sprache eine Äquivalenzrelation zugeordnet werden.

Definition 2.5.1

Sei $A \subseteq \Sigma^*$. Dann ist

$$R_A = \left\{ (x,y) \in \Sigma^* \times \Sigma^* | \bigwedge_{z\in\Sigma^*} (xz \in A \Leftrightarrow yz \in A) \right\}$$

$(x,y) \in R_A \Leftrightarrow \bigwedge_{z\in\Sigma^*} (xz \in A \Leftrightarrow yz \in A)$

$\Leftrightarrow$ beim Anhängen beliebiger Wörter an x und y verhalten sie sich bezüglich Mitgliedschaft zu A gleich.

R_A ist Äquivalenzrelation

$\bigwedge_{x\in\Sigma^*} (x,x) \in R_A$

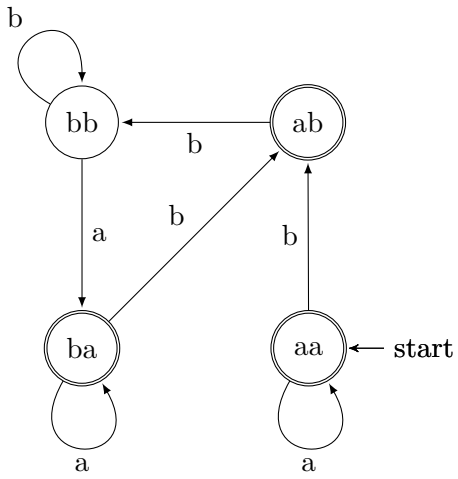
$\bigwedge_{x,y\in\Sigma^*} (x,y) \in R_A \Rightarrow (y,x) \in R_A$

$\bigwedge_{x,y,u\in\Sigma^*} (x,y) \in R_A \wedge (y,u) \in R_A \Rightarrow (x,u) \in R_A$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beispiel

$$L = \{w = (a,b)^* \in \Sigma^* | w \text{ endet nicht auf bb}\}$$

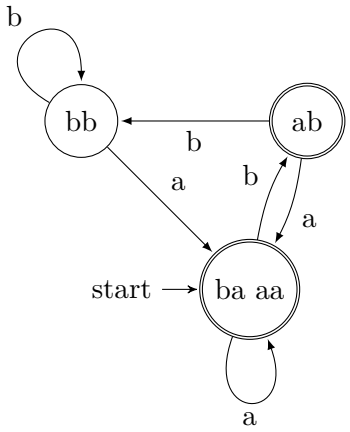


$(\ldots bb, \ldots ba) \in R_A$

$(\ldots bb, \ldots ab) \in R_A$

$\vdots$

$(\ldots ba, \ldots aa) \in R_A$



Behauptung:

$[bb] = \{w \in \{a,b\}^* | w \text{ endet auf bb}\}$

$[ab] = \{w \in \{a,b\}^* | w \text{ endet auf ab}\} \cup \{b\}$

$[wa] = \{w \in \{a,b\}^* | w \text{ endet auf a}\} \cup \{\lambda\}$

Wir werden zeigen, dass R_M eine Verfeinerung von R_A ist, also

$\bigwedge_{x,y \in \Sigma^*} (x,y) \in R_M \Rightarrow (x,y) \in R_A$

Das liefert uns ein Kriterium für Regularität.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

SATZ 2.5.2 Myhill-Nerode

Die folgenden Aussagen sind äquivalent:

1) A ist regulär
2) A ist die Vereinigung von Äquivalenzklassen einer rechtsinvarianten Äquivalenzrelation mit endlichem Index.
3) R_A hat endlichen Index

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

1) $\rightarrow$ 2)
Sei $A \subseteq \Sigma^*$ regulär und $M = (\Sigma, Z, \delta, z_0, Z_E)$ ein DFA mit $L(M) = A$. Dann ist R_M eine rechtsinvariante Äquivalenzrelation. Der Index von R_M ist aber höchstens $|Z|$, denn jede nichtleere Äquivalenzklasse von R_M kann mit genau einem Zustand $z \in Z$ identifiziert werden.
$[x]_M = \{y \in \Sigma^* | ((x,y) \in R_M)\} = \{y \in \Sigma^* | \delta^*(z_0, x) = \delta^*(z_0, y)\}$ wird mit $\delta^*(z_0, x)$ identifiziert.
Nun ist A gerade die Vereinigung derjenigen Äquivalenzklassen, die den Zustand aus Z_E entsprechen. $A = \bigcup_{\delta^*(z_0,x) \in Z_E} [x]_M$

2) $\rightarrow$ 3)
Sei R eine rechtsinvariante Äquivalenzrelation und gelte 2). Wir zeigen, dass R eine Verfeinerung von R_A ist, also:

$\bigwedge_{x,y \in \Sigma^*} (x,y) \in R \Rightarrow (x,y) \in R_A$ gilt 

Damit hatte R_A höchstens so viele Klassen wie R.
Sei $(x,y) \in R \xRightarrow[\text{rechtsinvariant}]{} \bigwedge_{z \in \Sigma^*} (xz, yz) \in R$
$\Rightarrow \bigwedge_{z \in \Sigma^*} xz \in [xz]_R$ und $yz \in [xz]_R$
$\Rightarrow \bigwedge_{z \in \Sigma^*} xz \in A \Leftrightarrow yz \in A$(denn A ist Vereinigung von Klassen von R)
$\Rightarrow (x,y) \in R_A$

3) $\rightarrow$ 1)
R_A habe endlichen Index.
Sei $[x]_A = \{y \in \Sigma^* | (x,y) \in R_A\}$ für alle $x \in \Sigma^*$
$\hookrightarrow Z = \{[x] | x \in \Sigma^*\}$ ist eine Menge
$Z_A = \{[x] | x \in A\}$ ebenfalls

Wir betrachten den DFA $M = (\Sigma, Z, \delta, [\lambda], Z_A)$ wobei $\delta([x], a) = [xa]$ für alle $x \in \Sigma^*, a \in \Sigma$
Es gilt nun $A = L(M)$, da $x \in A \Leftrightarrow [x] \in Z_A \Leftrightarrow \delta^*([x], \lambda) \in Z_A$■

Bemerkung:

- im Beweis wird der Äquivalenzklassenautomat konstruiert
- Äquivalenzklassenautomaten sind immer Minimalautomaten
- es gibt keinen Automaten, der dieselbe Sprache akzeptiert und weniger Zustände hat

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

- Satz von Myhill-Nerode kann sowohl zum Beweis von Regularität als auch zum Beweis von Nichtregularität genutzt werden.

Beispiel

$A_1 = \{w \in \{a,b\}^* \,|\#_a(w) \equiv_3 0\}$
Behauptung:
$K_1 = [\lambda] = \{w \in \{a,b\}^* \,|\#_a(w) \equiv_3 0\}$
$K_2 = [a] = \{w \in \{a,b\}^* \,|\#_a(w) \equiv_3 1\}$
$K_3 = [aa] = \{w \in \{a,b\}^* \,|\#_a(w) \equiv_3 2\}$
z.z. $x, y \in K_i \quad i = 1,2,3 \Rightarrow (x,y) \in R_{A_1}$
$x \in K_i, y \in K_j \quad i \neq j \Rightarrow (x,y) \notin R_{A_1}$
$K_1 \cup K_2 \cup K_3 = \{a,b\}^*$
K_i sind paarweise disjunkt
$\hookrightarrow$ Index $R_{A_1} = 3 \rightarrow A_1$ regulär

$A_2 = \{a^n b^n | n \geq 1\}$
Behauptung:
Jedes $[a^i] \quad i \in \mathbb{N}$ bildet eine Äquivalenzklasse
z.z. $\left(a^i, a^j\right) \notin R_{A_2}$ für $i \neq j$
sei $z = b^i$
Dann gilt: $a^i z = a^i b^i \in A_2$
$a^j z = a^j b^i \notin A_2$
$\hookrightarrow$ Index $(R_{A_2}) = \infty$
$\hookrightarrow$ A_2 ist nicht regulär

Bemerkung:
Zum Beweis der Nichtregularität einer Sprache müssen nicht alle Äquivalenzklassen bestimmt werden, es genügt z.z., dass es unendlich viele Klassen gibt.

Beispiel

Für komplette Äquivalenzklassenzerlegung von Σ^* durch eine Sprache
$A = \{w \in \{a,b\}^* \,|\#_a(w) = \#_b(w)\}$
$[\lambda] = \{w \in \{a,b\}^* \,|\#_a(w) = \#_b(w)\}$
$\left[a^i\right] = \{w \in \{a,b\}^* \,|\#_a(w) - \#_b(w) = i\} \qquad i \geq 1$
$\left[b^i\right] = \{w \in \{a,b\}^* \,|\#_b(w) - \#_a(w) = i\} \qquad i \geq 1$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

2.6 Abschlusseigenschaften

Definition 2.6.1

Sei $f : \gamma(\Sigma^*)^k \to \gamma(\Sigma^*)$ eine Funktion. Eine Sprachfamilie $\zeta \subseteq \gamma(\Sigma^*)$ heißt abgeschlossen bezüglich f (oder Anwendung von f) wenn gilt:

$$\bigwedge_{A_1,\ldots,A_k \subseteq \Sigma^*} \left(\bigwedge_{1 \leq i \leq k} A_i \in \zeta \right) \Rightarrow f(A_1, \ldots, A_k) \in \zeta$$

SATZ 2.6.2

Die Menge der regulären Sprachen REG ist abgeschlossen bezüglich:

1) Vereinigung
2) Durchschnitt
3) Komplement
4) Produkt
5) Kleene Abschluss
6) Differenz

Beweis

1), 4), 5) folgen unmittelbar aus den Beweis von Satz 2.3.4
3) Sei $A \in$ REG und $M = (\Sigma, Z, \delta, z_0, Z_E)$ ein DFA mit $L(A) = M$. Dann gilt für $M' = (\Sigma, Z, \delta, z_0, Z \setminus Z_E)$ gerade $L(M') = \overline{A} = \Sigma^* \setminus A$
2) Wegen $A \cap B = \overline{\overline{A} \cup \overline{B}}$ und 1) und 3)
6) Wegen $A \setminus B = A \cap \overline{B}$ und 2) und 3)

direkter Beweis für 2)
Sei $A = L(M_1)$ mit $M_1 = (\Sigma, Z_1, \delta_1, z_{0_1}, Z_{E_1})$ und $B = L(M_2)$ mit $M_2 = (\Sigma, Z_2, \delta_2, z_{0_2}, Z_{E_2})$.
Wir betrachten $M = (\Sigma, Z_1 \times Z_2, \delta, (z_{0_1}, z_{0_2}), Z_{E_1} \times Z_{E_2})$
wobei $\delta((z_1, z_2), a) = (\delta_1(z_1, a), \delta_2(z_2, a))$. Es gilt $L(M) = A \cap B$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

3 Kontextfreie Sprachen

Für alle Regeln außer λ-Regeln gilt $|p| \leq |q|$ und $R \subseteq N \times (\Sigma \cup N)^*$

Beispiel

$L = \{a^n b^n | n \geq 1\}$
$G = (\{a, b\}, \{S\}, S, \{S \to ab | aSb\})$
Menge der Palindrome $L = \left\{w \in \{a, b\}^* | w = w^R\right\}$
$G = (\{a, b\}, \{S, X\}, S, R), R = \{S \to \lambda | X, X \to aXa | bXb | a | b | aa | bb\}$

SATZ 3.0.1

REG $\subsetneq$ CF

Beweis

Beweis $L = \{a^n b^n | n \geq 1\}$ ist nicht regulär, aber kontextfrei.

3.1 Die Chomsky-Normalform

Definition 3.1.1

Eine Regel der Form $A \to B$ $A, B \in N$ heißt Kettenregel.

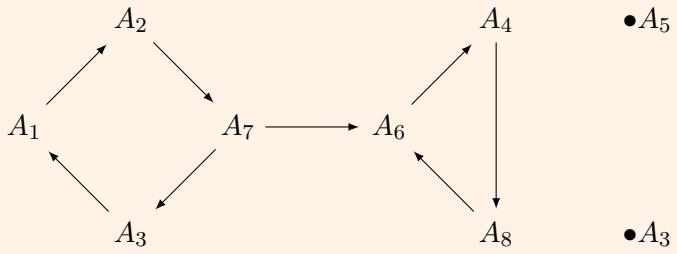
SATZ 3.1.2

Zu jeder kontextfreien Grammatik G gibt es eine äquivalente kfG G' ohne Kettenregel.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

Sei $G = (\Sigma, N, S, R)$ eine kfG. Die Kettenregeln von G definieren einen gerichteten Graph auf der Knotenmenge N.



1) Entfernen aller Zyklen
$A_{i_1} \to A_{i_2}, A_{i_2} \to A_{i_3}, \ldots, A_{i_{k-1}} \to A_{i_k}, A_{i_k} \to A_{i_1}$ mit $A_{i_j} \in N$. Indem all diese Regeln aus R entfernt werden und in den verbleibenden Regeln A_{i_j} $1 \leq j \leq k$ durch A_i ersetzt werde.

2) Wir nummerieren die verbleibenden Nichtterminale so $\{B_1, B_2, \ldots, B_k\}$ dass aus $B_i \to B_j$ stets $i < j$ folgt.

3) Für $i = l-1, l-2, \ldots$ ersetzen wir $B_i \to B_j \quad i < j$ so sind die Regeln mit B_j als linker Seite. $B_j \to u_1 \,|u_2| \ldots |u_m$ so entferne $B_i \to B_j$ und füge $B_i \to u_1 \,|u_2| \ldots |u_m$ hinzu.

Definition 3.1.3

Eine kfG $G = (\Sigma, N, S, R)$ heißt Grammatik in Chomsky-NF, wenn jede Regel aus R folgende Form hat:

- $A \to BC$ $\qquad A, B, C \in N, A \neq B, A \neq C$
- $A \to a$ $\qquad A \in N, a \in \Sigma$
- $S \to \lambda$ in diesem Fall darf S in keiner Regel auf der rechten Seite vorkommen

SATZ 3.1.4

Jede kontextfreie Sprache kann durch eine Grammatik in Chomsky-NF erzeugt werden.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

Sei $A \in$ CF. Gemäß Satz 3.1.2 gibt es ein kfG $G = (\Sigma, N, S, R)$ ohne Kettenregel mit $\mathcal{L}(G) = A$. Wir formen G zu G' in Chomsky-NF um.

1) Für jedes $a \in \Sigma$ führen wir ein neues Nichtterminal C_a ein $\{C_a | a \in \Sigma\} \cap N = \emptyset$
2) In jeder Regel von R wird a durch C_a ersetzt.
3) Alle Regeln $C_a \to a, a \in \Sigma$ werden hinzugefügt.
4) Nun kann es Regeln der Form $A \to B_1 B_2 \dots B_m, m \geq 2$ geben.

Für jede solche Regel führen wir m-1 neue Nichtterminale $D_1, D_2, \dots, D_{m-1}$ ein.
Wir ersetzen $A \to B_1 B_2 \dots B_m$ durch
$A \to B_1 D_1, D_1 \to B_2 D_2, D_2 \to B_3 D_3, \dots, D_{m-1} \to B_m D_m$. Die nun entstandene Grammatik heiße G'. Sie ist in Chomsky-NF und es gilt $\mathcal{L}(G') = \mathcal{L}(G)$.

3.2 Das Pumping-Lemma für kontextfreie Sprachen

SATZ 3.2.1

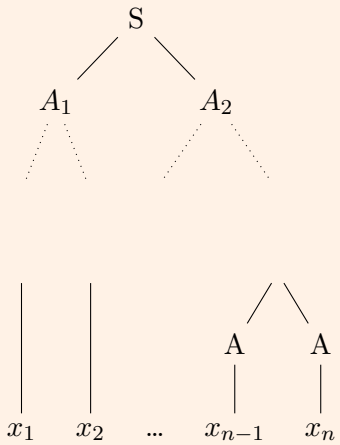
Sei $L \subseteq \Sigma^*$ eine kontextfreie Sprache. Dann gibt es eine Zahl $n \in \mathbb{N}$, sodass sich alle $x \in L$ mit $|x| \geq n$ derart in $x = uv_1\widetilde{v}v_2w$ zerlegen lassen, dass folgende Bedingungen erfüllt sind:

1) $|v_1 v_2| \geq 1$
2) $|v_1 \widetilde{v} v_2| \leq n$
3) Für alle $i \geq 0$ gilt $uv_1^i \widetilde{v} v_2^i w \in L$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

Sei $L \subseteq \Sigma^*, L \in$ CF. Sei $G = (\Sigma, N, S, R)$ eine Grammatik in Chomsky-NF, die L erzeugt. Wir wählen $n = 2^{|N|}$ Sei $x \in L$ mit $|x| \geq n$. Der Syntaxbaum von x sieht so aus:



Dieser Baum hat $|x| \geq 2^{|N|}$ Blätter. Folglich muss es einen Pfad geben, der mindestens Länge $|N|$ hat.

Lemma 3.2.2

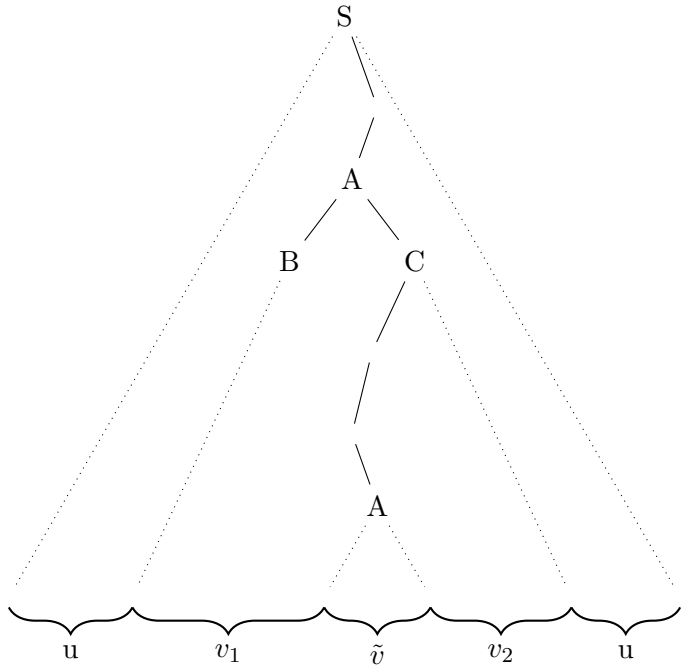
Jeder Binärbaum mit mindestens 2^k Blättern enthält einen Pfad der Länge $\geq k$.

Beweis

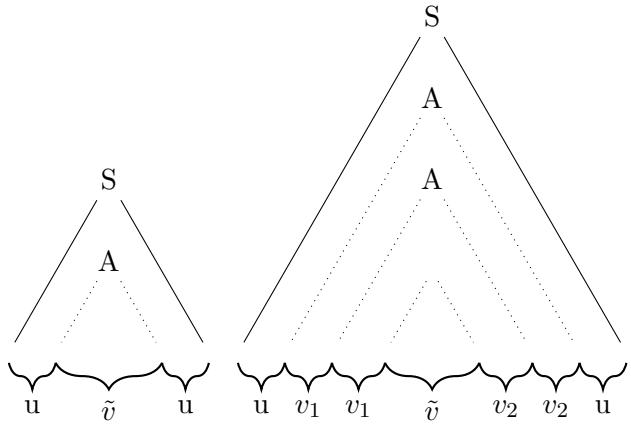
$k = 0$. Ein Baum mit $2^0 = 1$ Blättern enthält einen Pfad der Länge 0.
$k \to k + 1$. Sei T ein Binärbaum mit 2^{k+1} Blättern. Mindestens ein Teilbaum der Wurzel hat mindesten 2^k Blätter. In ihm gibt es nach IV einen Pfad der Länge k. Der wird zur Wurzel um eins verlängert.

Wir fixieren einen Pfad p maximaler Länge. p hat mindestens $|N|$ Kanten und $|N| + 1$ Knoten. $\to$ Auf p kommt ein Nichtterminal doppelt vor.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by



Wir fixieren ein solches Nichtterminal, indem wir auf p von unten nach oben nach dem ersten NT suchen das doppelt vorkommt. ↪ Das obere Vorkommen von A ist höchsten $|V|$ Kanten von der Blattebene entfernt. Wir betrachten nun die Teilwörter, die aus den A's abgeleitet werden können. Das gibt uns eine Zerlegung von x.
Das das obere A höchstens $|N|$ Kanten von der Blattebene entfernt ist,
folgt $|v_1\widetilde{v}v_2| \leq 2^{|N|} = n$. Aufgrund des Doppelvorkommen von A kann der Ableitungsbaum so modifiziert werden.



Das ergibt Ableitungsbäume für $uv_1\widetilde{v}v_2w \in L$ für alle $i \geq 0$.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beispiel

1) $L = \{a^n b^n c^n | n \geq 1\}$
Annahme: $L \in$ CF
Sei m die Pumpingzahl
Wir wählen $x = a^m b^m c^m, |x| = 3m \geq m$
Sei $x = uv_1\tilde{v}v_2w$ eine geeignete Zerlegung gemäß PL,
d.h. $|v_1v_2| \geq 1$ und $|v_1\tilde{v}v_2| \leq m$
Wegen $|v_1\tilde{v}v_2| \leq n$ kann v_1v_2 nicht a's, b's und c's enthalten.
Wegen $|v_1v_2| \geq 1$ ist $v_1v_2 \neq \lambda$
Damit ist $uv_1^0\tilde{v}v_2^0w \notin L$, da $u\tilde{v}w$ nicht gleich viele a's, b's und c's enthält.
$\Rightarrow L \notin CF$

2) $L = \left\{0^{n^2} \middle| n \geq 1\right\} \notin$ CF
Annahme $L \in$ CF
Sei m die Pumpingzahl, wähle $x = 0^{m^2}$
Sei $x = uv_1\tilde{v}v_2w$ eine geeignete Zerlegung gemäß PL,
d.h. $|v_1v_2| \geq 1$ und $|v_1\tilde{v}v_2| \leq m$
$\hookrightarrow v_1v_2 = 0^k, 1 \leq k \leq m$
$m^2 < |uv_1^2\tilde{v}v_2^2w| = |uv_1\tilde{v}v_2w| + |v_1v_2| = m^2 + |v_1v_2| \leq m^2 + m < (m+1)^2$

3.3 Abschlusseigenschaft

SATZ 3.3.1

Die Menge der CF ist abgeschlossen bezüglich:

1) Vereinigung
2) Produkt
3) Kleene-Abschluss (Stern)
4) Schnitt mit regulärer Menge

CF ist nicht abgeschlossen bezüglich:

5) Durchschnitt
6) Komplement
7) Differenz

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

Seien $A, B \in$ CF und $G_1 = (\Sigma, N_1, S_1, R_1), G_2 = (\Sigma, N_2, S_2, R_2)$ kontextfreie Grammatiken mit $N_1 \cap N_2 = \emptyset$ und $\mathcal{L}(G_1) = A, \mathcal{L}(G_2) = B$.

1) $\mathcal{L}((\Sigma, N_1 \cup N_2 \cup \{S\}, S, R_1 \cup R_2 \cup \{S \to S_1, S \to S_2\})) = A \cup B$

2) $\mathcal{L}((\Sigma, N_1 \cup N_2 \cup \{S\}, S, R_1 \cup R_2 \cup \{S \to S_1 S_2\})) = A \cdot B$

3) $\mathcal{L}((\Sigma, N_1 \cup \{S\}, S, R_1 \cup \{S \to \lambda, S \to S_1, S \to S_1 S_1\} \setminus \{S_1 \to \lambda\})) = A^*$

5) Wir betrachten $L_1 = \{a^i b^j c^j | i, j > 0\}$ und $L_2 = \{a^i b^i c^j | i, j > 0\}$
$L_1, L_2 \in$ CF, denn Übungsaufgabe … aber
$L_1 \cap L_2 = \{a^n b^n c^n | n > 0\} \notin$ CF

6) Wenn CF bezüglich Komplements abgeschlossen, so müsste CF wegen
$A \cap B = \overline{\overline{A} \cup \overline{B}}$ auch abgeschlossen bezüglich Durchschnittes sein ↯

7) $A \cap B = A \cup B \setminus ((A \setminus B) \cup (B \setminus A))$

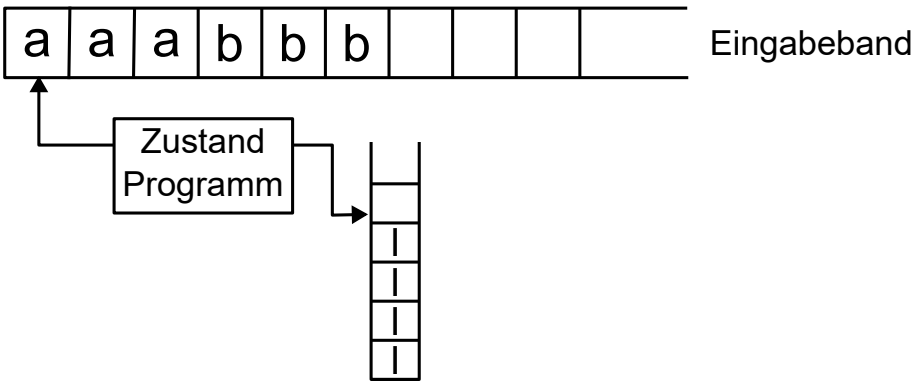
3.4 Kellerautomaten (Push-Down-Automaten)

$\{a^n b^n | n \geq 0\} \in$ CF $\to$ Zähler wird gebraucht $\to$ *Stack*

Definition 3.4.1

Ein nichtdeterministischer Kellerautomat (PDA) M ist ein 6-Tupel $M = (\Sigma, \Gamma, Z, \delta, z_0, Z_E)$ mit folgenden Eigenschaften:

1) Σ ist eine endliche Menge (Eingabealphabet)
2) Γ ist eine endliche Menge (Kelleralphabet, $\square \in \Gamma$)
3) Z ist eine endliche Menge (Zustandsmenge)
4) $\delta : Z \times (\Sigma \cup \{\lambda\}) \times \Gamma \to \mathcal{P}_{endlich}(Z \times \Gamma^*)$
5) $z_0 \in Z$ (Startzustand)
6) $Z_E \subseteq Z$ (Endzustandmenge)



studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Eingabeband:

- nur lesen
- jedes Symbol kann nur einmal gelesen werden
- am Anfang steht hier das Eingabewort

Keller:

- lesen und schreiben
- Topelement des Stack wird gelesen und durch ein Wort $w \in \Gamma^*$ ersetzt, neues Topelement ist der erste Buchstabe von w
- bei Arbeitsbeginn steht hier $\square$

Bemerkung:

1) $\delta(z, a, A) = \{(z_1, B_1 \dots B_k), (z_2, \lambda), (z_3, BA)\}$
   - $(z_1, B_1 \dots B_k)$ ... A wird durch $B_1 B_2 \dots B_k$ ersetzt
   - (z_1, λ) ... A wird entfernt
   - (z_1, BA) ... neues Symbol B wird aufgesetzt
2) $\delta(z, \lambda, A) = \{(z_1, B_1 \dots B_k), (z_2, \lambda), (z_3, BA)\}$ ist auch erlaubt. Veränderung ohne dass ein Symbol der Eingabe gelesen wird.
3) Nur ein Startzustand ist keine Einschränkung $\delta(z_0, \lambda, \square) = \{(z_0', \square), (z_0'', \square), (z_0''', \square)\}$ könnte man nutzen, um mit mehreren Startzuständen zu operieren.

Die von einem PDA akzeptierte Sprache ist die Menge aller Wörter, bei deren Eingabe der PDA in einem Zustand Z_E enden kann.

Definition 3.4.2

Sei $M = (\Sigma, \Gamma, Z, \delta, z_0, Z_E)$ ein PDA. Eine Konfiguration K von M ist ein Tripel $K \in Z \times \Sigma^* \times \Gamma^*$. Dabei ist für $K = (z, u, v)$:

- z ... der aktuelle Zustand
- u ... der noch nicht gelesenes Teil der Eingabe
- v ... der aktuelle Kellerinhalt

Anfangskonfiguration $(z_0, w, \square)$ $\quad w \in \Sigma^*$
akzeptierende Endkonfiguration (z, λ, p) $\quad z \in Z_E, p \in \Gamma^*$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Definition 3.4.3

Wir definieren auf $Z \times \Sigma^* \times \Gamma^*$ (Menge der Konfigurationen) eine binäre Relation (Nachfolgekonfiguration) wie folgt:

$$(z,u,v) \vdash_M (z',u',v')$$

$$\leftrightarrow_{def} \bigvee_{x \in \Sigma \cup \{\lambda\}} \bigvee_{y \in \Gamma \cup \{\lambda\}} \bigvee_{y' \in \Gamma^*} \bigvee_{r \in \Gamma^*} u = xu' \wedge v = yr \wedge v' = y' \wedge \left(z',y'\right) \in \delta\left(z,x,y\right)$$

Analog zu $\vdash_G^*$ definiert man $\vdash_M^*$ als die reflexive und transitive Hülle von $\vdash_M$
$K \vdash_M^* K' \leftrightarrow_{def}$ es gibt endliche Konfigurationen $K_1, \ldots, K_n$ mit
$K \vdash_M K_1, K_1 \vdash_M K_2, \ldots, K_{n-1} \vdash_M K_n$ und $K_n \vdash_M K'$

Definition 3.4.4

Die von einem PDA akzeptierte Sprache ist definiert als:

$$L\left(M\right) = \left\{ w \in \Sigma^* \middle| \bigvee_{z \in Z_E} \bigvee_{p \in \Gamma^*} \left(z_0, w, \square\right) \vdash_M^* \left(z, \lambda, p\right) \right\}$$

Beispiel

$$L = \left\{a^n b^n | n \geq 1\right\}, M = \left(\left\{a,b\right\}, \left\{\square, A\right\}, \left\{z_0, z_1, z_{ende}\right\}, \delta, z_0, \left\{z_{ende}\right\}\right)$$

$$\begin{aligned}
(\delta)\ \left(z_0, a, \square\right) &\to \left(z_0, A\square\right) \\
\left(z_0, a, A\right) &\to \left(z_0, AA\right) \\
\left(z_0, b, A\right) &\to \left(z_1, \lambda\right) \\
\left(z_1, b, A\right) &\to \left(z_1, \lambda\right) \\
\left(z_1, \lambda, \square\right) &\to \left(z_{ende}, \square\right)
\end{aligned}$$

$$L = \left\{w\$w^R \middle| w \in \left\{a,b\right\}^*\right\}, M = \left(\left\{a,b,\$\right\}, \left\{A,B,\square\right\}, \left\{z_0, z_1, z_{ende}\right\}, \delta, z_0, \left\{z_{ende}\right\}\right)$$

$$\begin{aligned}
(\delta)\ \left(z_0, a, \square\right) &\to \left(z_0, A\square\right) & \left(z_0, b, \square\right) &\to \left(z_0, B\square\right) \\
\left(z_0, a, A\right) &\to \left(z_0, AA\right) & \left(z_0, b, A\right) &\to \left(z_0, BA\right) \\
\left(z_0, a, B\right) &\to \left(z_0, AB\right) & \left(z_0, b, B\right) &\to \left(z_0, BB\right) \\
\left(z_0, \$, \square\right) &\to \left(z_1, \square\right) & \left(z_1, a, A\right) &\to \left(z_1, \lambda\right) \\
\left(z_0, \$, A\right) &\to \left(z_1, A\right) & \left(z_1, b, B\right) &\to \left(z_1, \lambda\right) \\
\left(z_0, \$, B\right) &\to \left(z_1, B\right) & \left(z_1, \lambda, \square\right) &\to \left(z_{ende}, \square\right)
\end{aligned}$$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Definition 3.4.5

Sei $M = (\Sigma, \Gamma, Z, \delta, z_0, Z_E)$ ein PDA. Wir sagen M akzeptiert leeren Keller, wenn

$$A = L(M) = \left\{ w \in \Sigma^* \middle| \bigvee_{z \in Z} (z_0, w, \square) \vdash_M^* (z, \lambda, \lambda) \right\}$$

Bemerkung: Auf Z_E kann verzichtet werden.

SATZ 3.4.6

Eine Sprache A kann genau dann von einem PDA akzeptiert werden, wenn sie von einem PDA mit leerem Keller akzeptiert werden kann. (ohne Beweis)

Definition 3.4.7

Sei $G = (\Sigma, N, S, R)$. Eine Ableitung $S \vdash_G^* w, S = p_1 \vdash p_2 \vdash \ldots \vdash p_n = w$ heißt Linksableitung $\Leftrightarrow_{def}$ Für alle $i = 1, \ldots, n-1$ gilt

$$\bigvee_{u \in \Sigma^*} \bigvee_{A \in N} \bigvee_{q \in (\Sigma \cup N)^*} \bigvee_{v \in N^*} (p_i = uAv, p_{i+1} = uqv, (A \to q) \in R)$$

oder $(p_1 = S, p_2 = \lambda, (S \to \lambda) \in R)$

Behauptung: Sei $G = (\Sigma, N, S, R)$ mit $\mathcal{L}(G) \in$ CF. $w \in \mathcal{L}(G)$ genau dann, wenn eine Linksableitung $S \vdash_G^* w$ gibt.

Beweis

mit Hilfe von Syntaxbäumen.

SATZ 3.4.8

Eine Sprache ist genau dann kontextfrei, wenn sie von einem PDA akzeptiert werden kann.

studocu

messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

„ $\Rightarrow$ “ Sei $G = (\Sigma, N, S, R)$ eine kfG mit $\mathcal{L}(G) = A$. Wir geben einen PDA $M = (\Sigma, \Gamma, Z, \delta, z_0, Z_E)$ an, der A mit leerem Keller akzeptiert, indem er die Ableitungen von M im Keller simuliert. Der Kellerinhalt repräsentiert die während der Ableitung entstehenden Zeichenketten aus $\Sigma \cup N$.

$\Gamma = \Sigma \cup N \cup \{\square\}$ $\square$ ... unteres Kellerzeichen

$Z = \{z\}$ $(Z_E = \{z\})$

$\delta(z, \lambda, \square) = \{(z, S)\}$

(1) $\delta(z, \lambda, A) = \{(z, \alpha) | A \to \alpha \in R, \alpha \in (\Sigma \cup N)^*\}$

(2) $\delta(z, a, a) = \{(z, \lambda)\}$

$\delta() = \emptyset$ sonst

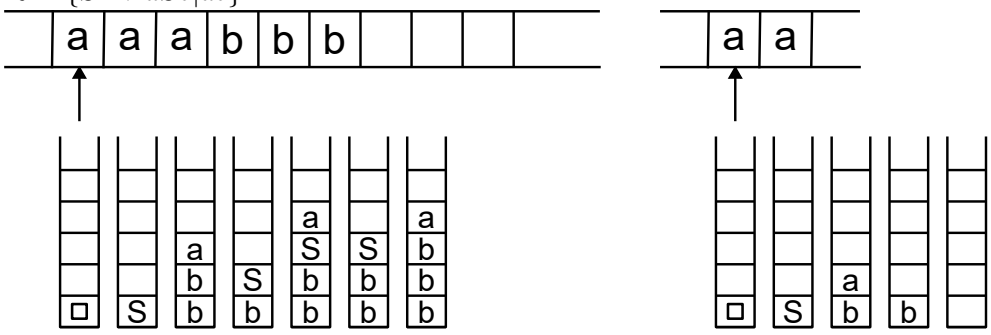
Bemerkung:

(1) Wenn das obere Kellersymbol ein Nichtterminal der Grammatik ist, wird ohne lesen eines Eingabesymbols das Nichtterminal im Keller entsprechend einer Regel aus R ersetzt.

(2) Ist das obere Kellerzeichen ein Terminal und stimmt mit den nächsten Eingabezeichen überein, wird es vom Keller entfernt. (pop)

Beispiel

$R = \{S \to aSb | ab\}$



Es gilt nun für alle $w \in \Sigma^*$

$w \in A \Leftrightarrow w \in \mathcal{L}(G)$

$\Leftrightarrow$ Es gibt eine Linksableitung der Form $S \vdash_G \alpha, \ldots, \alpha \vdash_G w$

$\Leftrightarrow$ Es gibt eine Folge von Konfigurationen von M

mit $(z, w, \square) \vdash (z, w, s) \vdash \ldots \vdash (z, \lambda, \lambda)$

$\Leftrightarrow$ w wird von M mit leerem Keller akzeptiert

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

„ $\Leftarrow$ “ 3.4.8 Sei $A \subseteq \Sigma^*$ und $M = (\Sigma, \Gamma, Z, \delta, z_0, Z_E)$ ein PDA mit

$$A = \left\{ w \in \Sigma^* \middle| \bigvee_{z \in Z} \left((z_0, w, \square) \vdash_M^* (z, \lambda, \lambda) \right) \right\}$$

Diesen gibt es gemäß SATZ 3.4.6

Wir nehmen an, dass für alle $z \in Z, a \in \Sigma, \gamma \in \Gamma$ gilt:

$(z', B_1 \dots B_k) \in \delta(z, a, \gamma) \Rightarrow k \leq 2$

Sollte $k > 2$ gelten, so können wir durch Einfügen neuer Zustände von

$(z, a, \gamma) \to (z', B_1 \dots B_k)$

durch $(z, a, \gamma) \to (z_1, B_{k-1} B_k)$

$(z_1, \lambda, B_{k-1}) \to (z_1, B_{k-2} B_{k-1})$

$(z_{k-2}, \lambda, B_2) \to (z', B_1 B_2)$ erreichen

Wir konstruieren nun einen Grammatik G, die die Rechenschritte von M durch Linksableitungen simuliert.

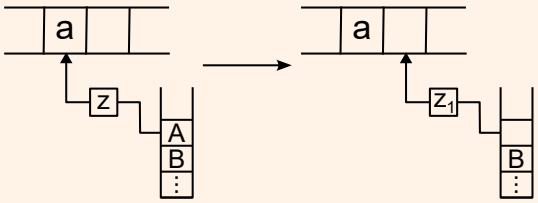
Es sei $G = (\Sigma, N, S, R)$ mit $N = \{S\} \cup Z \times \Gamma \times Z$

$R = \{S \to (z_0, \square, z') | z' \in Z\}$

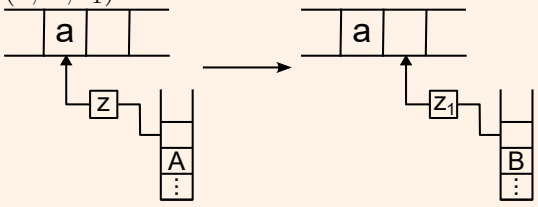
$\cup \{(z, A, z_1) \to a | (z_1, \lambda) \in \delta(z, a, A)\}$

$\cup \{(z, A, z') \to a (z_1, B, z') | z' \in Z, (z_1, B) \in \delta(z, a, A)\}$

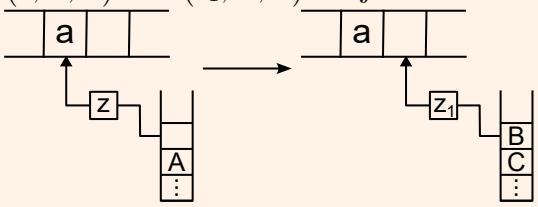
$\cup \{(z, A, z') \to a(z_1, B, z'')(z'', C, z') | z', z'' \in Z, (z_1, BC) \in \delta(z, a, A)\}$



$(z, A, z_1) \to a$



$(z, A, z') \to a (z_1, B, z')$ für jede $z' \in Z$



$(z, A, z') \to a (z_1, B, z'') (z'', C, z')$ für jedes $z', z'' \in Z$

In z_1 wird B und C geschrieben, in z'' B entfernt. Nun ist C Topelement und steht als wäre es gerade geschrieben.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Behauptung: Für alle $z, \widetilde{z} \in N, A \in \Gamma$ und alle $w \in \Sigma^*$ gilt:
$(z, a, \widetilde{z}) \vdash_G^* w$ gdw. $(z, w, A) \vdash_M^* (\widetilde{z}, \lambda, \lambda)$
(*) Offenbar gilt für alle $a \in \Sigma \cup \{\lambda\}$

$$\begin{aligned}(z, A, \widetilde{z}) \vdash_G a &\Leftrightarrow ((z, A, \widetilde{z}) \to a) \in R \\ &\Leftrightarrow (\widetilde{z}, \lambda) \in \delta(z, a, A) \\ &\Leftrightarrow (z, a, A) \vdash_M (\widetilde{z}, \lambda, \lambda)\end{aligned}$$

unmittelbar aus der Definition der Grammatik

Beweis

„ $\Leftarrow$ “ Induktion über n. Zahl der Rechenschritte von M
IA: $n = 1$ siehe (*)
IV: Für alle $z, \widetilde{z} \in Z, w \in \Sigma^*, A \in \Gamma$ gelte
$(z, w, A) \vdash_M^n (\widetilde{z}, \lambda, \lambda) \Rightarrow (z, A, \widetilde{z}) \vdash_G^* w$
IB: Diese Aussage gilt auch für $n + 1$
IS: $n \to n + 1$
Sei $n \geq 1$ und sei $(z, w, A) \vdash^{n+1} (z, \lambda, \lambda)$. Folglich gibt es ein $a \in \Sigma \cup \{\lambda\}$ und $y \in \Gamma^*$ mit $w = ay$ und $(z, ay, A) \vdash_M (z_1, y, \alpha) \vdash_M^+ (\widetilde{z}, \lambda, \lambda)$ für einen Zustand z_1 und Kellerinhalt α. Für alle α gibt es 3 Möglichkeiten.

1. Fall $\alpha = \lambda$
   $\hookrightarrow$ nicht möglich, da(z_1, y, λ) keine Folgekonfiguration hat
2. Fall $\alpha = B, B \in \Gamma$
   $\hookrightarrow$ nach IV ist dann $(z_1, B, \widetilde{z}) \vdash_G^* y$ und außerdem ist
   $(z, A, \widetilde{z}) \to a\,(z_1, B, \widetilde{z})$ eine Regel in R. Setzen wir die zwei Teile zusammen erhalten wir $(z, A, \widetilde{z}) \vdash_G a\,(z_1, B, \widetilde{z}) \vdash_G^* ay$
3. Fall $\alpha = BC, B, C \in \Gamma$
   also $(z, ay, A) \vdash_M^{n_1} (z_1, y_2, C) \vdash_M^{n_2} (\widetilde{z}, \lambda, \lambda)$
   Dabei ist $n_1 + n_2 = n$ und y_2 ein Endwort von $y \left(\bigvee_{y_1 \in \Sigma^*} y = y_1 y_2 \right)$
   Für y_1 gilt weiterhin
   $(z_1, y_1, B) \vdash^{n_1} (z_2, \lambda, \lambda)$
   Nach IV ist somit
   $(z_1, B, z_2) \vdash_G^* y_1$ und $(z, C, \widetilde{z}) \vdash_G^* y_2$
   Außerdem gibt es in R die Regel
   $(z, A, \widetilde{z}) \to a\,(z_1, B, z_2)\,(z_2, C, \widetilde{z})$
   Damit ist
   $(z, A, \widetilde{z}) \vdash_G a\,(z_1, B, z_2)\,(z_2, C, \widetilde{z}) \vdash_G^* ay_1\,(z_2, C, \widetilde{z}) \vdash_G ay_1y_2 = w$
   ein Ableitung in G ■

„ $\Rightarrow$ “ Induktion über K, Länge der Linksableitung von w (ähnlich)

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Aus der Behauptung folgt also:

$w \in L(M) \Leftrightarrow (z_0, w, \square) \vdash_M^* (z, \lambda, \lambda)$ für ein $z \in Z$

$\Leftrightarrow_{\text{Beh.}} (z_0, \square, z) \vdash_G^* w$ für ein $z \in Z$

$\Leftrightarrow S \vdash_G (z_0, \square, z) \vdash_G^* w$ für ein $z \in Z$

$\Leftrightarrow w \in \mathcal{L}(G)$

3.5 Deterministische kontextfreie Sprachen

Definition 3.5.1

Ein Kellerautomat M heißt deterministisch (DPDA) falls für alle $z \in Z, a \in \Sigma$ und $A \in \Gamma$ gilt:
$|\delta(z, a, A)| + |\delta(z, \lambda, A)| \leq 1$
Eine Sprache L heißt deterministisch kontextfrei, falls es einen DPDA M mit $L(M) = L$ gibt.
CF_{det} ... Menge der deterministisch kontextfeiern Sprachen

Beispiel

$$\{a^n b^n | n \geq 1\} \in CF_{det}$$

$$\{w\$w^R | w \in \{a, b\}^*\} \in CF_{det}$$

$$\{ww^R | w \in \{a, b\}^*\} \notin CF_{det}$$

Bemerkung: Nicht alle deterministisch kontextfreien Sprachen können durch einen DPDA mit leerem Keller akzeptiert werden → DPDA's müssen mit Endzustand akzeptieren!

SATZ 3.5.2

Zu jedem DPDA gibt es einen äquivalenten DPDA M' der für jede Eingabe $w \in \Sigma^*$ stets die gesamte Eingabe abarbeitet, also

$$\bigwedge_{w \in \Sigma^*} \bigvee_{z \in Z} \bigvee_{\alpha \in \Gamma^*} (z_0, w, \square) \vdash_{M'}^* (z, \lambda, \alpha)$$

Beweisidee:
Sei $M = (\Sigma, \Gamma, Z, \delta, z_0, Z_E)$ ein DPDA mit $L(M) = L$. Falls M eine Eingabe $w = x_1 \dots x_n$ nicht zu Ende liest, liegt einer der drei folgenden Gründe vor:

1) M gerät in eine Konfiguration mit leerem Keller
2) M gerät in eine Konfiguration $(q, x_i \dots x_n, A\gamma), i \leq n$ in der wegen $\delta(q, x, A) = \emptyset$ keine Anweisung ausführbar ist.
3) M gerät in eine Konfiguration $(q, x_i \dots x_n, A\gamma), i \leq n$ so dass M ausgehend von (q, λ, A) eine unendliche Folge von λ-Sprüngen ausführt.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

zu 1) neues Zeichen # wird auf den Keller platziert, S ist neuer Startzustand
$(S, \lambda, \square) \to (z_0, \square, \#)$

zu 2) neuer Zustand z_f ... Fehler,$\Gamma' = \Gamma \cup \{\#\}$
$(z, a, A) \to (z_f, A)$ für alle z, a, A mit $A = \#$ oder $\delta(z, a, A) = \delta(z, \lambda, A) = \emptyset$
$(z_f, a, A) \to (z_f, A)$ für alle $a \in \Sigma, A \in \Gamma'$

zu 3) Auch das ist zu beheben, wenn die λ-Sprünge umgeleitet werden.

Erinnerung:

- CF ist abgeschlossen bezüglich Vereinigung, Produkt, Kleene-Abschluss, Schnitt mit regulärer Menge
- CF ist nicht abgeschlossen bezüglich Komplement, Durchschnitt, Differenz

SATZ 3.5.3

CF_{det} ist abgeschlossen bezüglich
1) Komplement
2) Schnitt mit regulären Mengen

CF_{det} ist nicht abgeschlossen bezüglich
3) Durchschnitt
4) Vereinigung
5) Differenz
6) Produkt
7) Kleene-Abschluss

zu 1) Idee: $A \in CF_{det}$ und $M = (\Sigma, \Gamma, Z, \delta, z_0, Z_E)$ sei DPDA mit $L(M) = A$, der stets die gesamte Eingabe abarbeitet.
Aus M kann $\bar{M}$ konstruiert werden, der $\bar{A}$ akzeptiert.

zu 2) Es sei $L = L(M_1)$ für einen PDA $M_1 = (\Sigma, \Gamma, Z_1, \delta_1, z_{0_1}, Z_{E_1})$ und sei $L = L(M_2)$ für einen DFA $M_2 = (\Sigma, Z_2, \delta_2, z_{0_2}, Z_{E_2})$. Wir konstruieren den Produktautomaten von M_1 und M_2. Das ist ein PDA der $L_1 \cap L_2$ akzeptiert.
$M = (\Sigma, \Gamma, Z_1 \times Z_2, \delta, (z_{0_1}, z_{0_2}), Z_{E_1} \times Z_{E_2})$ mit $\left((p', q'), \gamma\right) \in \delta((p, q), a, A)$ genau dann, wenn $(p', \gamma) \in \delta_1(p, a, A)$ und $\delta_2(q, a) = q\prime$ für $A \in \Gamma, \gamma \in \Gamma^*$
und $\left((p', q'), \gamma\right) \in \delta((p, q), \lambda, A)$ genau dann, wenn $(p', \gamma) \in \delta_1(p, \lambda, A)$
Man zeigt durch Induktion über K
$((p, q), uv, \gamma) \vdash_M^K ((p', q'), v, \beta)$ genau dann, wenn
$(p, uv, \gamma) \vdash_{M_1}^K (p', v, \beta) und (q, u) \vdash_M^K q\prime$

Bemerkung: falls M_1 deterministisch ist, ist auch M deterministisch. Damit ist sowohl CF als auch CF_{det} abgeschlossen bezüglich Durchschnitt mit regulären Mengen.

zu 3) $L_1 = \left\{a^i b^j c^j \middle| i, j > 0\right\} \in CF_{det}$
$L_2 = \left\{a^i b^i c^j \middle| i, j > 0\right\} \in CF_{det}$
$L_1 \cap L_2 = \{a^n b^n c^n | n \geq 0\} \notin CF$, also ebenfalls $L_1 \cap L_2 \notin CF_{det}$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

zu 4) $A \cap B = \overline{\bar{A} \cup \bar{B}}$
Da CF_{det} abgeschlossen bezüglich Komplement, würde bei Abgeschlossenheit bezüglich $\cup$ Abgeschlossenheit bezüglich $\cap$ folgen.

zu 5) $A \cap B = A \setminus \bar{B}$

zu 6) L_1, L_2 wie oben (siehe 3))
$L_3 = \#L_1 \cup L_2$
Das Vorhandensein von # (oder nicht) sagt uns, ob wir nach einem Wort aus L_1 oder L_2 zu suchen haben
$\{\#\}^* \in \text{REG}$
$\{\#\}^* \cdot L_3 \notin CF_{det}$ (intuitiv, bzz)

zu 7) folgt aus 6

Bemerkung: Die Abschlusseigenschaften ermöglichen uns zu zeigen, dass Sprachen nicht (deterministisch) kontextfrei sind.

Beispiel

$L = \{w \in \{a,b,c\}^* | \#_a(w) = \#_b(w) = \#_c(w)\}$
$A = \left\{a^i b^j c^k | i,j,k \in \mathbb{N}\right\} \in \text{REG}$
$L \cap A = \{a^n b^n c^n | n \in \mathbb{N}\} \notin \text{CF}$
Wäre $L \in \text{CF}$ folgt wegen 2) $L \cap A \in \text{CF}$. Das ist ein Widerspruch $\rightarrow L \notin \text{CF}$

SATZ 3.5.4

$$\text{REG} \subsetneq CF_{det} \subsetneq \text{CF}$$

Beweis

a) $\{a^n b^n | n \geq 1\} \in CF_{det} \setminus \text{REG}$

b) $CF_{det} \subsetneq \text{CF}$ wegen Komplementabgeschlossenheit

3.6 Das Wortproblem für kontextfreie Sprachen

Gegeben: kontextfreie Grammatik G und ein Wort $x \in \Sigma^*$

Frage: Ist $x \in \mathcal{L}(G)$?

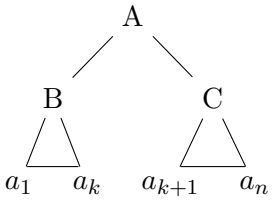
Gibt es einen Algorithmus, der bei Eingabe einer kontextfreien Grammatik $G = (\Sigma, N, S, R)$ und eines Wortes $x \in \Sigma^*$ in endlicher Zeit entscheidet, ob $x \in \mathcal{L}(G)$ oder $x \notin \mathcal{L}(G)$?

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

CYK-Algorithmus (Cocke, Younger, Kasumi)

CYK-Algorithmus
Eingabe: $x = a_1 a_2 \dots a_n$
1. **for** $i = 1$ **to** n **do**
2. $\quad T[i,1] := \{A \in V \mid A \to a_i \in R\}$
3. **end for**
4. **for** $j = 2$ **to** n **do**
5. $\quad$ **for** $i = 1$ **to** $n + 1 - j$ **do**
6. $\quad\quad T[i,j] := \emptyset$
7. $\quad\quad$ **for** $k = 1$ **to** $j - 1$ **do**
8. $\quad\quad\quad T[i,j] := T[i,j] \cup \{A \in V \mid A \to BC \in R \wedge B \in T[i,k] \wedge C \in T[i+k, j-k]\}$
9. $\quad\quad$ **end for**
10. $\quad$ **end for**
11. **end for**
12. **if** $S \in T[1,n]$ **then**
13. $\quad$ print('x liegt in $L(G)$')
14. **else**
15. $\quad$ print('x liegt nicht in $L(G)$')
16. **end if**

- G ist in Chomsky-Normalform gegeben
- Wenn $x = a, a \in \Sigma$ abgeleitet werden kann, dann nur durch $A \to a$ für ein $A \in N$
- für $x = a_1 a_2 \dots a_n, n \geq 2, a_i \in \Sigma$ kann x aus $A \in N$ abgeleitet werden, wenn:
  - eine Regel $A \to BC$ angewendet wurde und
  - es gibt ein $k, 1 \leq k \leq n$ mit $B \vdash_G^* a_1 \dots a_k$ und $C \vdash_G^* a_{k+1} \dots a_n$ (Ein Anfangswort von x wird aus B abgeleitet und das entsprechende Endstück aus C)



- k steht nicht fest, d.h. es müssen alle Werte 1,2,...,n-1 überprüft werden
  ↪ Dynamisches Programmieren
  - alle Teilwörter von x werden (beginnend mit Länge 1) auf eventuelle Ableitbarkeit aus den Nichtterminalen gesucht
  - diese Informationen werden in einer Tabelle abgelegt
  - wenn ein Teilwort der Länge n untersucht wird, stehen die Informationen für alle kurzen Teilwörter schon zu Verfügung
- Notation: für $x = a_1 \dots a_n$ bezeichnet $x_{i,j} = a_i a_{i+1} \dots a_{i+j-1} \dots x_{ij}$ beginnend mit a_i und hat Länge j
- $x_{1,k} = a_1 \dots a_k \quad x_{k+1,n-k} = a_{k+1} \dots a_n$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

- Tabelle T$[1 \ldots n, 1 \ldots n]$
  für $j = 1 \ldots n$ und $i = 1 \ldots n + j - 1$ stehen in $T[i, j]$ alle Nichtterminale, aus denen $x_{i,j}$ abgeleitet werden kann
  $\hookrightarrow$ Eingabewort $x = a_1 \ldots a_n \in \mathcal{L}(G)$ gdw. $\delta(1, n]$

Beispiel

$L = \{0^n 10^m | n, m \in \mathbb{N}, n > m\}$
$G = (\{0, 1\}, \{S, A\}, S, R)$ mit
$R = \{S \to 0S0 \,|A, A \to 0A|\, 01\}$

1) G umformen zu G' in Chomsky-Normalform, Kettenregeln entfernen $(S \to A)$
   $R' = \{S \to 0S0|0A|01, A \to 0A|01\}$

2) „lange rechte Seiten“umformen $S \to 0S0$ ersetzen durch $S \to 0T$ und $T \to S0$

3) Terminale aus Regeln $p \to q$ mit $|q| \geq 2$ entfernen
   $R'' = \{S \to N_0 T \,|N_0 A|\, N_0 N_1$
   $T \to SN_0$
   $A \to N_0 A | N_0 N_1$
   $N_0 \to 0$
   $N_1 \to 1\}$

Satzposition i

Wortlänge j	0	0	0	1	0
1	N_0	N_0	N_0	N_1	N_0
2	$\emptyset$	$\emptyset$	A, S	$\emptyset$	
3	$\emptyset$	A, S	T		
4	A, S	S, T			
5	S, T				

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

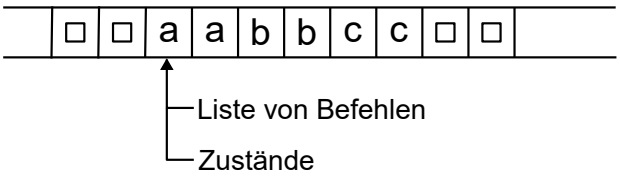
4 kontextsensitiv und L_0-Sprachen

kontextsensitiv oder Typ 1 ... nichtverkürzende Grammatik
$\mathcal{L}_0$ ... erzeugbar durch Grammatik
Maschinenmodell: Wie könnte der Speicher erweitert werden?

4.1 Turingmaschinen

Alan Turing 1936 - Was heißt es etwas zu berechnen?

Ziel: mathematisch klar beschreibbare Maschine, allgemein genug, um jeden beliebigen Berechnungsprozess darzustellen.



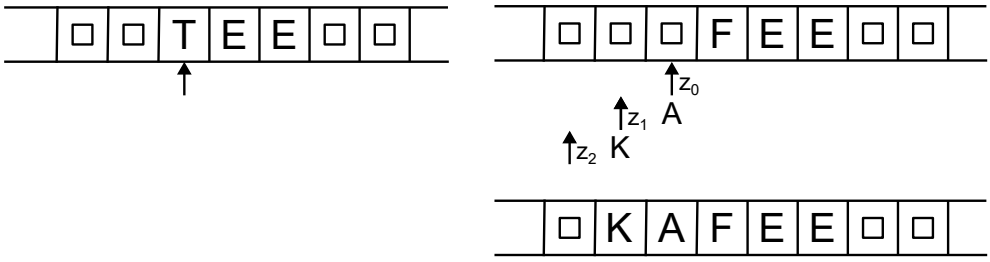
Arbeitsspeicher ist ein Band eingeteilt in Zellen, pro Zelle ein Buchstabe

Arbeitsweise:

- Alle Befehle in der Befehlsliste haben die Gestalt
  $(\text{Zustand}_{alt}, \text{Symbol}_{alt}) \to (\text{Zustand}_{neu}, \text{Symbol}_{neu}, \text{Kopfbewegung})$
- Turingmaschine im Zustand q liest ein a, so wird der Befehl mit (q, a) auf der linken Seite ausgeführt
- $(q, a) \to (r, b, R)$: a wird durch b ersetzt, Zustand geändert, Kopf bewegt sich um eine Zelle nach rechts

Beispiel

$(z_0, T) \to (z_0, F, L)$ $\quad (z_0, \square) \to (z_1, A, L)$
$(z_1, \square) \to (z_2, K, L)$ $\quad (z_2, \square) \to (z_e, \square, R)$



studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

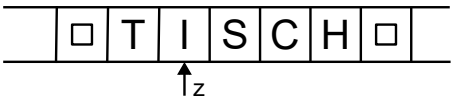
Definition 4.1.1

Ein 7-Tupel $(\Sigma, \Gamma, Z, \delta, z_0, Z_E, \square)$ heißt Turingmaschine (TM) genau dann, wenn:

1) Σ ... endliche Menge
2) Γ ... endliche Menge $\quad \square \in \Gamma \setminus \Sigma$
3) Z ... endliche Menge
4) $\delta : Z \times \Gamma \to \gamma_{endlich}\,(Z \times \Gamma \times \{L, R, N\})$
   DTM (deterministische Turingmaschine) $\to Z \times \Gamma \times \{L, R, 0\}$
5) $z_0 \in Z$
6) $Z_E \subseteq Z$
7) $\square \in \Gamma \setminus \Sigma$ ... Leersymbol

statt $\delta\,(q, a) \in \{(r, b, R), \ldots\}$ schreiben wir auch $(q, a) \to (r, b, R)$

Konfiguration einer TM (Augenblicksbeschreibung)
Was steht auf dem Band?
Wo steht der Kopf?
In welchem Zustand befindet sich die TM?



Konfiguration TIzSCH

- Konfiguration ist ein Wort aus $\Gamma^* Z \Gamma^*$
- Berechnung ist eine endliche Folge von Konfigurationen, die gemäß Überführungsfunktion auseinander hervorgehen

Bemerkung:
In der Konfiguration steht nur der Bandinhalt zwischen linkem und rechtem $\square$

Definition 4.1.2

Sei $M = (\Sigma, \Gamma, Z, \delta, z_0, Z_E, \square)$ eine TM. Wir definieren eine binäre Relation $\vdash_M$ über $\Gamma^* Z \Gamma^*$ (Menge der Konfigurationen von M), wobei $K \vdash_M K'$ gerade bedeuten soll, dass K' in einem Takt aus K hervorgeht (mittels einmaliger Anwendung von δ aus K gewonnen werden kann)
Sei $K = u\alpha u\beta v$ mit $u, v \in \Gamma^*, \alpha, \beta \in \Gamma, z \in Z$

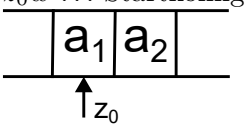
$$K \vdash_M \begin{cases} u\alpha y z' v & \text{falls } z\beta \to z', y, R \\ u z' y \alpha v & \text{falls } z\beta \to z', y, L \\ u\alpha z' y v & \text{falls } z\beta \to z', y, N \end{cases}$$

$\alpha z \beta \in \Gamma^* Z \Gamma^*$
$\alpha z \beta \vdash_M \alpha' z' \beta' \qquad \alpha\prime z\prime \beta\prime$ geht in einem Schritt aus $\alpha z \beta$ hervor
Schreibweise: $\nvdash_K$ ... K hat keine Folgekonfiguration

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Erweiterte Konfigurationsrelation: $\vdash_M^*$

- $\vdash_M^*$ ist die reflexive und transitive Hülle von $\vdash_M$
- $K_1 \vdash_M^* K_2 \ldots K_1$ geht in endlich vielen Schritten in K_2 über
  $K_1 \vdash_M^* K_2$ gdw. $\bigvee_{n \in \mathbb{N}} \bigvee_{Q_1,\ldots,Q_n} K_1 \vdash Q_1, Q_1 \vdash Q_2, \ldots, Q_n \vdash K_2$
- $z_0 w$ ... Startkonfiguration bei Eingabe $w = a_1 \ldots a_n$



Sei $w \in \Sigma^*$. Eine Berechnung von M für w ist eine maximale (nicht verlängerbare) Folge $z_0 w \vdash \alpha_1 z_1 \beta_1 \vdash \alpha_2 z_2 \beta_2 \vdash \ldots$ von Konfigurationen.

Berechnung - 3 mögliche Fälle

- Berechnung ist *akzeptierend*, d.h. sie endet in einer Konfiguration $\alpha_n z_n \beta_n$, wobei $z_n \in Z_E, \alpha_n \beta_n \in \Gamma^*$
- Berechnung ist *verwerfend*, d.h. sie endet in einer Konfiguration $\alpha_n z_n \beta_n$, wobei $z_n \notin Z_E$ und $\delta(z_n, b) = \emptyset$. Dabei ist b das erste Zeichen von β_n
- die Berechnung ist *unendlich*

Bei nichtdeterministische Turingmaschine (NTM) kann es zu jedem Wort mehrere Berechnungen geben.

Berechnungsbäume
Der Berechnungsbaum von $w \in \Sigma^*$ ist ein Baum, dessen maximale Pfade den möglichen Berechnungen entsprechen

- Wurzel ... Startkonfiguration $z_0 w$
- v innerer Knoten, der $\alpha z b \beta$ enthält $(\alpha, \beta \in \Gamma^*, b \in \Gamma)$ und
  $\delta(z, b) = \{(z_1, b_1, X_1), (z_2, b_2, X_2), \ldots, (z_r, b_r, X_r)\}$
  $\hookrightarrow$ v hat genau r Kinder, die jeweils eine Nachfolgekonfiguration von $\alpha z b \beta$ enthalten.

Bemerkung: Berechnungsbäume können akzeptierende, verwerfende als auch unendliche Berechnungen enthalten.

Definition 4.1.3

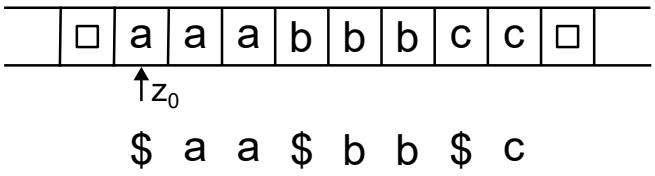
Die von einer NTM akzeptierte Sprache ist
$L(M) = \{w \in \Sigma^* |$ es gibt eine akzeptierende Berechnung von M für w$\}$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Bemerkung: $w \notin L(M)$ gdw. alle Berechnungen sind verwerfend oder unendlich

Beispiel

TM mit $L(M) = \{a^n b^n c^n | n \geq 1\}$
$M = (\{a,b,c\}, \{a,b,c,\$,\square\}, \{z_0,\ldots,z_6,z_e\}, \delta, z_0, \{z_e\}, \square)$



	z_0	z_1	z_2	z_3	z_4	z_5	z_6
a	$z_1\$R$	z_1aR				z_5aL	
b		$z_2\$R$	z_2bR			z_5bL	
c			$z_3\$R$	z_3cR	z_5cL	z_5cL	
$	$z_0\$R$	$z_1\$R$	$z_2\$R$		$z_6\$L$	$z_5\$L$	$z_0\$L$
□				$z_4\square L$		$z_0\square R$	$z_e\square N$

	Absicht
z_0 ... Startzustand	neuer Zyklus
z_1 ... ein a gemerkt	nächstes b suchen
z_2 ... ein a und ein b gemerkt	nächstes c suchen
z_3 ... ein a,b,c getilgt	rechten Rand suchen
z_4 ... rechter Rand erreicht	Test, ob alle c's getilgt sind
z_5 ... Test nicht erfolgreich	zum linken Rand und neuen Zyklus beginnen
z_6 ... alle c's getilgt	prüfe, ob alle a's und b's getilgt

4.2 linear beschränkte Turingmaschine (LBA)

LBA's sind spezielle „TM's", die niemals den Bereich des Bandes verlassen, auf dem die Eingabe steht. Dazu ist es zweckmäßig, sich den rechten und linken Rand der Eingabe zu markieren.
linker Rand: im ersten Arbeitsschritt markieren
rechter Rand: $\Sigma' \cup \Sigma \cup \{\hat{a} | a \in \Sigma\}$
Vereinbarung: Die eigentliche Eingabe wird auf dem Band repräsentiert durch $a_1 a_2 \ldots \hat{a}_n$

Definition 4.2.1

Eine NTM heißt linear beschränkt (LBA), falls für alle Wörter $x = a_1 a_2 \ldots a_n \in \Sigma^*$ und für alle Konfigurationen $\alpha z \beta$ mit $z_0 a_1 \ldots \hat{a}_n \vdash_M^* \alpha z \beta$ gilt $|\alpha\beta| = n$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

SATZ 4.2.2

Eine Sprache ist genau dann kontextsensitiv, wenn sie von einem LBA akzeptiert werden kann.

Beweis

„ $\Rightarrow$ “ sei A kontextsensitiv, also $A = L(G)$ mit $G = (\Sigma, N, S, R)$ vom Typ 1 (nicht verkürzend). Wir beschreiben informal eine Turingmaschine, die A akzeptiert.
Bei Eingabe von $x = a_1 \dots a_n$ wählt M nichtdeterministisch eine Regel $u \to v \in R$ aus. M sucht ein Vorkommen von v auf dem Band und ersetzt es durch u (Falls u kürzer als v werden die restlichen Buchstaben herausgerutscht). Die Maschine stoppt im Endzustand, falls nur noch das Startsymbol s auf dem Band steht.

$x \in L \Leftrightarrow$ es gibt eine Ableitung $S \vdash \dots \vdash X$

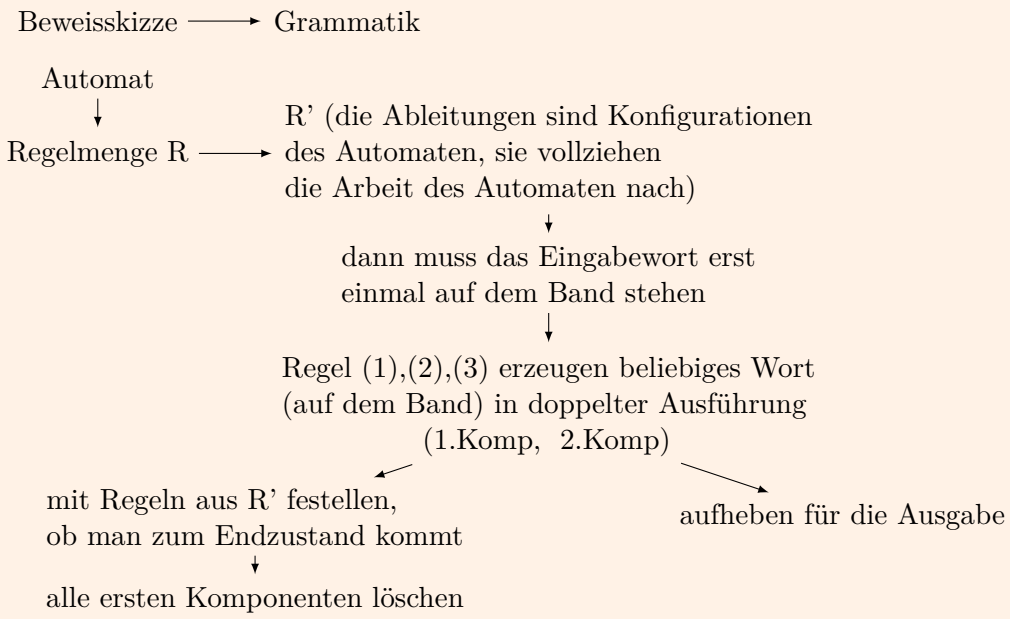
$\Leftrightarrow$ es gibt eine Berechnung von M, die diese Ableitung in umgekehrter Richtung simuliert.

$\Leftrightarrow x \in L(M)$

Da für alle $(u, v) \in R$ gilt $|u| \leq |v|$ ist M linear beschränkt.

„ $\Leftarrow$ “ Sei $A = L(M)$ für ein LBA M
Wir beschreiben eine kontextsensitive Grammatik, deren Zeichenketten Konfigurationen von M darstellen. Konfigurationen (ihre Beschreibungen) dürfen nicht länger als die Eingabe sein.
Sei $\delta = \Gamma \cup (Z \times \Gamma)$
azbcd wird dargestellt als $a(z, b)cd$ $\quad |a(z, b)cd| = 4$ bezogen auf das erweiterte Alphabet.
$\delta(z, a) \ni (z', b, L)$ kann beschrieben werden durch kontextsensitive Regeln der Form $c(z, a) \to (z', c)\, b$ für alle $c \in \Gamma$
Diese Regelmenge sei R'
Falls $\vdash_M^* K' gilt \widetilde{K} \vdash \widetilde{K}'$ mittels R' wobei $\widetilde{K}$ die oben angegebene Darstellung der Konfigurationen ist.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by



Sei $G = (\Sigma, N, S, R)$ mit $N = \{S, A\} \cup \{\Delta \times \Sigma\}$

$$R = \{S \to A\,(\hat{a}, a) | a \in \Sigma\} \tag{1}$$
$$\cup \{A \to A\,(a, a) | a \in \Sigma\} \tag{2}$$
$$\cup \{A \to ((z_0, a), a) | a \in \Sigma\} \tag{3}$$
$$\cup \left\{(\alpha_1, a)\,(\alpha_2, b) \to (\beta_1, a)\,(\beta_2, b) \middle| \alpha_1\alpha_2 \to \beta_1\beta_2 \in R', a, b \in \Sigma\right\} \tag{4}$$
$$\cup \{((z, a), b) \to b | z \in Z_E, a \in \Gamma, b \in \Sigma\} \tag{5}$$
$$\cup \{(a, b) \to b | a \in \Gamma, b \in \Sigma\} \tag{6}$$

Durch (1),(2),(3) sind Ableitungen der Form
$S \vdash^* ((z_0, a_1), a_1)\,(a_2, a_2) \ldots (a_{n-1}, a_{n-1})(\widehat{a_n}, a_n)$ möglich.

1) Komponente: Startzustand
2) Komponente: Eingabewort

$c\,(z, a) \to (z', c)\,b \qquad c \in \Gamma$

Auf der 1. Komponente wird mit R' (Regelart (4)) die Rechnung von M simuliert, bis ein Endzustand erreicht ist.

$\vdash^* (\gamma, a_1) \ldots (\gamma_{k-1}, a_{k-1})\,((z, \gamma_k), a_k), (\gamma_{k+1}, a_{k+1}) \ldots (\gamma_n, a_n)$
mit $z \in Z_E, \gamma_i \in \Gamma, a \in \Sigma$
Danach mittels (5),(6) alle ersten Komponenten löschen. Es bleibt $a_1 \ldots a_n$ übrig. □

SATZ 4.2.3

Eine Sprache ist genau dann von einer Grammatik erzeugbar, wenn sie von einer Turingmaschine akzeptiert werden kann.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

Analog zu Beweisen von SATZ 4.2.2 unter weglassen der linearen Beschränkung.

4.3 Grammatiken

$G = (\{a,b,c\},\{S,A,B,C\},S,R)$

$R = \{S \to aSBC \mid aBC$		$S \vdash aSBC \vdash aaSBCBC \vdash aaaSBCBCBC$
(2)	$CB \to BC$	$\vdash aaaaBCBCBCBC \vdash aaaaBBCCBCBC$
(3)	$\begin{cases} aB \to ab \\ bB \to bb \end{cases}$	$\vdash^5_{(2)} aaaaBBBBCCCC \vdash^4_{(3)} aaaabbbbCCCC$
(4)	$\begin{cases} bC \to bc \\ cC \to cc\} \end{cases}$	$\vdash^4_{(4)} aaaabbbbcccc$

Behauptung: $\mathcal{L}(G) = \{a^n b^n c^n \mid n \geq 1\}$

„ $\subseteq$ “ für jede abgeleitete Zeichenkette $z \in \{a,b,c,A,B,C,S\}^*$
gilt $\#_a(z) + \#_A(z) = \#_b(z) + \#_B(z) = \#_c(z) + \#_C(z)$
$\hookrightarrow$ für jedes abgeleitete Wort $w \in \{a,b,c\}^*$ gilt: $\#_a(z) = \#_b(z) = \#_c(z)$
Reihenfolge der Buchstaben:

- a's entstehen links und werden nie getauscht
- B's können nur zu Terminalen b werden (3), wenn sie über alle C's nach links getauscht werden

„ $\supseteq$ “ Jedes $w = a^n b^n c^n, n \geq 1$ kann erzeugt werden

$(n-1) \times S \to aSBC$	$a^{n-1}S(BC)^{n-1}$
$1 \times S \to aBC$	$a^n(BC)^n$
mit Regel (2)	$a^n B^n C^n$
$1 \times aB \to ab$	$a^n b B^{n-1} C^n$
$(n-1) \times bB \to bb$	$a^n b^n c C^{n-1}$
$1 \times bC \to bc$	$a^n b^n c C^{n-1}$
$(n-1) \times cC \to cc$	$a^n b^n c^n$

$G = (\{a\},\{S,L,R\},S,R)$
$R = \{S \to a \mid aa \mid LSR, Laa \to aaaaL, LaaR \to aaaa\}$
$S \vdash LSR \vdash LLSRR \vdash LLLSRRR$
$\vdash LLLaaRRR \vdash LLaaaaRR \vdash LaaaaLaaRR$
$\vdash LaaaaaaaaR \vdash a^4La^6R \vdash a^8La^4R$
$\vdash a^{12}La^2R \vdash a^{12}a^4 \vdash a^{16}$

$\mathcal{L}(G) = \{a^{2^n} \mid n \in \mathbb{N}\}$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

SATZ 4.3.1

$$\text{CF} \subsetneq \text{CS}$$

Beweis

$\{a^n b^n c^n | n \geq 1\} \in \text{CS} \setminus \text{CF}$

SATZ 4.3.2

$$\text{CS} \subsetneq L_0$$

Beweis

Beweis später.

SATZ 4.3.3

Es gibt Sprachen die von keiner Grammatik erzeugt werden können.

Beweis

Es sei Σ ein Alphabet. Σ^* ist die Menge aller Wörter über Σ ist abzählbar unendlich groß.
$\hookrightarrow \gamma(\Sigma^*)$, die Menge aller Sprachen über Σ ist überabzählbar unendlich groß.
Wie viele Grammatiken $G = (\Sigma, N, S, R)$ gibt es?
N und R sind endliche Mengen und lassen sich damit geeignet kodieren,
z.B. $A1, A2, \ldots, Ak$ und $R1, R2, \ldots, Rl$
Damit ist G als Wort über einem geeigneten Alphabet beschreibbar:
$\widetilde{\Sigma} = \Sigma \cup \{A\} \cup \{0, 1, \ldots, 9\} \cup \rightarrow, (,), ,, j, \ldots$
$\widetilde{\Sigma}^*$, die Menge aller Grammatiken mit Alphabet Σ ist also abzählbar unendlich groß.

4.4 Das Wortproblem für kontextsensitive Sprachen

gegeben: kontextsensitive Grammatik G und ein Wort $w \in \Sigma^*$
Frage: Ist $w \in \mathcal{L}(G)$? Gibt es einen Algorithmus, der bei Eingabe einer kontextsensitiven Grammatik G und eines Wortes $w \in \Sigma^*$ in endlicher Zeit $w \in \mathcal{L}(G)$ oder $w \notin \mathcal{L}(G)$ ausgibt?
JA!

Idee: $|p| \leq |q|$ für $(p, q) \in \delta \hookrightarrow$ für $w \in \mathcal{L}(G), |w| = n$ und $S \vdash_G^* w$ folgt, dass alle $z \in (\Sigma \cup N)^*$, die während der Ableitung von w entstehen, höchstens Länge n haben.
Es gibt nur endlich viele $z \in (\Sigma \cup N)^*, |z| \leq n$
$\hookrightarrow$ Durch systematisches Probieren kann entschieden werden,
ob $w \in \mathcal{L}(G)$ oder $w \notin \mathcal{L}(G)$.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

$T_m^n = \{x \in \{\Sigma \cup N\}^* \,|\, |x| \leq n$ und x lässt sich in höchstens m Schritten aus S ableiten$\}$
$T_m^n n \geq 1 \ldots$ induktiv über m definieren
$T_0^n = \{S\}$
$T_{m+1}^n = Abl_n(T_m^n)$ wobei
$Abl_n(X) = X \cup \left\{x \in \{\Sigma \cup N\}^* \,|\, |x| \leq n \text{ und } w' \vdash w \text{ für ein } w' \in X\right\}$

Bemerkung: das gilt für Typ-1 Grammatiken, bei Typ-0 können Zeichenketten der Länge n auch aus längeren Zeichenketten abgeleitet werden.
$G = (\{a, b, c\}, \{A, B, C, S\}, S, R)$
$R = \{S \to aSBC | aBC, CB \to BC, aB \to ab, bB \to bb, bC \to bc, cC \to cc\}$
gesucht: alle ableitbaren Zeichenketten der Länge höchstens 4
$T_0^4 = \{S\}$
$T_1^4 = \{S, aSBC, abC\}$
$T_2^4 = \{S, aSBC, aBC, aBC\}$
$T_3^4 = \{S, aSBC, aBC, abC, abc\}$
$T_4^4 = T_3^4 = \{S, aSBC, aBC, abC, abc\}$
Es gibt nur endlich viele Zeichenketten der Länge $\leq n$ in $(\Sigma \cup N)^*$
$\hookrightarrow \bigcup_{m \geq 0} T_m^n$ ist eine endliche Menge
$\hookrightarrow$ es gibt $m \geq 0$ so dass gilt: $T_m^n = T_{m+1}^n = \ldots$,da $T_0^n \leq T_1^n \leq \ldots$
Falls w mit$|w| = n$ in $\mathcal{L}(G)$ liegt, muss $w \in \bigcup_{m \geq 0} T_m^n$, also $w \in T_m^n$ für ein m gelten.

Algorithmus

INPUT: $G = (\Sigma, N, S, R)$ kontextsensitiv $w \in \Sigma^*, |w| = n$
OUTPUT: $w \in \mathcal{L}(G)$ oder $w \notin \mathcal{L}(G)$?
berechne $T_0^n, T_1^n, \ldots$ bis $T_{m+1}^n = T_m^n$ gilt
teste ob $w \in T_m^n$, falls ja $w \in \mathcal{L}(G)$, sonst $w \notin \mathcal{L}(G)$

4.5 Abschlusseigentschaften von CS

SATZ 4.5.1

SATZ 4.5.1 CS ist abgeschlossen bezüglich:
1) Vereinigung
2) Produkt
3) Kleene-Abschluss
4) Komplement
5) Durchschnitt

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

$L_1 \subseteq \Sigma_1^*, L_2 \subseteq \Sigma_2^*$
$G_1 = (\Sigma_1, N_1 \cup N_2 \cup S, S_1, R_1)$ mit $\mathcal{L}(G_1) = L_1, N_1 \cap N_2 = \emptyset$
$G_2 = (\Sigma_1, N_1, S_1, R_1)$ mit $\mathcal{L}(G_2) = L_2, \Sigma = \Sigma_1 \cup \Sigma_2$

1) $G = (\Sigma, N_1 \cup N_2 \cup S, R)$

$$R = \begin{cases} \{R_1 \cup R_2 \cup \{S \to S_1, S \to S_2\}\} & \text{, falls } \lambda \notin L_1, \lambda \notin L_2 \\ \{R_1 \cup R_2 \cup \{S \to S_1, S \to S_2\} \cup \{S \to \lambda\}\} & \\ \quad \setminus \{S_1 \to \lambda, S_1 \to \lambda\} & \text{, falls} \lambda \in L_1 \text{ oder } \lambda \in L_2 \end{cases}$$

$\mathcal{L}(G) = L_1 \cup L_2$ und G kontextsensitiv

2) G_1, G_2 Normalformgrammatiken (*)
(aut linken Seiten nur Nichtterminale, für jedes $a \in \Sigma, A_a \to a$ und in allen Regeln A_a durch a ersetzten.)
$G = (\Sigma, N_1 \cup N_2 \cup S, S, R)$
$R = R_1 \cup R_2 \cup \{S \to S_1 S_2\} \setminus \{S_1 \to \lambda, S_1 \to \lambda\} \cup R\prime$

$$R' = \begin{cases} \{S \to S_1, S \to S_2, S \to \lambda\} & \text{, falls } \lambda \in L_1 \text{ und } \lambda \in L_2 \\ \{S \to S_2\} & \text{, falls } \lambda \in L_1 \text{ und } \lambda \notin L_2 \\ \{S \to S_1\} & \text{, falls } \lambda \notin L_1 \text{ und } \lambda \in L_2 \\ \emptyset & \text{, falls } \lambda \notin L_1 \text{ und } \lambda \notin L_2 \end{cases}$$

(*) sonst könnte es in $R_1 \cup R_2$ Regeln $\alpha \to \beta$ geben, die auf ein Teilwort α von $\gamma\delta$ mit $S_1 \vdash_{G_1} \gamma, S_2 \vdash_{G_2} \delta$ anwendbar sind, wobei α die Grenze zwischen γ und δ überragt.
$\mathcal{L}(G) = L_1 L_2$ und G ist kontextsensitiv

3) $G = (\Sigma, N, S, R), \mathcal{L}(G) = L$
$G' = \left(\Sigma, N \cup \{S'\}, S', S \cup \{S' \to \lambda, S' \to SS'\}\right)$
$\mathcal{L}(G') = L^*$ aber G' ist nicht kontextsensitiv (S' kommt rechts vor)
$\hookrightarrow G'' = \left(\Sigma, N \cup \{S', S''\}, S'', R \cup \{S'' \to \lambda, S'' \to S', S' \to S, S' \to SS'\}\right.$
$\left. \setminus \{S \to \lambda\}\right)$

4) Übung

5) $A \cap B = \overline{\bar{A} \cup \bar{B}}$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

5 Berechenbarkeit

5.1 Intuitiver Begriff der Berechenbarkeit

Was ist berechenbar?

$f(m,n) = m+n$
99!
$f(n) = n!$
$g(n) =$ Anzahl der Primfaktoren von $2^{67n}-1$
$h(m,n) = ggT(m,n)$

Eine Funktion $f : \mathbb{N}^k \to \mathbb{N}$ heißt im intuitiven Sinne berechenbar, wenn es einen Algorithmus gibt, der zu beliebig vorgegebenen Argument $(x_1, \ldots, x_k)$ aus dem Definitionsbereich von f nach endlich vielen Schritten den Funktionswert $f(x_1, \ldots, x_k)$ liefert.

Beispiel

$f(x) = x+1, f(x,y) = x+y$
$f(x_1, \ldots, x_n) = (x_{i_1}, \ldots, x_{i_n})$ wobei $x_{i_1}, \ldots, x_{i_n}$ paarweise verschieden und $x_{i_1} \leq x_{i_2} \leq \ldots \leq x_{i_n}$

5.2 Grundlagen

$\mathbb{N} = \{0,1,2,\ldots\}$
$\mathbb{N}^{\mathbb{N}} = \{f : f : \mathbb{N} \vdash \mathbb{N}\}$ „von $\mathbb{N}$ in $\mathbb{N}$“, $Df = \mathbb{N}$
$\widetilde{\mathbb{F}_1} = \mathbb{N}^{\mathbb{N}}, \mathbb{F}_1 = \{f | f : \mathbb{N} \to \mathbb{N}\}$ „aus $\mathbb{N}$ in $\mathbb{N}$“, $Df \subseteq \mathbb{N}$
$\widetilde{\mathbb{F}_2} = \mathbb{N}^{\mathbb{N}\times\mathbb{N}}, \mathbb{F}_2 = \{f | f : \mathbb{N}\times\mathbb{N} \to \mathbb{N}\}$

Menge der totalen beliebig stelligen Funktionen über $\mathbb{N}$:
$$\widetilde{\mathbb{F}} = \bigcup_{i=1}^{\infty} \widetilde{\mathbb{F}_i}$$
Menge der beliebig stelligen Funktionen über $\mathbb{N}$:
$$\mathbb{F} = \bigcup_{i=1}^{\infty} \mathbb{F}_i$$

SATZ 5.2.1

Es gibt Funktionen (bereits in $\widetilde{\mathbb{F}_1}$) die nicht berechenbar sind.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

Behauptung: Es gibt überabzählbar viele Funktionen (in $\widetilde{\mathbb{F}_1}$)

Beweis: Diagonalisierung

Anmerkung: Es gibt nur abzählbar viele Funktionen: $f_0, f_1, f_2, f_3, \ldots$

$$\begin{array}{llllll} f_0 & f_0(0) & f_0(1) & f_0(2) & f_0(3) & f_0(4) & \ldots \\ f_1 & f_1(0) & f_1(1) & f_1(2) & f_1(3) & f_1(4) & \ldots \\ f_2 & f_2(0) & f_2(1) & f_2(2) & f_2(3) & f_2(4) & \ldots \\ f_3 & f_3(0) & f_3(1) & f_3(2) & f_3(3) & f_3(4) & \ldots \\ f_4 & f_4(0) & f_4(1) & f_4(2) & f_4(3) & f_4(4) & \ldots \\ \vdots & & & & & & \end{array}$$

Wir konstruieren die Funktion g:
seien $i, j \in \mathbb{N}, i \neq j$, für alle $k \in \mathbb{N}$

$$g(k) := \begin{cases} i & \text{,falls } f_k(k) \neq i \\ j & \text{, falls } f_k(k) = i \end{cases}$$

g ist somit von allen Funktionen f_i verschieden und kommt in der Folge $f_0, f_1, f_2, f_3, \ldots$ nicht vor. Das ist ein Widerspruch zu der Annahme, dass die Folge alle Funktionen enthält.

Behauptung: Die Menge der berechenbaren Funktionen ist abzählbar.

Beweis

jeder Algorithmus repräsentiert eine berechenbare Funktion
$\hookrightarrow$ es gibt höchstens so viele berechenbare Funktionen wie Algorithmen

- jeder Algorithmus ist als Wort über einem geeigneten Alphabet A darstellbar
- da A^* abzahlbar ist, ist die Menge der Algorithmen abzählbar und somit auch die Menge der berechenbaren Funktionen

---

FAZIT: Das was ein Computer kann ist im Vergleich zu dem, was er nicht kann vernachlässigbar!

Wie sehen sie denn nun aus die nicht berechenbaren Funktionen?

Versuche:

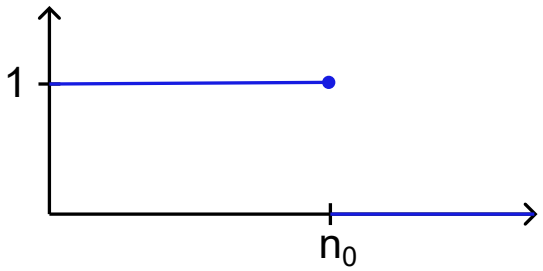
$$f_1(n) = \begin{cases} 1 & \text{, falls n Anfangsabschnitt der Dezimalbruchentwicklung von } \pi \text{ ist} \\ 0 & \text{, sonst} \end{cases}$$

$f_1(314) = 1 \qquad f_1(3) = 1 \qquad f_1(5) = 0$
f_1 ist berechenbar: Näherungsverfahren bis zur gewünschten Genauigkeit

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

$$f_2(n) = \begin{cases} 1 & \text{, falls n irgendwo in der Dezimalbruachentwicklung von } \pi \text{ vorkommt} \\ 0 & \text{, sonst} \end{cases}$$

$$f_3(n) = \begin{cases} 1 & \text{, falls irgendwo in der Dezimalbruchentwicklung von } \pi \\ & \quad \text{gibt es } n \times \text{ hintereinander eine 7} \\ 0 & \text{, sontst} \end{cases}$$



Es gibt einen Algorithmus der f_3 berechnet, auch wenn wir ihn nicht angeben können, weil wir n_0 nicht kennen!
Um zu zeigen, dass gewisse Funktionen nicht berechenbar sind, reicht der intuitive Begriff nicht aus. Wir brauchen eine mathematisch korrekte Definition. Diese benötigt ein Berechnungsmodell.

5.3 Turing-Berechenbarkeit

bisher: Turingmaschine akzeptiert Sprachen

- Wir wollen den Begriff modifizieren, um das Berechnen von Funktionen zu erfassen
- für Berechenbarkeit nutzen wir deterministische Turingmaschinen (DTM)
  - δ ... partielle Funktion
  - $\delta : Z \times \Gamma \to Z \times \Gamma \times \{L, R, N\}$

Vereinbarung: $\alpha z \beta \nvdash$ für alle $\alpha, \beta \in \Gamma^*$ (TM hält, sobald eine Endzustand erreicht ist)

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

	NTM	DTM
Berechnung: maximale (nicht verlängerbare) Folge $z_0w \vdash \alpha_1 z_1 \beta_1 \vdash \alpha_2 z_2 \beta_2 \vdash \dots$ von Konfigurationen	viele Berechnungen für ein Wort möglich	genau eine
Art der Berechnung:	akzeptierend ... endet in Konfiguration mit Endzustand verwerfend ... endet in $\alpha_n z_n \beta_n, z_n \in Z_E, \delta(z_n, b) = \emptyset$ für b erstes Zeichen von β_n unendlich	
M akzeptiert Wort $w \in \Sigma^*$	es gibt einen akzeptierende Berechnung von M für w $\bigvee_{z \in Z_E} \bigvee_{\alpha,\beta \in \Gamma^*} z_0 w \vdash_M^* \alpha z \beta$	Die Berechnung von M für w ist akzeptierend, d.h. sie endet in $z \in Z_E$
$w \notin L(M)$	Alle Berechnungen von M für w sind verwerfend oder endlos	Die Berechnung von M für w ist verwerfend oder endlos

Definition 5.3.1

Eine Funktion $f : \mathbb{N}^i \to \mathbb{N}$ heißt Turingberechenbar gdw.
Es gibt eine deterministische TM
$M = (\{0, 1, 2, \dots, 9\#\}, \Gamma, Z, \delta, z_0, Z_E, \square)$ sodass für alle $(n_1, \dots, n_i) \in \mathbb{N}^i$ gilt

1) $(n_1, \dots, n_i) \in D_f$ gdw. $\bigvee_{z \in Z_E} \bigvee_{\alpha,\beta \in \Gamma^*} z_0 n_1 \# n_2 \# \dots \# n_i \vdash_M^* \alpha z \beta$

2) Für alle $(n_1, \dots, n_i) \in f(n_1, \dots, n_i) = m$ gdw.
$\bigvee_{\alpha \in \Gamma^*} \bigvee_{z \in Z_E} z_0 n_1 \# n_2 \# \dots \# n_i \vdash_M^* \alpha z m$

Definition 5.3.2

Eine Funktion $f : \Sigma^* \to \Delta^*$ heißt Turingberechenbar gdw.
Es gibt DTM $M = \{\Sigma, \Gamma, Z, \delta, z_0, Z_E, \square\}$ mit $\Delta \leq \Gamma \setminus \{\square\}$ sodass für alle $w \in \Sigma^*$ gilt

1) $w \in D_f \Leftrightarrow \bigvee_{z \in Z_E} \bigvee_{\alpha,\beta \in \Gamma^*} z_0 w \vdash_M^* \alpha z \beta$

2) $f(w) = y \Leftrightarrow \bigvee_{\alpha \in \Gamma^*} \bigvee_{z \in Z_E} z_0 w \vdash_M^* \alpha z y$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beispiel

1) Nachfolgefunktion: S: $bin\,(n) \to bin\,(n+1)$
$M = (\{0,1\}, \{0,1,\square\}, \{z_0, z_1, z_2, z_E\}, \delta, z_0, \{z_E\}, \square)$

δ	z_0	z_1	z_2
0	$(z_0, 0, R)$	$(z_2, 1, L)$	$(z_2, 0, L)$
1	$(z_0, 1, R)$	$(z_1, 0, L)$	$(z_2, 1, L)$
$\square$	$(z_1, \square, L)$	$(z_E, 1, N)$	$(z_E, \square, R)$

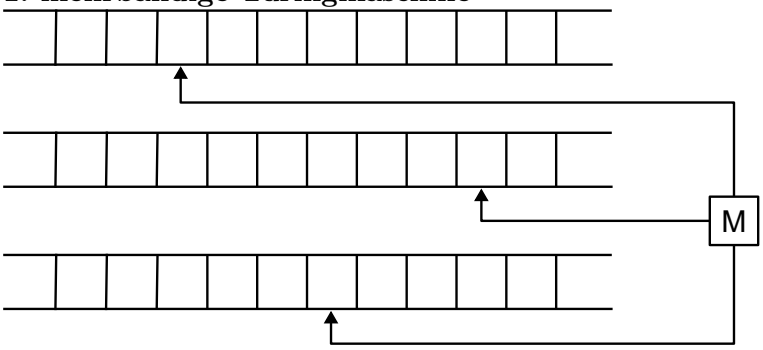
2) nirgends definierte Funktion $f\,(w) = n.d.$
$M = (\{a,b\}, \{a,b,\square\}, \{z_0\}, \delta, \emptyset, \square)$

δ	z_0
a	(z_0, a, R)
b	(z_0, a, R)
$\square$	(z_0, a, R)

5.4 Andere Typen von Turingmaschinen

der bisher eingeführte Typ wird oft als Standard TM bezeichnet
man kann die Definition in verschieden Weisen erweitern

1. mehrbändige Turingmaschine



$\delta : z \times \Gamma^* \to z \times \Gamma^* \times \{L, R, N\}^*$
K ... Anzahl der Bänder
Befehle haben die Form (K=3) $zx_1x_2x_3 \to z'x_1'x_2'x_3'\sigma_1\sigma_2\sigma_3$
x_i ... gelesenes Zeichen auf Band i
x_i' ... zu schreibendes Zeichen auf Band i
σ_i ... Kopfbewegung auf Band i

- die Eingabe steht auf Band 1, die andere Bänder sind leer
- nächster Schritt hängt von aktuelle Zustand und den k Bandinhalten unter dem Lese-Schreib-Kopf ab
- jedes der k Symbole kann überschrieben werden
- jeder Kopf kann sich bewegen (unabhängig von einander)

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Konfiguration einer k-Band TM: $\langle \alpha_1, \alpha_2, \ldots, \alpha_k \rangle \, z \, \langle \beta_1, \ldots, \beta_k \rangle \quad \alpha, \beta \in \Gamma^*, z \in Z$
α_i ... Bandinhalt auf i-ten Band vor dem LS-Kopf
β_i ... Bandinhalt auf i-ten Band ab dem LS-Kopf

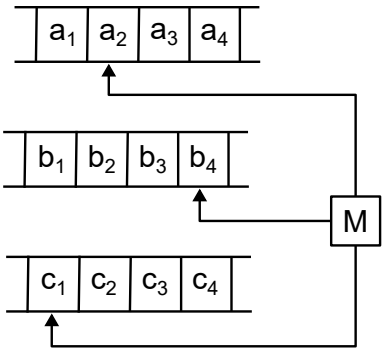
$L = \{a^n b^n c^n | n \geq 1\}$
$k = 3$
- a's und b's auf Band 2 bzw. 3 kopieren
- alle gleichzeitig vergleichen

$k = 2$
- a's auf Band 2 kopieren (als |)
- Anzahl b's und Anzahl | abgleichen
  Band 1 ... von links nach rechts über b's
  Band 2 ... von rechts nach links über |
- Anzahl c's mit Anzahl | vergleichen auf Band 2 wieder von links nach rechts

SATZ 5.4.1

Zu jeder Mehrband TM M gibt es eine (Standard) TM M' mit $L(M) = L(M')$
(bzw. M' berechnet dieselbe Funktion wie M)

Beweisidee:
Sei k die Anzahl der Bänder von M
Wir konstruieren $M' = (\Sigma, \Gamma', Z', \delta', z_0', z_E', \square)$
Das Band M' ist in 2k Spuren unterteilt
$\Gamma' = \Gamma \cup (\Gamma \cup \{*\})^{2k} \qquad * \notin \Gamma$



a_1	a_2	a_3	a_4
	$*$		
b_1	b_2	b_3	b_4
			$*$
c_1	c_2	c_3	c_4
$*$			

in jeder Zelle des Bands steht ein 6-Tupel
Spur $2 \cdot i - 1$ ... Bandinhalt von Band i
Spur $2 \cdot i$ ... Kopfposition von Kopf i
Eingabe $x_1 \ldots x_n \in \Sigma^*$
M' erzeugt Startkonfiguration im Spurendarstellung

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

	x_1	x_2	x_3	x_4		x_n	
□	□						□
	∗						
	□	□	□				
	∗						

Simulation einer Arbeitsschritte von M

- M' führt mehrere Schritte aus, um einen Schritt von M zu simulieren
- LS-Kopf steht zu Beginn links von allen $*$ Markierungen
- M' läuft von links nach rechts, trifft sie auf eine $*$ Zelle, merkt sie sich das gelesene Symbol (dazu werden (viele) neue Zustände benötigt)
- nach dem letzten $*$ sind alle zu lesende Symbole bekannt
  $\hookrightarrow$ M' kann Überführungsregel aus δ anwenden
- auf dem Weg nach links über alle $*$-Markierungen hinweg führt sie die entsprechenden Anweisungen aus

2. einige Bänder können reine Eingabebänder (nur lesen) oder Ausgabebänder sein

3. mehrere Köpfe auf einem Band erlaubt (Vorsicht bei Kollision)

4. Mehrdimensionale Bänder z.B.: $d = 2$
$\delta : Z \times \Gamma \to Z \times \Gamma \times B \qquad B = \{N, S, W, O, H\}$

		x_1	x_2		
	x_3			x_5	
				x_4	

- auch hier ist eine Simulation durch eine (Standard) Turingmaschine möglich
- in jeder Zeile stehen endlich viele Symbole $\hookrightarrow$ der Bandinhalt kann durch ein Rechteck eingegrenzt werden $\hookrightarrow$ Zeile für Zeile auf eindimensionalen Band
- N und S Übergänge haben weiteren Weg auf 1-dim Band
- alle diese Typen können auf einer (Standard) Turingmaschine simuliert werden
- das funktioniert auch bei Nichtdeterministischen Turingmaschinen

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

eine TM kann auf einer TM mit einseitig unendlichen Band simuliert werden

−2	−1	1	2	3	
	a	a	b		

1	2	3	
a	b		
a			
-1	-2	-3	

(a, a) $(b, \square)$

es kann gezeigt werden, dass spezielle Programmierkonzepte (Loop, While, Goto-Berechenbarkeit) äquivalent zur Turing-Berechenbarkeit sind.

5.5 Churchsche These

Frage: Beschreibt der Begriff der Turing-Berechenbarkeit den intuitiven Begriff zufriedenstellend?

- es gibt mehrere verschiedene Berechnungsmodelle
- man kann zeigen, dass alle dieselbe Klasse von Funktionen beschreiben
  ↪ alle anderen Berechnungsmodelle lassen sich durch TM's simulieren

Dann gehen wir davon aus, dass es den intuitiven Begriff beschreibt.
BEWEISEN KÖNNEN WIR ES NICHT!

These 5.5.1 Churchsche These

Die durch die formale Definition der Turing-Berechenbarkeit erfasste Klasse von Funktionen, stimmt genau mit der Klasse der im intuitiven Sinn berechenbaren Funktionen überein.

5.6 Deterministische und nichtdeterministische TM

DTM: $Z \times \Gamma \to Z \times \Gamma \to L, R, N$ partiell

SATZ 5.6.1

Jede von einer nichtdeterministischen Turingmaschine (NTM) akzeptierte Sprache, kann auch von einer deterministischen TM (DTM) akzeptiert werden.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

konstruieren einen DTM T' die die NTM T simuliert

- DTM durchläuft den Berechnungsbaum der NTM für w mit Breitensuche
- gibt es eine akzeptierende Berechnung der NTM (akzeptierendes Blatt) wird die DTM sie finden
  $\hookrightarrow$ DTM hält akzeptierend
- gibt es keine akzeptierende Berechnung, verwirft die DTM (alle Blätter gehören zu verwerfend Berechnungen) oder die Berechnung der DTM ist unendlich, weil es unendlich Pfade im Berechnungsbaum gibt $\mathcal{L}(T) = \mathcal{L}(T')$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

6 Entscheidbarkeit und Aufzählbarkeit

6.1 Entscheidbarkeit

Funktionen: Berechenbarkeit
Mengen (Sprachen): Entscheidbarkeit, Aufzählbarkeit
Es sei $A \subseteq \Sigma^*$. Die charakteristische Funktion c_A von A ist wie folgt definiert

$$c_A(w) = \begin{cases} 1 & w \in A \\ 0 & w \notin A \end{cases}$$

Diese Funktion ist total.

Definition 6.1.1

Eine Menge sei $A \subseteq \Sigma^*$ heißt entscheidbar genau dann, wenn c_A berechenbar ist.

REC (recursive sets) ... Menge aller entscheidbaren Mengen
Bemerkung: c_A ist Turing-Berechenbar bedeutet, es gibt DTM, die für jedes $w \in \Sigma^*$ anhält, 1 ausgibt, falls $w \in A$ und 0 ausgibt falls $w \notin A$.
Wegen Churchsche These heißt das: Es gibt einen Algorithmus der 1 ausgibt, falls $w \in A$ und 0 ausgibt, falls $w \notin A$. Dieser Algorithmus entscheidet A.

Beispiel

$A = \{n \in \mathbb{N} \mid \text{n ist Primzahl}\}$	INPUT: $n \in \mathbb{N}$ Frage: Ist n eine Primzahl?
$B = \{n \in \mathbb{N} \mid \text{n und n+2 sind Primzahl}\}$	INPUT: $n \in \mathbb{N}$ Frage: Ist n und $n+2$ eine Primzahl?

$C = WORT_2 = \{(G, w) \mid \text{G kontextfreie Grammatik und } w \in \mathcal{L}(G)\} \in$ REC (CYK)
INPUT: G kontextfrei, $w \in \Sigma^*$
Frage: $w \in \mathcal{L}(G)$?

$D = \{t : 2^t - 1 \text{ ist Primzahl}\} \in$ REC

$E = \{n \in \mathbb{N} \mid \text{n ist gerade und als Summe zweier Primzahlen darstellbar}\} \in$ REC

SATZ 6.1.2

1) REG $\subseteq$ REC
2) CF $\subseteq$ REC
3) CS $\subsetneq$ REC

Die Frage, ob eine Sprachklasse in REC enthalten ist, ist gleichbedeutend mit der Frage, ob das Wortproblem für die entsprechend Klasse entscheidbar ist.
$WORT_i = \{(G, w) \mid \text{G ist Grammatik vom Typ i und } w \in \mathcal{L}(G)\}$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

(es genügt natürlich 3) zu zeigen, trotzdem:

1) $WORT_3 \in$ REC
   DFA's liefern nach Abarbeitung eines Wortes stets eine Antwort in Form eines akzeptierenden oder nicht akzeptierenden Zustands.
   Sei $A \in$ REG, $A = \mathcal{L}(M)$ für ein DFA M.
   Wir konstruieren eine TM, die bei Eingabe w den DFA simuliert und 1 ausgibt, falls der DFA in einem akzeptierenden Zustand endet und 0 sonst. Diese DTM berechnet c_A.

2) $WORT_2 \in$ REC (CYK-Algorithmus)

3) $WORT_1 \in$ REC
   Abschnitt 4.4. Das Wortproblem für kontextsensitive Sprachen.
   bleibt zu zeigen: CS $\subsetneq$ REG
   Wir konstruieren eine Sprache, die entscheidbar ist, aber nicht kontextsensitiv.
   Diagonalisierung: Es seien $G_1, G_2, \ldots$ alle kontextsensitiven Grammatiken und $w_1, w_2, \ldots$ alle Wörter über einem Alphabet Σ.

$$\text{Tabelleneintrag}\,(i,j) = \begin{cases} 1 & \text{falls } w_j \in \mathcal{L}\,(G) \\ 0 & \text{sonst} \end{cases}$$

	w_1	w_2	w_3	w_4	$\ldots$
G_1	1	0	0		
G_2	0	1			
G_3	1	0	0		
G_4				0	
$\ldots$					

Es sei $C = \{w_i | w_i \notin \mathcal{L}\,(G_i)\}$ „alle Wörter in deren Spalte auf der Hauptdiagonalen 0 steht“
$C \in$ REC, da $WORT_1 \in$ REC
Es gibt $w_j \in \mathcal{L}\,(G_j) \Leftrightarrow w_j \notin C$ für alle $j \in \mathbb{N}$ (*)
Annahme: C ist kontextfrei. Dann gibt es $j \in \mathbb{N}$ mit $\mathcal{L}\,(G_j) = C$. Das ist ein Widerspruch zu (*)

SATZ 6.1.3

REC ist abgeschlossen bezüglich Komplement, Vereinigung, Durchschnitt und Produkt.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

Es seien A, B entscheidbare Sprachen $A, B \in \Sigma^*, A, B \in$ REC.
TM T_A berechnet c_A und TM T_B berechnet c_B
2) konstruieren $T_{A \cup B}$, die $A \cup B$ berechnet

Eingabe: $w \in \Sigma^*$
simuliere T_A auf Eingabe w
if T_A 1 ausgibt **then return** 1
if T_A 0 ausgibt **then**
simuliere T_B auf Eingabe w

6.2 Semi- Entscheidbarkeit und Aufzählbarkeit

Sei $A \subseteq \Sigma^*$. Die semi-charakteristische Funktion von A ist wie folgt definiert.

$$\chi_A(w) = \begin{cases} 1 & \text{falls } w \in A \\ \text{n.d.} & \text{sonst} \end{cases}$$

Definition 6.2.1

Eine Menge $A \subseteq \Sigma^*$ heißt semi-entscheidbar genau dann, wenn χ_A berechenbar ist.

Bemerkung: Aus Def 6.2.1 folgt: A ist semi-entscheidbar gdw. Es gibt TM, die akzeptiert

Die semi-charakteristische Funktion einer Sprache die nicht von einer TM akzeptiert werden kann, ist nicht berechenbar!

Beispiel für semi-entscheidbar

$$A = \left\{ (n,m) \in \mathbb{N}^2 \,\middle|\, \bigvee_{k \geq 1} n^k + m^k \text{ prim} \right\} \qquad INPUT: n, m \in \mathbb{N}$$
Frage: Gibt es $k \geq 1 \quad n^k + m^k$ prim

$$B = \left\{ (x,y,z) \in \mathbb{N}^3 \,\middle|\, \bigvee_{k \in \mathbb{N}, k>2} x^k + y^k = z^k \right\} \qquad INPUT: (x,y,z) \in \mathbb{N}^3$$
Frage: Gibt es $k > 2 \quad x^k + y^k = z^k$

$C = \{n \in \mathbb{N} |$ n ist gerade und als Differenz zweier Primzahlen darstellbar$\}$

Definition 6.2.2

Eine Menge $A \subseteq \Sigma^*$ heißt rekursiv aufzählbar genau dann, wenn entweder

- $A = \emptyset$ oder
- es gibt eine berechenbare, totale Funktion $\mathbb{N} \to \Sigma^*$ mit $W_f = A$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

RE (recursive enumerable) ... Menge aller rekursiven aufzählbaren Mengen

Bemerkung:

- $A = \{f(0), f(1), \ldots\}$ „f zählt A auf“
- $f(i) = f(j)$ für $i \neq j$ zulässig
  Entscheidung ob $w \in A$ ist nicht möglich
  $w \in A$: w kommt in der Aufzählung nach endlich vielen Schritten vor
  $w \notin A$: w kommt nicht in der Aufzählung vor, man kann nach endlich vielen Schritten nicht sagen, dass $w \notin A$
- Aufzählbarkeit ist noch algorithmisch angebbar
- rekursive Aufzählbarkeit und Abzählbarkeit ist nicht das Gleiche!
  A rekursive aufzählbar $\Rightarrow$ A abzählbar
  $\not\Leftarrow$
  Aufzählbarkeit ist die stärkere Bedingung da f berechenbar sein muss
  Für $A = \emptyset$ ist $\emptyset = w_f$ für $f =$ n.d.

SATZ 6.2.3

Eine Sprache $A \subseteq \Sigma^*$ ist rekursiv aufzählbar genau dann, wenn sie semi-entscheidbar ist

Beweis

„ $\Rightarrow$ “ Sei A aufzählbar mittels f. Ein semi-entscheidungsverfahren für A ist.

Eingabe: $w \in \Sigma^*$
  for $i = 1, 2, 3, \ldots$ **do**
    if $f(i) = w$ **then return** 1

„ $\Leftarrow$ “ Wir geben einen Algorithmus an, der eine aufzählende Funktion berechnet
T_A ... TM, die A semi-entscheidet, d.h. χ_A berechnet

$j = 0$
$i = 0$
repeat
  erzeuge $\Sigma^{\leq t} = \{w_1, \ldots, w_l\} = \Sigma^0 \cup \Sigma^1 \cup \ldots \Sigma^t$
  for $m = 1$ to l **do**
    simuliere erste t Takte von T_A auf w_m
    if $\chi_A(w_m) = 1$ ausgegeben wird **then**
      gebe $f(j) = w_m$ aus
      $j = j + 1$
  $t = t + 1$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Behauptung: $W_f = A$
„ $\subseteq$ “ Der Algorithmus gibt nur Wörter aus für die T_A auf Eingabe w 1 ausgibt
„ $\supseteq$ “ $w \in A \hookrightarrow$ Es gibt z_0, sodass T_A angewendet auf w nach z Takten 1 ausgibt
$\hookrightarrow$ Für ein $t \geq z$ wird obiger Algorithmus w ausgegeben

Folgerung 6.2.4

Eine Sprache ist rekursiv aufzählbar genau dann, wenn sie von einer Turingmaschine akzeptiert werden kann.

SATZ 6.2.5

$$\text{RE} = \mathcal{L}_0$$

Beweis

Satz 4.3.3 und Folgerung 6.2.4

$$\mathcal{L}_0 = \left\{ A \bigvee_{TMM} L(M) = A \right\}$$

SATZ 6.2.6

$$\text{REC} \subseteq \text{RE}$$

Beweis

1) Fall: $A = \emptyset \rightarrow A \in$ RE nach Definition

2) Fall: $A \neq \emptyset \rightarrow A \in$ REC
$\hookrightarrow$ Es gibt eine TM M_1, die c_A berechnet
Wir konstruieren eine TM M_2, die χ_A berechnet
M_1 wird modifiziert: Immer, wenn M_1 0 ausgibt, soll M_2 in eine Endlosschleife gehen.

Aus den Sätzen 6.2.6 (REC $\subseteq$ RE), 6.2.5 (RE $= \mathcal{L}_0$) und 6.1.2 (CS $\subsetneq$ REC) ergibt sich der Beweis für Satz 4.1.2 (CS $\subsetneq \mathcal{L}_0$)

Zusammenfassung: Folgende Aussagen sind äquivalent:

- A ist rekursiv aufzählbar
- A ist semi-entscheidbar
- A ist vom Typ 0
- $A = L(M)$ für eine Turingmaschine M
- χ_A ist Turingberechenbar
- A ist Definitionsbereich einer berechenbaren Funktion
- A ist Wertebereich einer berechenbaren Funktion

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

SATZ 6.2.7

RE ist abgeschlossen bezüglich Durchschnitt, Vereinigung und Produkt (Beweis Übungsblatt 10)
Ist RE abgeschlossen bezüglich Komplement?

6.3 Beziehungen zwischen REC und RE

SATZ 6.3.1

Für alle $A \subseteq \Sigma^*$ gilt $A \in \text{RE}$ und $\bar{A} \in \text{RE} \Leftrightarrow A \in \text{REC}$

Beweis

„ $\Leftarrow$ “ Ist $A \in \text{REC}$, so ist auch $\bar{A} \in \text{REC}$. Wegen $\text{REC} \subseteq \text{RE}$ folgt $A \in \text{RE}$ und $\bar{A} \in \text{RE}$.
„ $\Rightarrow$ “ Sei M_A eine TM, die χ_A berechnet und $M_{\bar{A}}$ eine TM die $\chi_{\bar{A}}$ berechnet. Eingabe $w \in \Sigma^*$. Simuliere M_A und $M_{\bar{A}}$ schrittweise parallel auf Eingabe w.

- Falls M_A 1 ausgibt, gebe 1 aus
- Falls $M_{\bar{A}}$ 1 ausgibt, gebe 0 aus

Diese Maschine hält immer an und liefert damit ein Entscheidungsverfahren für A.

6.4 Kodieren von Turingmaschinen über 0,1

$M = (\Sigma, \Gamma, Z, \delta, z_0, Z_E, \square)$
$\Sigma = \{0, 1\}$
$\Gamma = a_0, \ldots, a_k$
$Z = z_0, \ldots, z_n$

Die Nummern der Symbole $0, 1, \square$ seien festgelegt z.B. $a_0 = \square, a_1 = 1, a_2 = 0$
sei $\# \notin \Gamma \cup Z$
Codewort von M über $\{0, 1, \#\}$
$c(M) = \#bin(k)\#bin(n)\#bin(\delta)\#\#code(F)\#$
Für jedes $(i, j) \in \{0, 1, \ldots, n\} \times \{0, 1, \ldots, k\}$ mit $\delta(z_i, a_j) = (z_e, a_m, x)$
sei $code(\delta(i, j)) = \#\#bin(i)\#bin(j)\#bin(l)\#bin(m)\#bin(x)\#\#$

$$\text{mit } bin(x) = \begin{cases} 00 & x = L \\ 01 & x = R \\ 10 & x = N \end{cases}$$

Das Codewort für δ ergibt sich durch Hintereinanderschreiben aller $code(\delta(i, j))$ mit $\delta(z, a_j) \neq$ n.d. in beliebiger Reihenfolge.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Für $F = \{z_{i_1}, \ldots, z_{i_l}\}$ ist $code\,(F) = \#bin(i_1)\#bin\,(i_2)\,\# \ldots \#bin\,(i_l)\,\#$
$c\,(M)$ ... Codewort über 0,1,#
$c\,(M)$ können wir $code\,(M) = \langle M \rangle$Zuordnen: $0 \to 00, 1 \to 01, \# \to 11$

Bemerkung:

- $\langle M \rangle$ ... Kodierung der TM m
- aus $\langle M \rangle$ lässt sich M rekonstruieren (bis auf die Namen der Zustände und Bandsymbole)
- ebenso können Grammatiken, DFA's, NFA's usw. kodiert werden
- nicht jedes $w \in \{0,1\}^*$ ist code einer Turingmaschine

Folgende Sprachen sind entscheidbar:

- $A_{DFA} = \{\langle B, w \rangle |$B ist DFA und B akzeptiert w$\}$
- $E_{DFA} = \{\langle A \rangle |$A ist DFA und $L\,(A) = \emptyset\}$
- $S_{REG} = \{\langle G_1, G_2 \rangle | G_1, G_2$ reguläre Grammatiken und $L\,(G_1) \cap L\,(G_2) = \emptyset\}$
- $EQ_{DFA} = \{\langle A, B \rangle |$A und B sind DFA's und $L\,(A) = L\,(B)\}$
- $WORT_2 = \{\langle G, w \rangle |$G ist kontextfreie Grammatik und G erzeugt w$\}$
- $E_{CF} = \{\langle G \rangle |$G ist kontextfreie Grammatik und $\mathcal{L}\,(G) = \emptyset\}$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

1) Wir konstruieren eine TM, die die Arbeit des DFA's nachvollzieht und entsprechen antwortet.
TM M: Eingabe: $\langle B, w \rangle$
simuliere DFA B auf Eingabe w, falls B akzeptiert, gebe 1 aus, sonst gebe 0 aus

2) Markierungsalgorithmus für die Zustände:
   - Startzustand markieren
   - alle Zustände markieren, die aus markierten Zustand erreichbar sind
   - wiederhole, bis keine neuen Zustände markiert werden
   - teste, ob Endzustände markiert sind

3) DFA's A_1, A_2 mit $L(A_1) = \mathcal{L}(G_1)$ und $L(A_2) = \mathcal{L}(G_2)$
   - konstruieren Produktautomaten aus A_1 und A_2
   - Teste ob $L(A) = \emptyset$ (mit 2))

4) 11. Aufgabenblatt

5) CYK-Algorithmus

6) Es kann eine TM konstruiert werden, die folgenden Algorithmus ausführt.
Markierungsalgorithmus: Es werden alle Nichtterminale markiert aus denen ein Wort abgeleitet werden kann.
   a) forme G in Chomsky-Normalform um
   b) markieren alle Nichtterminale aus Regeln der Form $A \to a$
   c) markiere alle Nichtterminale A aus den Regeln der Form $A \to BC$ falls B und C schon markiert sind
   d) Wiederhole 3., bis keine neuen Nichtterminale markiert werden
   e) Teste, ob das Startsymbol S markiert ist

6.5 Das Halteproblem

6.5.1 Problemstellung und intuitive Argumentation

- Programmierer machen Fehler beim Programmschreiben
  $\hookrightarrow$ Programme gelangen in Endlosschleife und halten nicht
- es wäre von Nutzen, wenn man Programme auf das Vorhandensein von Endlosschleifen untersuchen kann, bevor sie ausgeführt werden

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Goldbachsche Vermutung: Jede gerade Zahl größer als 2 lässt sich als Summe zweier Primzahlen darstellen.
$n = 4$
Wiederhole
 Teste, ob n als Summe zweier Prime darstellbar ist
  Ja $\rightarrow n = n + 2$
  Nein $\rightarrow$ stopp

letzter Satz von Fermat: Es gibt keine $k \leq 2$ für die es $(x, y, z) \in \mathbb{N}^3$ gibt mit $x^k + y^k = z^k$

Fermatsche Primzahlen: Prime der Form 2^{t+1}; bekannte t: 3, 5, 17, 256, 65537

Zunäschst intuitive Argumentation, dass das Halteproblem nicht entscheidbar ist. $A_0, A_1, A_2, \ldots$ Auflistung aller Algorithmen die als Eingabe eine natürliche Zahl haben.

Halteproblemtabelle

Algorithmus \ Eingabe	0	1	2	3	4	5
A_0	J	N	J	J	N	...
A_1	N	N	N	J	J	
A_2	N	J	J	J		
A_3	$\vdots$					
$\vdots$						

$$t(i,j) = \begin{cases} J & A_i \text{ terminiert mit Eingabe j} \\ N & \text{sontst} \end{cases}$$

Annahme: Das Halteproblem ist entscheidbar, d.h. es gibt einen Algorithmus A_H, der bei Eingabe (i, j) den Tabelleneintrag $t(i, j)$ berechnet

Algorithmus A: Eingabe $i \in \mathbb{N}$
 berechnet mit A_k $t(i, j)$
 falls $t(i, j) = J$ gehe in Endlosschleife
 else stopp

Da alle Algorithmen in der Tabelle aufgeführt sind, gibt es $n \in \mathbb{N}$ mit $A_n = A$
$t(n, n) = J$ heißt A_n stoppt auf Eingabe n, andererseits läuft $A = A_n$ für $t(n, n) = J$ endlos ↯
Für $t(n, n) = N$ ergibt sich ebenfalls eine Widerspruch, somit gibt es Algorithmus A nicht und damit kann es auch A_H nicht geben.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

6.5.2 Formale Argumentation mittels Turingmaschinen

Gibt es eine Turingmaschine, die für jede beliebige Turingmaschine entscheiden kann, ob sie mit beliebigen Eingabe anhält oder nicht?

Definition 6.5.2.1

Unter dem allgemeinen Halteproblem verstehen wir die Menge
$A_{TM} = \{\langle M, w\rangle |$ M ist TM und M akzeptiert w$\}$

SATZ 6.5.2.2

$$A_{TM} \notin \text{REC}$$

Beweis

Annahme: Es gibt eine TM H, die A_{TM} entscheidet, d.h.

$$H\left(\langle M, w\rangle\right) = \begin{cases} 1 & \text{falls M w akzeptiert} \\ 0 & \text{falls M w nicht akzeptiert} \end{cases}$$

Dann gibt es auch folgende TM D, die H als Unterprogramm nutzt

D: Eingabe $\langle M\rangle$

1) Simuliere H auf Input $\langle M, \langle M\rangle\rangle$
2) if H $\langle M, \langle M\rangle\rangle = 1$ (d.h. M akzeptiert $\langle M\rangle$)
   gehe in Endlosschleife
   else gebe 1 aus

$$D\left(\langle M\rangle\right) = \begin{cases} 1 & \text{falls } H\left(\langle M, \langle M\rangle\rangle\right) = 0 \\ n.d. & \text{falls } H\left(\langle M, \langle M\rangle\rangle\right) = 1 \end{cases}$$

Die TM D wird nun auf ihren eigenen Code angewendet.

$$D\left(\langle D\rangle\right) = \begin{cases} 1 & \text{falls } H\left(H\left(\langle D, \langle D\rangle\rangle\right) = 0, \text{ d.h. D akzeptiert } \langle 0\rangle \text{ nicht}\right) \\ n.d. & \text{falls } H\left(\langle D, \langle D\rangle\rangle\right) = 1, \text{ d.h. D gibt bein Eingabe } \langle D\rangle \, 1 \text{ aus} \end{cases}$$

Eine solche Maschine D gibt es nicht, also kann es auch die Maschine H nicht geben! ■

SATZ 6.5.2.3

$$\text{REC} \subsetneq \text{RE}$$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

$A_{TM} \in$ RE
konstruiere TM, die $\chi_{A_{TM}}$ berechnet
Eingabe $\langle H \rangle, w$
  simuliere M auf Eingabe w
  falls M akzeptiert, gebe 1 aus

Die $A_{TM} \notin$ REC folgt $A_{TM} \in$ RE $\setminus$ REC

SATZ 6.5.2.4

RE ist nicht abgeschlossen bezüglich Komplement

Beweis

Annahme RE ist abgeschlossen bezüglich Komplement,
dann ist wegen Satz 6.3.1 RE = REC ↯

SATZ 6.5.2.5

$$\bar{A}_{TM} \notin \text{RE}$$

Beweis

$A_{TM} \in$ RE
Falls auch $\bar{A}_{TM} \in$ RE folgt $A_{TM} \in$ REC ↯

6.6 Reduzierbarkeit

Methode, um zu zeigen, dass ein Problem nicht entscheidbar ist.

Strategie:

- wissen $A \notin$ REC
- Annahme $B \in$ REC
- zeigen, dass A mit Hilfe von B gelöst werden kann
- es folgt $A \in$ REC
- das ist ein Widerspruch zur Annahme, also folgt $B \notin$ REC

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beispiel

a) $E_{TM} = \{\langle M \rangle \,|\text{M ist TM und } L(M) = \emptyset\}$

Behauptung: $E_{TM} \notin$ REC

Beweis

Annahme: $E_{TM} \in$ REC
Sei R eine TM, die E_{TM} entscheidet. Wir konstruieren eine TM S, die A_{TM} entscheidet.
zunächst: TM M wird zu M_w modifiziert
M_w:

Eingabe: $x \in \Sigma^*$
  if $x \neq w$ **then**
    gebe 0 aus
  if $x = w$ **then**
    simuliere M auf Input w
    falls $M(w)$ 1 ausgibt, gebe 1 aus

Diese Maschine hängt von w ab.
S:

Eingabe: $\langle M, w \rangle$
  konstruiere aus M und w TM M_w
  starte R auf Input $\langle M_w \rangle$
  if R 1 ausgibt **then**
    gebe 1 aus
  if R 0 ausgibt **then**
    gebe 0 aus

Wenn M_w überhaupt einmal 1 ausgibt, dann nur für das Wort w
$L(M_w) = \emptyset \Leftrightarrow M$ akzeptiert w nicht
$\langle M_w \rangle \in E_{TM} \Leftrightarrow \langle M, w \rangle \notin A_{TM}$
Wenn R E_{TM} entscheidet, dann entscheidet S A_{TM}

b) $EQ_{TM} = \{\langle M_1, M_2 \rangle | M_1 \text{ und } M_2 \text{ sind TM und } L(M_1) = L(M_2)\}$

Behauptung: $EQ_{TM} \notin$ REC

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

Annahme: $EQ_{TM} \in$ REC
Sei R TM, die EQ_{TM} entscheidet. Wir konstruieren TM S die E_{TM} entscheidet.
M_1 ... TM, die immer 0 ausgibt
S:

Eingabe: $\langle M \rangle$
simuliere R auf $\langle M, M_1 \rangle$
if R 1 ausgibt **then**
gebe 1 aus
if R 0 ausgibt **then**
gebe 0 aus

Falls die Maschine R EQ_{TM} entscheidet, entscheidet S E_{TM}

Definition 6.6.1

Eine Sprache $A \subseteq \Sigma^*$ ist reduzierbar auf eine Sprache $B \subseteq \Delta^*$, falls es eine totale und berechenbare Funktion $f : \Sigma^* \to \Delta^*$ gibt, so dass für alle $w \in \Sigma^*$ gilt $w \in A \Leftrightarrow f(w) \in B$.
Bezeichnung: $A \leq B$

Bemerkung:

- um zu testen, ob $w \in A$ gilt, kann man f(w) bestimmen
- dann testen ob $f(w) \in B$
- man erhält eine Antwort für den Test $w \in A$

Beispiel

b) $E_{TM} \leq EQ_{TM}$
$\langle M \rangle \xrightarrow{f} \langle M, M_1 \rangle$
$\langle M \rangle \in E_{TM} \Leftrightarrow \langle M, M_1 \rangle \in EQ_{TM}$

a) $\overline{A_{TM}} \leq E_{TM}$
$\langle M, w \rangle \rightsquigarrow \langle M_w \rangle$
$\langle M, w \rangle \notin A_{TM} \Leftrightarrow \langle M_w \rangle \in E_{TM}$
$\langle M, w \rangle \in \overline{A_{TM}} \Leftrightarrow \langle M_w \rangle \in E_{TM}$

SATZ 6.6.2

Aus $A \leq B$ und $B \in$ REC folgt $A \in$ REC

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

Es sei M eine entscheidende Maschine für B und f die Reduktionsfunktion von A auf B. Folgende Maschine N entscheidet A.
N:

Eingabe: w
berechne $f(w)$
simuliere M auf Input $f(w)$
gebe die Ausgabe von m aus

Folgerung

Falls $A \leq B$ und $A \notin$ REC folgt $B \notin$ REC

SATZ 6.6.3

Falls $A \leq B$ und $B \in$ RE folgt $A \in$ RE

Beweis

Es sei M eine semi-entscheidende TM für B und f die Produktionsfunktion von A zu B. Folgende Maschine N semi-entscheidet A.
N:

Eingabe: w
berechne $f(w)$
simuliere M auf Input $f(w)$
falls M terminiert gebe 1 aus

1.Fall $w \in A \overset{A \leq B}{\Longrightarrow} f(w) \in B \rightarrow$ Simulation von M auf $f(w)$ terminiert $\rightarrow$ N gibt 1 aus

2.Fall $w \notin A \overset{A \leq B}{\Longrightarrow} f(w) \notin B \rightarrow M$ auf $f(w)$ terminiert nicht $\rightarrow N$ terminiert nicht $\hookrightarrow$ N semi-entscheidet A

Folgerung

$A \leq B$ und $A \notin$ RE folgt $B \notin$ RE

SATZ 6.6.4

$E_{TM} \notin$ RE und $EQ_{TM} \notin$ RE

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

Beispiel a) von Reduzierbarkeit $\overline{A_{TM}} \leq E_{TM}$
Da $\overline{A_{TM}} \notin$ RE, folgt $E_{TM} \notin$ RE
Beispiel b) von Reduzierbarkeit $E_{TM} \leq EQ_{TM}$
$\hookrightarrow EQ_{TM} \notin$ RE

SATZ 6.6.5 (RICE)

Es sei P eine Menge von Kodierungen von Turingmaschinen, die beiden folgenden Bedingungen erfüllt

a) Für Turingmaschinen M_1 und M_2 mit $L(M_1) = L(M_2)$ gilt $\langle M_1 \rangle \in P \Leftrightarrow \langle M_2 \rangle \in P$

b) Es gibt Turingmaschinen M_1 und M_2 mit $\langle M_1 \rangle \in P$ und $\langle M_2 \rangle \notin P$, dann gilt $P \notin$ REC

Bemerkung:

zu a) Die Zugehörigkeit von $\langle M \rangle$ zu P hängt nur von der von M akzeptierten Sprache ab

zu b) P enthält Turingmaschine, aber nicht alle

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

Wir reduzieren A_{TM} bzw. $\overline{A_{TM}}$ auf P.
Es sei R die Menge aller Kodierungen von Turingmaschinen.
Es sei Ω die sie haltende Maschine.
Es ist entweder $\langle\Omega\rangle \in P$ oder $\langle\Omega\rangle \notin P$

1. Fall $\langle\Omega\rangle \in P$
   wähle Q mit $\langle Q\rangle \in R \setminus P$
   für $\langle M, w\rangle$ sei $f(M, W)$ folgende TM M_w
   M_w:
   Eingabe: $y \in \Sigma^*$
   1. simuliere M auf Eingabe w
      falls M nicht akzeptiert, terminiere nicht
   2. falls M die Eingabe w akzeptiert
      simuliere Q auf Eingabe y

   Falls M w nicht akzeptiert hält M_w auf keiner Eingabe y
   Falls M akzeptiert, verhält sich M_w auf jeder Eingabe wie die Maschine Q auf Eingabe y
   $$M_w(y) = \begin{cases} \Omega(y) = n.d. & \text{falls } M_w \text{ nicht akzeptiert, d.h. } \langle M, w\rangle \notin A_{TM} \\ Q(y) & \text{sonst} \end{cases}$$
   $\langle M, w\rangle \rightsquigarrow \langle M, w\rangle$
   $M, w \notin A_{TM} \Leftrightarrow \langle M, w\rangle \in \overline{A_{TM}} \Leftrightarrow M_w = \Omega \Leftrightarrow f(M, w) = M_w \in P$
   Da f berechenbar ist (Algorithmus kann aus M und M_w bauen) folgt $\overline{A_{TM}} \leq P$ und somit $P \notin$ REC

2. Fall $\langle\Omega\rangle \notin P$
   wähle Q mit $\langle Q\rangle \in P$
   $M_w = f(M, w)$ wie oben
   Nun gilt: $\langle M, w\rangle \in A_{TM} \Leftrightarrow M_w = Q \Leftrightarrow f(M, w) = M_w \in P$
   $\hookrightarrow A_{TM} \leq P$ und somit $P \notin$ REC

FAZIT: Es ist hoffnungslos irgendein Aspekt des funktionalen Verhalten einer Turingmaschine algorithmisch zu bestimmen.

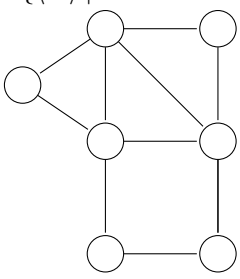
studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

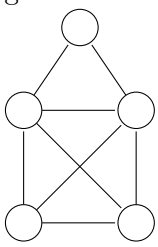
7 NP-Vollständigkeit

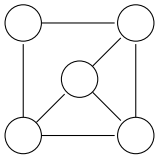
- bisher ging es um Machbarkeit, jetzt soll es schnell gehen
- betrachten ab jetzt nur noch entscheidbare Probleme

Wiederholung:
- Entscheidungsprobleme (Ja/Nein: Problem)
- können als Mengen dargestellt werden.

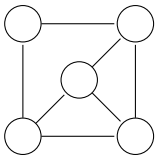
Eulerkreis
gegeben: Graph $G = (V, E)$
Frage: Besitzt G einen Eulerkreis?
(Gibt es einen Pfad wie G der jede Kante genau einmal benutzt und wieder am Startpunkt endet)
$EC = \{\langle G\rangle \,|\text{G hat einen Eulerkreis}\}$ „gehört G zur Menge EC?“

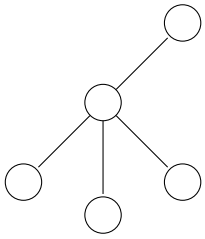

Eulerkreis


Eulerpfad
keine Eulerkreise


kein Eulerkreis

Hamiltonkreis
gegeben: Graph $G = (V, E)$
Frage: Besitzt G einen Hamiltonkreis?
Permutation π der Knotenindizes $\left(V_{\pi(1)}, V_{\pi(2)}, \ldots, V_{\pi(n)}\right)$ sodass für alle $i = 1, \ldots, n-1$ $\left\{V_{\pi(i)}, V_{\pi(i+1)}\right\}$ und $\left\{V_{\pi(n)}, V_{\pi(1)}\right\} \in E$
Pfad der jeden Knoten genau einmal besucht und zum Startknoten zurückkehrt.


Hamiltonkreis

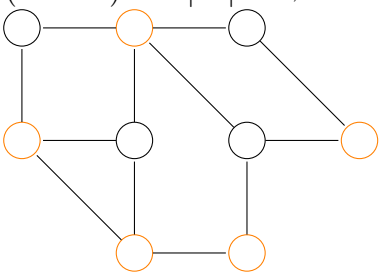

kein Hamiltonkreis

$HC = \{\langle G\rangle|\text{G hat Hamiltonkreis}\}$

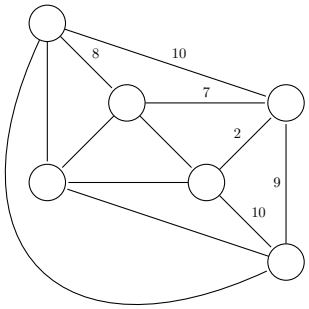
CLIQUE
gegeben: $G = (V, E), k \in \mathbb{N}$
Frage: Besitzt G eine Clique der Größe mindestens k?
$\left(V' \subseteq V\right)$ mit $\left|V'\right| \geq k$ und für alle $u, v \in V'$ mit $u + v$ gilt $\{u, v\} \in E$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

CLIQUE= $\{\langle G,k\rangle|$G ist Graph und besitzt eine Clique der Größe mindestens k$\}$
INDEPENDENTSET= $\{\langle G,k\rangle|$G ist eine Graph der eine unabhängig Menge der Größe k besitzt$\}$
$\left(V' \subseteq V\right)$ mit $\left|V'\right| \geq k$ und für alle $u,v \in V'$ gilt $\{u,v\} \notin E$
VERTEXCOVER= $\{\langle G,k\rangle|$G ist eine Graph der ein Vertexcover der Größe k besitzt$\}$
$\left(V' \subseteq V\right)$ mit $\left|V'\right| \leq k$, sodass für alle $\{u,v\} \in E$ gilt $u \in V'$ oder $v \in V'$



Traveling Salesperson
gegeben: $m \times n$ Matrix $M_{i,j}$ von Entfernungen zwischen n „Städten“, $k \leq \mathbb{N}$
Frage: Gibt es eine Permutation (eine Rundreise), sodass $\sum_{i=1}^{n-1} M_{\pi(i),\pi(i+1)} + M_{\pi(n),\pi(1)} \leq k$



7.1 Die Klasse P

Wie misst man Zeit?

Definition 7.1.1

Es sei M eine deterministische TM, die bei jeder Eingabe hält
(d.h. sie berechnet eine total definierte Funktion)

- $time_M : \Sigma^* \to \mathbb{N}$
  $time_M(w)$ = Zahl der Takte (Konfiguration über g) bis M auf Eingabe w hält
- $Time_M : \mathbb{N} \to \mathbb{N}$
  $Time_M(n) = \max\{time_M(w)|\ |w| = n\}$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Sei t: $\mathbb{N} \to \mathbb{R}$ eine Funktion
Die Komplexitätsklasse $DTIME(t(n))$ ist

$$\left\{ A \subseteq \Sigma^* \middle| \bigvee_{DTMM} \left(\text{M berechnet } c_A \text{ und } \bigwedge_n Time_M(n) \leq t(n) \right) \right\}$$

Definition 7.1.2

Sei Pol die Menge der Polynome der Form $p(n) = a_k n^k + \ldots a_1 n + a_0$ mit $n, a \in \mathbb{N}$ für alle $0 \leq i \leq k$. Die Komplexitätsklasse P (deterministische Polynomialzeit) ist definiert als $P = \bigcup_{p \in Pol} DTIME(p(n))$

- P ist die Menge aller Probleme, die sich in Polynomialzeit lösen (entscheiden) lassen
- P ist die Klasse aller effizient lösbaren Probleme
- P ist unabhängig vom zugrunde gelegten Maschinenmodell (hier TM)
  (die Überführung der Berechnungsmodelle ist in polynomellen Zeitaufwand möglich)

Beispiel Eulerkreis

naiver Algorithmus: prüfe jede Permutation der m Kanten, ob sie einen Eulerkreis darstellt
$t(m) \geq m! \geq \left(\frac{m}{2}\right)^{\left(\frac{m}{2}\right)} \notin Pol$
aber: G hat Eulerkreis gdw. G ist zusammenhängend und jeder Knoten hat gerade Valenz
$\hookrightarrow EC = \{\langle G \rangle | \text{G ist eine Graph mit Eulerkreis}\} \in P$

Beispiel Hamiltonkreis

naiver Algorithmus: prüfe jede Permutation der n Knoten, ob sie einen Hamiltonkreis bildet
Geht es besser?

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

7.2 Die Klasse NP

Definition 7.2.1

Rechenzeit einer nichtdeterministischen TM
Sei N eine NTM mit $L(N) \leq \Sigma^*$
$time_N : \Sigma^* \to \mathbb{N}$

$$time_N(w) = \begin{cases} \min\{\text{Zahl der Konfigurationsübergange}\} & w \in L(N) \\ n.d. & \text{sonst} \end{cases}$$

$Time_N : \mathbb{N} \to \mathbb{N}$
$Time_N(n) = \max\{time_N(w) | \, |w| = n \text{ und } w \in L(n)\}$
$t : \mathbb{N} \to \mathbb{R}$ Die Komplexitätsklasse $NTIME(t(n))$

$$= \left\{ A \subseteq \Sigma^* \middle| \bigvee_{NTMN} L(N) = A \text{ und } \bigwedge_n Time_N(n) \leq t(n) \right\}$$

Bemerkung:

- Eine Eingabe wird in Zeit t akzeptiert, wenn der kleinste akzeptiert Pfad höchstens t lang ist
- Es kann unendlich lange nicht akzeptierende Pfade geben

Definition 7.2.2

Die Komplexitätsklasse NP (nichtdeterministische Polynomialzeit) ist gerade $NP = \bigcup_{p \in Pol} NTIME(p(n))$

Wir sehen nichtdeterministische Algorithmen für unsere eingangs genannten Problem aus?
Das Raten der Lösung und ihr anschließendes Verifizieren ist ein nichtdeterministischer Algorithmus!

Beispiel Hamiltonkreis

- raten einer Knotenpermutation
- testen ob diese Permutation eines Hamiltonkreis beschreibt

Das ist in Polynomialzeit möglich

CLIQUE

- raten einer Menge von K Knoten
- testen, ob es eine Clique ist

INDSET (INDEPENDENTSET), VERTEXCOVER
Alle diese Probleme gehören zur Klasse NP. Noch niemand hat einen P Algorithmus für diese Probleme gefunden.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

- NP enthält sehr viele interessante und schwer lösbare praktische Problem
- $P \subseteq NP$ klar
- $P \neq NP$? Eine der großen offenen Fragen der Mathematik

7.3 NP-Vollständigkeit

Definition 7.3.1

Eine Sprache $A \subseteq \Sigma^*$ ist in Polynomialzeit reduzierbar auf eine Sprache $B \subseteq \Delta^*$, falls eine totale definierte und in polynomialer Zeit berechenbar Funktion $f : \Sigma^* \to \Delta^*$ gibt, sodass für alle $x \in \Sigma^*$ gilt $x \in A \Leftrightarrow f(x) \in B$
Bezeichnung: $A \leq_p B$

Definition 7.3.2

Eine Sprache A heißt NP schwer, falls für alle Sprachen $L \in NP$ gilt: $L \leq_p A$
Eine Sprache A heißt NP-Vollständig, falls A NP schwer ist und $A \in NP$ gilt

SATZ 7.3.3

Falls $A \leq_p B$ und $B \in P$, gilt $A \in P$

Beweis

Sei T_B TM, die B in Polynomialzeit entscheidet
Sei f die Reduktionsfunktion und T_f TM die f in Polynomialzeit berechnet.
Wir konstruieren TM T, die A in Polynomialzeit entscheidet.
T:
Eingabe: $w \in \Sigma^*$
berechne $f(w)$ (T_f auf w ansetzen)
simuliere T_B auf $f(w)$
gebe Ausgabe von T_B aus

SATZ 7.3.4

Falls $A \leq_p B$ und $B \in NP$ gilt folgt $A \in NP$

Beweis

analog mit T_B ist nichtdeterministische Maschine und damit T auch.

SATZ 7.3.5

Sei A NP-Vollständig. Dann gilt $A \in P \Leftrightarrow P = NP$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

„ $\Leftarrow$ “ Wenn A NP-vollständig ist, gilt $A \in NP$ und damit wegen $P = NP$ auch $A \in P$
„ $\Rightarrow$ “ Sei $A \in P$ und L eine beliebige Sprache aus NP. Da A NP-vollständig ist, gilt $L \leq_p A$. Damit gilt auch $L \leq P_f$, da L beliebige gewählt war, folgt $P = NP$

Bemerkung:

- falls man für eine einziges NP-vollständiges Problem zeigen kann, dass es in P liegt, gilt $P = NP$
- falls man für eine einziges NP-vollständiges Probleme zeigen kann, dass es nicht in P liegt, gilt $P \neq NP$

SATZ 7.3.6

Falls A NP-vollständig und $A \leq_p B$ folgt, B ist NP schwer

Beweis

Für alle $L \in NP$ gilt $L \leq_p A$ und wegen $A \leq_p B$ auch $L \leq_p B$
Damit ist B NP-schwer

SAT ... Erfüllbarkeitsproblem der Aussagenlogik
gegeben: eine Formel F der Aussagenlogik
Frage: Ist F erfüllbar, d.h. gibt es eine Belegung β der Variablen, sodass $I_\beta(F) = 1$
SAT $= \{\langle F \rangle | \text{F ist erfüllbare Formel der Aussagenlogik}\}$
$(((A \to B) \wedge (B \vee C)) \leftrightarrow ((A \wedge B) \leftrightarrow C)) \to A$

SATZ 7.3.7 Cook-Levin

SAT is NP-vollständig

ohne Beweis: SAT $\in NP$ ... Belegung raten und $I_\beta(F)$ bestimmt
zu zeigen: für alle $A \in NP$ gilt $A \leq_p$ SAT
Idee: Arbeitsweise einer nichtdeterministische Turingmaschine durch eine Formel beschreiben

$$3SAT = \left\{ \langle F \rangle \middle| \begin{array}{l} \text{F ist erfüllbare Formel in konjunktiver Normalform} \\ \text{mit höchstens 3 literalen pro Klausel} \end{array} \right\}$$

Man kann zeigen: SAT $\leq_p 3SAT$ mit $3SAT \leq NP$ folgt 3SAT ist NP vollständig

SATZ 7.3.8

INDSET ist NP-vollständig

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Beweis

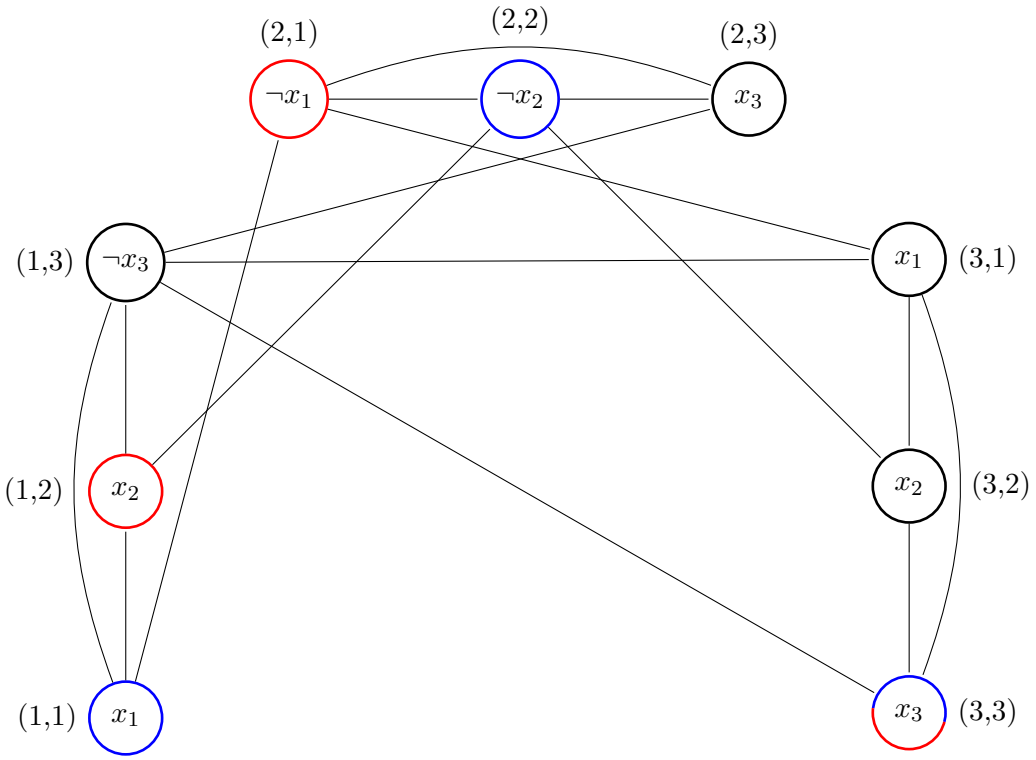
Wir zeigen $3SAT \leq_p$ INDSET

- benötigen eines Algorithmus, der einen (beliebige) Input für 3SAT in einen Input für INDSET überführt
- zu beliebiger Formel F in 3KNF muss $G = (V, E), k \in \mathbb{N}$ konstruiert werden, sodass F erfüllbar ist gdw. G eine unabhängige Menge der Größe k hat.

m ... Anzahl der Klauseln in F
n ... Anzahl der Variablen

$V = \{(1,1), (1,2), (1,3), \ldots, (m,1), (m,2), (m,3)\}$
$E = \{((i,j), (p,q)) | i = p \text{ oder } z_{i,j} = z_{p,q}\}$

$k = m$
$(x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3)$



$x_1 = 1, x_2 = 0, x_3 = 1$ ist erfüllende Belegung
$\rightarrow (1,1)\,(2,2)\,(3,3)$ sind paarweise nicht verbunden
• • • paarweise nicht verbunden $\rightarrow x_1 = 0, x_2 = 1, x_3 = 1$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

$\langle F \rangle \in SAT$
$\Leftrightarrow$ F ist erfüllbar durch Belegung β
$\Leftrightarrow$ es gibt in jeder Klausel ein Literal, dass unter β den Wert 1 enthält
z.B. $z_1, j_1, z_2 j_2, z_3 j_3, \ldots, z_m j_m$
$\Leftrightarrow$ es gibt Literale $z_{1,j_1}, z_{2,j_2}, \ldots, z_{m,j_m}$ die paarweise nicht verbunden sind
$\Leftrightarrow$ G hat eine unabhängige Menge der Größe K
$\Leftrightarrow \langle G, K \rangle \in$ INDSET
$F \to (G, K)$ ist in Polynomialzeit berechenbar $\square$

BOXCOVER
gegeben: n Punkte in der Ebene, $k \in \mathbb{N}$
Frage: können die Punkte durch k achsenparallele Quadrate mit Seitenlänge 1 überdeckt werden

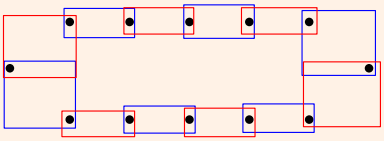
SATZ 7.3.9

BOXCOVER ist NP-schwer

Beweis

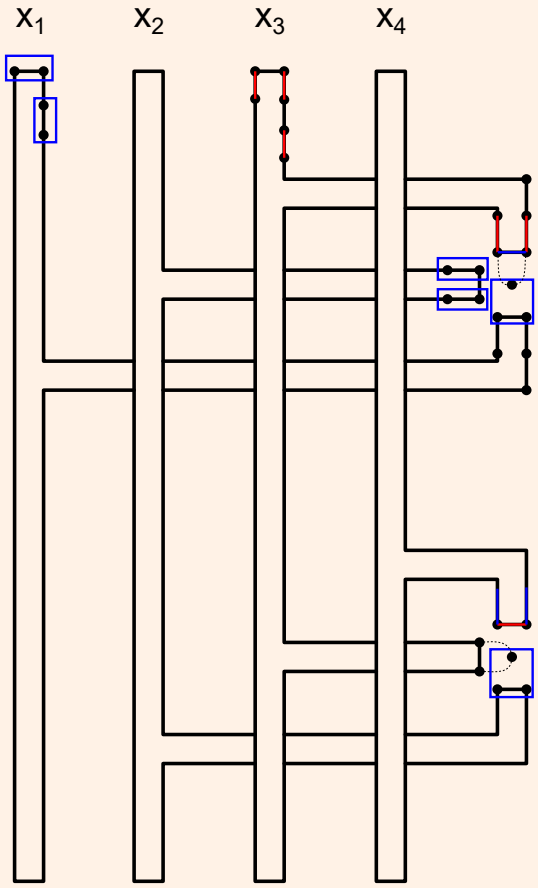
3SAT $\leq_P$ BOXCOVER
konstruieren zu beliebiger konkreter Eingabe I für 3SAT eine konkrete Eingabe für BOXCOVER, sodass die k Boxen ausreichen gdw. I erfüllbar ist.

(1) Jede boolesche Variable wird durch eine Punktschleife simuliert die auf genau 2 verschiedene Weisen mit einer minimalen Anzahl an Quadraten überdeckt werden kann. Eine Überdeckung entspricht TRUE, die andere FALSE



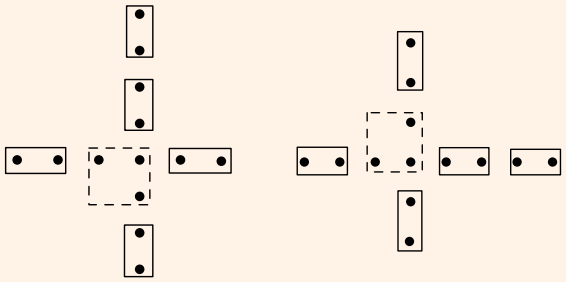
studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

(2) Gesamte Konstruktion



Schleife für Variable i benötigt k_i Boxen

(3) Kreuzkomponenten



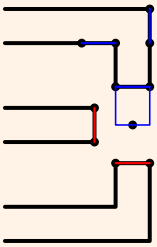
- pro Kreuzungspunkt kann eine BOX gespart werden
- Schleifen beeinflussen sich nicht

N_c ... Anzahl der Kreuzprodukte

$$K = \sum_{i=1}^{n} K_i - N_c$$

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

(4) Klauselkomponente
$x_1 \vee \neg x_2 \vee \neg x_3$



Schleife für x_1 deckt im Falle der blauen (TRUE) Belegung den zusätzlichen Punkt ab. Schleifen für x_2 und x_3 im Falle der FALSE-Belegung.
Für jede Klausel werden die entsprechenden Variablenschleifen so zusammengefügt, dass für die positiven Literale die blaue (TRUE) Überdeckung den zusätzlichen Punkt überdeckt und für die negativen die rote (FALSE)

Lemma 7.3.10

Sei I eine Eingabe für 3SAT und P(I) die beschriebene Punktkonstruktion bestehend aus n Schleifen, N_c Kreuzungen und m Klausel Komponenten. Falls die einzelnen Schleifen jeweils mit K_i Boxen überdeckt werden können, kann P(I) genau dann mit $K = \sum_{i=1}^{n} K_i - N_c$ Boxen überdeckt werden, wenn I erfüllbar ist.

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by

Abkürzungen

CS	Klasse der kontextsensitiven Sprachen
CF	Klasse der kontextfreien Sprachen
CF_{det}	Menge der deterministisch kontextfeiern Sprachen
REG	Klasse der regulären Sprachen
DFA	deterministischer endlicher Automat
NFA	nichtdeterministischer endlicher Automat
PDA	nichtdeterministischer Kellerautomat
DPDA	deterministischer Kellerautomat
TM	Turingmaschine
DTM	deterministische Turingmaschine
NTM	nichtdeterministische Turingmaschine
LBA	linear beschränkte Turingmaschine
REC	Menge aller entscheidbaren Mengen
RE	Menge aller rekursiven aufzählbar Mengen
INDSET	INDEPENDENTSET
SAT	Erfüllbarkeitsproblem der Aussagenlogik

studocu
messages.pdf_cover_qr_code_label
messages.downloaded_by